

The System Design Interview Guide

High-Level Design (HLD) + Low-Level Design (LLD) for Product-Based Company Interviews

Extended Edition — with Code Walkthroughs, Expanded LLD Coding-Round Patterns, and Deep-Dive Microservices

All code in Java. This edition adds line-by-line code explanations to every LLD/design-pattern question, plus new sections: A5 (More LLD Coding-Round Patterns) and A6 (Microservices Deep Dive).

How to Use This Guide

This is Part 1 of a larger interview prep series — this volume covers System Design end to end (HLD and LLD). Companion volumes covering Node.js/Express, TypeScript, Angular, GraphQL, Kafka, and PostgreSQL follow separately. System design is the single highest-leverage topic to prepare deeply: it's weighted heavily starting around the 2–3 YOE mark at most product-based companies (Amazon, Google, Microsoft, Meta, Netflix, Uber, Atlassian, Flipkart, Swiggy, Razorpay, and similar), and its importance only grows as you interview for more senior roles.

Structure:

- **A1 — HLD Fundamentals:** the vocabulary and building blocks you must know cold — scalability, load balancing, caching, CDN, database internals (sharding, replication, CAP theorem, indexing), message queues, API design, distributed systems concepts, storage systems, microservices patterns, and reliability/monitoring concepts.
- **A2 — HLD Case Studies:** full “design X” interview walkthroughs — URL shortener, rate limiter, distributed key-value store, unique ID generator, notification system, chat/messaging, news feed, video streaming, ride-sharing, search autocomplete, web crawler, distributed file storage, payment processing, ticket booking, logging/monitoring, job scheduler, RAG/AI search, e-commerce/inventory, distributed locks, leaderboards, and API gateways.
- **A3 — LLD Fundamentals:** OOP principles, all 5 SOLID principles with violation/fix examples and code, and the most commonly asked creational, structural, and behavioral design patterns with code, plus UML basics.

- **A4 — LLD Case Studies:** full machine-coding walkthroughs with real code — parking lot, elevator system, tic-tac-toe/game framework, library management, expense-sharing (Splitwise), LRU cache, rate limiter, vending machine, chess, and food delivery order assignment.
- **A5 — More LLD Coding-Round Patterns (new):** additional standalone machine-coding problems that show up frequently in product-based-company LLD rounds — a thread-safe bounded blocking queue, a snake-and-ladder game, a file system in memory, a notification/observer-based logger, a movie ticket booking seat-lock system, and a consistent-hashing ring.
- **A6 — Microservices Deep Dive (new):** an expanded, code-backed treatment of microservices patterns — API composition, distributed transactions with Sagas (orchestration code), circuit breakers, outbox pattern, service mesh, contract testing, and more — going well beyond the fundamentals covered in A1.

Each question is answered at true interview depth — long enough to actually explain the concept and the trade-offs an interviewer wants to hear, short enough to review the night before an interview. Code snippets are in Java, and every code block in the LLD and microservices sections is followed by a **“Walking through the code”** explanation that goes line by line (or block by block) through what’s happening and why — not just what the pattern is, but how the code actually implements it.

How to prepare with this:

1. **First pass** — read A1 and A3 (fundamentals) fully to build vocabulary. Don’t skip this to jump straight to case studies; the case studies constantly reference these building blocks.
2. **Second pass** — work through the case studies (A2, A4, A5) out loud, on a whiteboard or in a doc. Try to derive the answer yourself first, then check against the guide — don’t just read passively. For the code blocks, close the guide and try to write the class skeleton from the entity list alone before reading the implementation.
3. **Third pass** — redo the case studies from memory, timing yourself to 35–40 minutes for HLD and 30–40 minutes for LLD, matching real interview pacing.
4. **Notice the recurring patterns** — Strategy, State, atomic conditional updates for race conditions, CQRS-style read/write separation, idempotency — these same handful of ideas resurface across nearly every case study in this guide (see Q228 for why that’s the actual point).

A note on depth vs. breadth: No guide substitutes for building things and reading real post-mortems (the engineering blogs of Uber, Netflix, Discord, Figma, and Stripe are excellent free sources of real trade-off discussions). Treat this as your structured review and reference, not your only source.

PART A — SYSTEM DESIGN

A1. High-Level Design (HLD) — Fundamentals & Building Blocks

Scalability Basics

Q1. What is the difference between vertical scaling and horizontal scaling?

Vertical scaling (scaling up) means adding more resources — CPU, RAM, disk — to a single machine. It's simple (no code changes, no distributed-systems complexity) but has a hard ceiling (the biggest machine you can buy) and a single point of failure. Horizontal scaling (scaling out) means adding more machines and distributing load across them. It has near-unlimited ceiling and better fault tolerance, but introduces complexity: you now need load balancing, data partitioning, network calls between nodes, and you have to handle partial failures. Most production systems at scale use horizontal scaling for stateless tiers (web/app servers) and a mix of vertical + horizontal (sharding, read replicas) for the data tier.

Q2. What is throughput vs. latency, and why do they sometimes trade off against each other? Latency is the time to complete one unit of work (e.g., 50ms per request). Throughput is the number of units of work completed per unit time (e.g., 10,000 requests/ sec). They aren't the same thing — a system can have high throughput with high latency (think batch processing: huge volume, but each job takes minutes) or low latency with low throughput (a single fast machine serving few clients). The trade-off shows up with batching: batching requests (e.g., grouping DB writes) increases throughput because of amortized overhead, but increases latency for any individual request that has to wait for the batch to fill.

Q3. What does “P99 latency” mean and why do engineers care about it more than average latency? P99 latency is the latency value below which 99% of requests complete — i.e., the worst latency experienced by the “unlucky” 1% is right at or below this number. Averages hide tail behavior: if 99% of your requests take 10ms but 1% take 5 seconds, your average might look fine (~60ms) while a meaningful fraction of real users have a terrible experience. At scale, “the unlucky 1%” is a lot of real users — 1% of 10M daily requests is 100,000 bad experiences. High-percentile latency (P95, P99, P999) is what SRE teams actually set SLOs against.

Q4. What is capacity estimation and why is it done first in a system design interview? Capacity estimation (or “back-of-envelope” estimation) means computing rough numbers — QPS (queries per second), storage growth, bandwidth — before designing, so your architecture is grounded in the actual scale required instead of over- or under- engineered. Example: for a system with 100M daily active users averaging 5

requests each, $\text{QPS} \approx (100\text{M} \times 5) / 86,400 \approx 5,800$ QPS average, and you'd typically assume peak traffic is 2-3x average, so design for $\sim 15,000$ QPS. Interviewers use this step to see if you can reason quantitatively and to calibrate whether your later design (e.g., single Postgres instance vs. sharded cluster) actually matches the numbers.

Q5. How do you estimate storage requirements for a system like a photo-sharing app? Multiply (data generated per user action) $\times$ (frequency) $\times$ (retention period), then add overhead. Example: 50M daily active users, 2 photos/day average, 2MB/photo average = $50\text{M} \times 2 \times 2\text{MB} = 200\text{TB/day}$ of raw photo data. Over 5 years that's roughly $365\text{TB} \times 5 \approx 365\text{PB}$ (ignoring growth), which immediately tells you: this needs to go into object storage (like S3) with CDN in front, not a single filesystem, and probably needs a tiered storage strategy (hot/warm/cold) given the retention cost.

Load Balancing

Q6. What is a load balancer and where does it sit in the architecture? A load balancer distributes incoming traffic across multiple backend servers so no single server is overwhelmed, and so the system can survive individual server failures. It typically sits between clients (or the CDN/edge layer) and the application servers. Load balancers can operate at Layer 4 (transport layer — routes based on IP/port, faster, protocol-agnostic) or Layer 7 (application layer — routes based on HTTP headers, URL path, cookies; enables smarter routing like A/B testing or API versioning, but adds overhead).

Q7. Name and explain 4 common load balancing algorithms. - Round robin — requests distributed sequentially across servers. Simple, but ignores server load/capacity differences. - Least connections — routes to the server with the fewest active connections. Better for long-lived or variable-duration connections. - Weighted round robin / weighted least connections — same as above but accounts for servers with different capacities (a bigger box gets proportionally more traffic). - IP hash / consistent hashing — routes based on a hash of the client IP (or another key), ensuring the same client consistently hits the same server — useful for session affinity ("sticky sessions") without a shared session store.

Q8. What is a single point of failure (SPOF), and how do you eliminate the load balancer itself as one? An SPOF is any component whose failure takes down the whole system. Ironically, a load balancer designed to add resilience becomes an SPOF if there's only one. The fix is to run load balancers in an active-passive or active-active pair behind a floating/virtual IP (using something like keepalived + VRRP), or to use a managed, inherently-redundant LB service (AWS ELB/ALB, GCP Load Balancer) where the cloud provider handles the redundancy for you.

Q9. What's the difference between DNS-based load balancing and a dedicated load balancer? DNS load balancing (round-robin DNS) returns different IPs for the same hostname to different clients/resolvers. It's cheap and works at global scale (routing users to the nearest regional data center — GeoDNS), but it has poor failure responsiveness because of DNS caching/TTL — a dead server can stay in rotation until

the TTL expires and clients re- resolve. A dedicated load balancer sits in the request path, does active health checks, and can remove an unhealthy backend within seconds. In practice, large systems use both: DNS/ GeoDNS for coarse regional routing, and L4/ L7 load balancers within each region for fine- grained, fast failover.

Q10. What is a health check and what are the two common types? A health check is a periodic probe the load balancer (or orchestrator) uses to decide if a backend instance should receive traffic. Passive health checks infer health from real traffic (e.g., a spike in 5xx errors or timeouts from a backend marks it unhealthy). Active health checks proactively hit a dedicated endpoint (like /healthz) on a timer, independent of real traffic, and can catch a broken instance even before user traffic would have revealed it. Good health check endpoints check real dependencies (DB connectivity, disk space) — not just “is the process alive,” which can mask a deeper problem.

Caching

Q11. What is caching and why is it one of the highest-leverage tools in system design? Caching stores a copy of expensive-to-compute or expensive-to-fetch data closer to where it’s needed (in memory, on an edge node, in the browser) so repeat requests are served fast without redoing the work. It’s high-leverage because most real-world traffic is extremely skewed (a small number of “hot” items get most requests — the Pareto principle), so caching even a small hot set can absorb the vast majority of load off your database or backend.

Q12. Compare the main cache eviction policies: LRU, LFU, and FIFO. - LRU (Least Recently Used) — evicts the item that hasn’t been accessed for the longest time. Good general-purpose default; assumes recently used items are likely to be used again (temporal locality). - LFU (Least Frequently Used) — evicts the item with the fewest total accesses. Better when access frequency is a more stable signal than recency, but can be slow to adapt when an item’s popularity changes (needs aging/decay to avoid “cache pollution” from historically popular but now-cold items). - FIFO (First In First Out) — evicts the oldest inserted item regardless of usage. Simple, cheap, but ignores access patterns entirely — rarely used alone in production caches.

Q13. What is a cache-aside (lazy loading) pattern, and what are its trade-offs? In cache-aside, the application checks the cache first; on a miss, it reads from the database, then writes the result into the cache before returning it. Trade-offs: simple to reason about and only caches data that’s actually requested (no wasted memory on cold data), but every cache miss incurs the full DB-read latency, and there’s a window where cache and DB can be inconsistent if not invalidated carefully on writes.

GET flow: app -> cache (miss) -> db -> app writes to cache -> return

Q14. What is write-through caching, and how does it differ from write-back (write- behind)? Write-through: every write goes to the cache and the database synchronously, together. Keeps cache and DB always consistent, but write latency is bounded by the slower of the two (usually the DB). Write-back (write-behind): writes go

to the cache first, and are asynchronously flushed to the database later (often batched). This gives very fast writes and can reduce DB load (batching), but risks data loss if the cache node crashes before the flush, and adds complexity around consistency and flush ordering.

Q15. What causes cache stampede (a.k.a. thundering herd), and how do you prevent it? Cache stampede happens when a popular cache key expires and a large number of concurrent requests all miss at once, all hammering the database simultaneously to recompute the same value — which can overload the DB even though the system was “protected” by a cache. Prevention techniques: (1) request coalescing / locking — only one request recomputes the value while others wait for it; (2) probabilistic early expiration — refresh the cache slightly before actual expiry, with jitter, so not all keys expire at exactly the same time; (3) stale-while-revalidate — serve slightly stale data while asynchronously refreshing in the background, rather than blocking on a synchronous rebuild.

Q16. What is cache invalidation and why is it considered one of the “two hard problems in computer science”? Cache invalidation is the process of removing or updating stale cached data when the underlying source changes. It’s hard because there’s an inherent trade-off: invalidate too eagerly and you lose the performance benefit of caching; invalidate too lazily (or miss an invalidation path) and clients see stale/incorrect data, which can be worse than no cache at all for correctness-sensitive data. Common strategies are TTL-based expiry (simple but imprecise), explicit invalidation on write (precise but requires tracking every write path that touches the data), and versioned/keyed cache entries (e.g., including a version number or hash in the cache key so old entries are naturally orphaned rather than needing active deletion).

Q17. Where can caching be applied across a typical request path? Name at least 5 layers. 1. Client/browser cache (HTTP cache headers, service workers) 2. CDN / edge cache (static assets, sometimes API responses) 3. Reverse proxy cache (e.g., Varnish, Nginx caching) 4. Application-level cache (in-process memory cache, or a distributed cache like Redis/Memcached) 5. Database query cache / buffer pool (Postgres `shared_buffers`, MySQL query cache) 6. CPU/OS-level caches (usually outside application control, but worth mentioning to show depth)

Q18. Redis vs. Memcached — when would you pick one over the other? Memcached is a simple, multi-threaded, in-memory key-value store — great for pure caching of simple objects, and its multi-threading can give it an edge in raw throughput for simple GET/SET workloads. Redis is a richer in-memory data structure store (strings, hashes, lists, sets, sorted sets, streams), supports persistence (RDB/AOF), replication, pub/sub, Lua scripting, and built-in data expiry — making it suitable not just as a cache but as a lightweight message broker, rate-limiter backend, leaderboard (sorted sets), or session store. In practice, Redis has become the default choice for most new systems because the extra capability rarely costs much, but Memcached is still chosen sometimes for pure large-scale caching where its multi-threaded model gives simpler horizontal scaling per node.

CDN

Q19. What is a CDN and how does it reduce latency? A Content Delivery Network is a geographically distributed set of edge servers that cache and serve content (static assets, and increasingly dynamic content) from a location physically closer to the end user, reducing round-trip network latency. Instead of every request crossing continents to a single origin server, the CDN edge node — often within the user’s own city or ISP — serves the cached copy directly. CDNs also absorb massive traffic spikes away from the origin (protecting against both load and some DDoS patterns) and typically handle TLS termination at the edge.

Q20. What’s the difference between a push CDN and a pull CDN? Push CDN: the origin proactively uploads content to CDN edge nodes ahead of time. Good for sites with a relatively small amount of content that changes infrequently (you control exactly what’s cached and when). Pull CDN: the CDN fetches content from the origin lazily, on the first request for a given resource, then caches it for subsequent requests (with a TTL). Good for large or frequently changing catalogs, since you don’t need to manage explicit uploads — but the very first request for any resource in a region is a cache miss (“cold” edge).

Databases — Core Concepts

Q21. SQL vs. NoSQL — what’s the actual decision framework, beyond “SQL has schemas and NoSQL doesn’t”? The real decision factors: (1) Data shape — highly relational data with many join patterns favors SQL; document/nested or highly variable- schema data favors document stores; simple key-value access patterns favor KV stores. (2) Consistency needs — if you need multi-row ACID transactions (e.g., financial transfers), SQL databases give this natively; most NoSQL stores trade this away for availability/partition tolerance. (3) Scale pattern — if you need to scale writes horizontally across many nodes with flexible schema evolution, many NoSQL stores (Cassandra, DynamoDB) are built for this out of the box, whereas horizontally scaling writes in SQL requires manual sharding. (4) Query flexibility — SQL’s declarative query language and joins are far more flexible for ad hoc/analytical queries; NoSQL access patterns often must be designed upfront (denormalized for your specific query needs). In interviews, the strongest answer names the actual access pattern of the system in question and derives the choice from it, rather than reciting the trade-off table abstractly. **Q22. What is database sharding, and what are the common sharding strategies?** Sharding splits a large dataset across multiple database instances (shards), each holding a subset of the data, so no single machine needs to hold or serve the entire dataset. Strategies: - Range-based sharding — split by a key range (e.g., user IDs 1-1M on shard A, 1M-2M on shard B). Simple, but can create hotspots if access is skewed toward one range (e.g., newest users are most active). - Hash-based sharding — hash the shard key and mod by shard count. Spreads load evenly, but makes range queries across shards expensive and complicates resharding (adding a shard reshuffles almost all keys — mitigated by consistent hashing). - Directory-based sharding — a

lookup service maps keys to shards explicitly. Most flexible (can rebalance without a rehashing formula), but the directory service itself becomes a critical dependency. - Geo-based sharding — shard by user region, which also helps latency and data residency/compliance needs.

Q23. What is consistent hashing and what problem does it solve? Consistent hashing maps both servers and keys onto a common hash ring (0 to $2^{32}-1$, say). A key is assigned to the first server clockwise from its hash position. When a server is added or removed, only the keys between it and its predecessor on the ring need to move — roughly K/N keys (K = total keys, N = server count) — instead of nearly all keys, which is what happens with naive $\text{hash}(\text{key}) \% N$ sharding when N changes. This makes horizontal scaling and node failure handling far cheaper. Virtual nodes (each physical server mapped to many points on the ring) are used on top of this to avoid uneven load distribution when server count is small.

Q24. What is database replication, and what's the difference between synchronous and asynchronous replication? Replication maintains copies of data across multiple database nodes for durability and read scalability. In synchronous replication, a write isn't acknowledged to the client until it's confirmed on the replica(s) too — this guarantees no data loss on primary failure, but adds write latency and can reduce availability (if a replica is slow/down, writes can stall). In asynchronous replication, the primary acknowledges the write immediately and propagates to replicas afterward — faster writes, higher availability, but a primary crash right after acknowledging a write (before replication completes) can lose that write, and reads from a replica can return stale data (replication lag).

Q25. What is the difference between leader-follower (master-slave) and leader-leader (multi-master) replication? In leader-follower, all writes go to a single leader, which propagates changes to followers; followers typically serve only reads. This is simple and avoids write conflicts, but the leader is a write bottleneck and a failover (promoting a follower) has some downtime/complexity. In leader-leader (multi-master), multiple nodes can accept writes, which improves write availability and can reduce latency (write to the nearest leader), but introduces the hard problem of conflict resolution when the same data is written differently on two leaders concurrently (resolved via techniques like last-write-wins, vector clocks, or CRDTs).

Q26. Explain the CAP theorem and why you can't have all three of Consistency, Availability, and Partition Tolerance. CAP theorem states that in a distributed system, when a network partition occurs (nodes can't communicate), you must choose between Consistency (every read gets the most recent write, or an error) and Availability (every request gets a non-error response, though possibly stale). You can't have both during a partition. Since partitions are a fact of life in any real distributed system (network links fail), the practical framing is really "CP vs. AP" — partition tolerance isn't optional, it's a given. CP systems (e.g., traditional RDBMS with synchronous replication, Zookeeper, HBase) reject requests rather than serve stale/inconsistent data. AP systems (e.g., Cassandra, DynamoDB in default config, Couchbase) keep serving requests during a partition, accepting that some responses may be stale, and reconcile afterward.

Q27. What is eventual consistency, and give a real-world example where it's an acceptable trade-off. Eventual consistency means that if no new updates are made to a piece of data, all replicas will eventually converge to the same value — but at any given moment, different replicas might return different (stale) values. It's acceptable when the cost of brief staleness is low relative to the value of availability/latency. Example: a social media "like count" — if it shows 1,047 instead of 1,050 for a few seconds after a burst of likes, no real harm is done, and the system stays fast and available. By contrast, a bank account balance is a poor candidate for eventual consistency, because a stale read could let a user double-spend funds.

Q28. What is the difference between strong consistency, eventual consistency, and causal consistency? Strong consistency — every read reflects the latest write, as if there were only one copy of the data (linearizability). Requires coordination, adding latency, and reduces availability under partition. Eventual consistency — replicas converge over time, no ordering guarantee on when. Causal consistency — a middle ground: writes that are causally related (e.g., a reply to a comment) are seen in the correct order by all nodes, but causally unrelated writes may be seen in different orders on different nodes. Causal consistency is often "good enough" for things like comment threads or chat, giving much better performance than strong consistency while avoiding the confusing "reply before the original comment" problem eventual consistency alone could produce.

Q29. What is database indexing, and what's the trade-off of adding more indexes? An index is an auxiliary data structure (commonly a B-tree or hash structure) that lets the database find rows matching a query condition without scanning the entire table, dramatically speeding up reads (SELECTs with WHERE/JOIN/ORDER BY on the indexed column). The trade-off: every index must be updated on every INSERT/UPDATE/DELETE that touches the indexed column, so more indexes mean slower writes and more storage. The general rule: index columns that are frequently filtered/joined/sorted on, and avoid over-indexing write-heavy tables.

Q30. What is a composite index, and does column order in it matter? A composite (multi-column) index is built over two or more columns together, e.g. (user_id, created_at). Column order matters a great deal: the index is only efficiently usable for queries that filter on a left-to-right prefix of the indexed columns (the "leftmost prefix rule"). An index on (user_id, created_at) speeds up WHERE user_id = ? and WHERE user_id = ? AND created_at > ? , but does not efficiently help a query that filters only on created_at without user_id .

Q31. What is database partitioning vs. sharding — are they the same thing? They're related but distinct. Partitioning is splitting a table's data into smaller pieces (partitions) that can live on the same database instance (e.g., Postgres table partitioning by date range) — it mainly helps query performance and maintenance (easier to drop/archive old partitions) without necessarily involving multiple machines. Sharding specifically means distributing partitions across multiple separate database instances/machines to scale beyond what one machine can handle. Sharding is a form of horizontal partitioning combined with physical distribution.

Q32. What is a read replica, and what problem does it solve? A read replica is a copy of the database that serves read traffic, kept in sync with the primary (usually via async replication). It solves the problem of read-heavy workloads overwhelming a single database instance — you scale reads horizontally by adding replicas while writes continue to go to the single primary. Trade-off: replicas can lag behind the primary (replication lag), so reads from a replica can be stale — fine for many use cases (an activity feed), risky for others (checking your own just-placed order status, where you’d want to read from the primary or a replica guaranteed to be caught up).

Message Queues & Asynchronous Processing

Q33. What problem do message queues solve in system design? Message queues decouple producers (who create work) from consumers (who process it), letting them operate independently in time and at different rates. This gives you: load leveling/buffering (absorb traffic spikes without overwhelming downstream services), resilience (if a consumer is down, messages just wait in the queue instead of being lost or failing the producer’s request), async processing (producer doesn’t block waiting for slow work like sending an email or resizing an image), and easier horizontal scaling of consumers (add more workers to drain the queue faster).

Q34. What’s the difference between a message queue (point-to-point) and a pub/ sub system? In a message queue, each message is typically consumed by exactly one consumer (competing consumers pattern) — good for distributing discrete units of work across a worker pool. In pub/sub, a message published to a topic is delivered to all subscribers of that topic — good for broadcasting events to multiple independent downstream systems (e.g., an “order placed” event needing to trigger email, inventory update, and analytics, all independently). Kafka technically supports both patterns depending on how consumer groups are configured.

Q35. What is at-most-once, at-least-once, and exactly-once delivery, and which is hardest to achieve? At-most-once — a message is delivered zero or one times; it may be lost but is never duplicated (fire-and-forget, no retries). At-least-once — a message is delivered one or more times; it’s never lost, but consumers may see duplicates (achieved via acknowledgment + retry on failure). Exactly-once — the hardest and most expensive to achieve in a distributed system; it typically requires idempotent consumers plus transactional/deduplication mechanisms (e.g., Kafka’s idempotent producer + transactional API), since true exactly-once delivery across a network is provably impossible without some form of deduplication logic — what’s really achieved is “exactly-once processing” via at-least-once delivery + idempotency.

Q36. What is idempotency and why does it matter for distributed systems and message processing? An operation is idempotent if performing it multiple times has the same effect as performing it once. This matters because network failures are common and ambiguous — if a client doesn’t get a response, it doesn’t know if the request succeeded, failed, or succeeded but the response was lost, so retrying is the natural response. If the operation isn’t idempotent (e.g., “charge \$10”), a retry could

double-charge the user. Idempotency is achieved via techniques like idempotency keys (client generates a unique key per logical operation; server deduplicates by that key) or designing operations to be naturally idempotent (e.g., “set balance to \$50” instead of “add \$10”).

Q37. Kafka vs. RabbitMQ vs. SQS — how do you decide which to use? Kafka is a distributed log built for high-throughput event streaming, with messages retained for a configurable period (not deleted on consumption) so multiple consumer groups can independently replay/process the same stream — best for event sourcing, analytics pipelines, and very high-throughput scenarios. RabbitMQ is a traditional message broker (AMQP) with rich routing (exchanges, bindings, priority queues) and strong support for complex routing topologies and per-message acknowledgment — better fit for task queues and lower-throughput, complex-routing scenarios. SQS is a fully managed, simple queue service (AWS) — minimal ops overhead, good default for straightforward decoupling within an AWS-based architecture, though historically had fewer features than Kafka/RabbitMQ (FIFO queues now close some of this gap). The interview-quality answer is: pick based on throughput needs, whether you need replay/multiple independent consumers of the same stream (favors Kafka), routing complexity (favors RabbitMQ), or operational simplicity (favors managed SQS).

Q38. What is a dead-letter queue (DLQ) and why is it important? A DLQ is a separate queue where messages are routed after they fail processing repeatedly (exceeding a retry limit) instead of being retried forever or silently dropped. It’s important because it prevents a single malformed or “poison” message from blocking the queue indefinitely (if consumers keep failing and re-queueing it) while preserving the message for later inspection/ debugging/manual reprocessing, rather than losing it.

API Design & Communication Patterns

Q39. REST vs. GraphQL vs. gRPC — when would you choose each? REST — resource- oriented, uses standard HTTP verbs/status codes, widely understood, cacheable by default (GET), simple to reason about — the default choice for most public/external APIs. GraphQL — client specifies exactly the fields it needs in one request, solving over-fetching/under- fetching and reducing round trips for complex nested data — good for frontend-heavy apps with varied client needs (mobile vs. web needing different subsets of data) and rapidly evolving UIs, at the cost of harder caching and more complex server-side query cost control. gRPC — binary protocol (Protocol Buffers) over HTTP/2, very low latency/overhead, supports streaming natively, strongly typed contracts via .proto files — the default for internal service-to-service communication in a microservices architecture where performance matters and both ends are systems you control (not ideal for public browser-facing APIs without a proxy, since browser gRPC support is limited).

Q40. What is the difference between synchronous and asynchronous communication between services, and what are the trade-offs? Synchronous communication (e.g., a direct HTTP/gRPC call where the caller waits for a response) is

simple to reason about and gives immediate feedback/errors, but couples the availability and latency of the caller to the callee — if the downstream service is slow or down, the caller is blocked or fails too, and this can cascade (a slow service can take down its callers, which take down theirs). Asynchronous communication (via a message queue/event) decouples the two — the caller doesn't wait, and the callee processes when ready — improving resilience and the ability to absorb spikes, but adds complexity (eventual consistency, harder debugging/tracing across the async boundary, no immediate error feedback to the original caller).

Q41. What is an API Gateway and what responsibilities does it typically centralize? An API Gateway is a single entry point that sits in front of a system's backend services (especially in a microservices architecture), routing external requests to the appropriate internal service. It commonly centralizes cross-cutting concerns so individual services don't each reimplement them: authentication/authorization, rate limiting, request routing, request/response transformation, SSL termination, logging/metrics, and sometimes response caching or aggregation (calling multiple backend services and combining their results into one response for the client).

Q42. What is service discovery and why is it needed in a microservices architecture? Service discovery is the mechanism by which services find the network location (IP/port) of other services they need to call, without hardcoding addresses — necessary because in a dynamic environment (containers/instances being created, destroyed, or rescheduled by an orchestrator), IP addresses change constantly. Client-side discovery (e.g., Netflix Eureka): the client queries a registry and picks an instance itself, applying its own load balancing. Server-side discovery (e.g., Kubernetes Service + kube-proxy, AWS ALB): the client calls a stable endpoint, and a router/load balancer looks up the registry and forwards the request, keeping the client simpler.

Q43. What is a circuit breaker pattern, and how does it prevent cascading failures? A circuit breaker wraps calls to a potentially-failing downstream service and monitors failure rate. It has three states: Closed (normal — requests pass through; failures are counted) → if failures exceed a threshold, it trips to Open (requests fail immediately without even attempting the call, for a cooldown period) → after the cooldown, it moves to Half-Open (a limited number of trial requests are allowed through to test if the downstream has recovered) → success returns it to Closed, failure sends it back to Open. This prevents a struggling downstream service from being hammered with requests that will likely fail anyway (giving it room to recover) and prevents the calling service from wasting its own resources (threads, connections) waiting on calls that are likely to time out — which is exactly the mechanism behind many real cascading-failure outages.

Q44. What is rate limiting and name 4 common algorithms. Rate limiting caps how many requests a client (or the system overall) can make in a given time window, protecting against abuse, ensuring fair usage, and protecting backend capacity. Algorithms: - Fixed window counter — count requests in discrete time windows (e.g., per minute); simple, but allows a burst of up to 2x the limit right at a window boundary (end of one window + start of next). - Sliding window log — store a timestamp per

request and count requests within the trailing window; accurate, but memory-heavy at high volume. - Sliding window counter — approximates the sliding log using weighted counts from the current and previous fixed windows; good accuracy/memory trade-off, common in production. - Token bucket — a bucket holds tokens, refilled at a fixed rate; each request consumes a token, and requests are rejected/queued when the bucket is empty. Naturally allows short bursts up to the bucket size while enforcing a long-run average rate — the most commonly used algorithm in real systems (e.g., AWS API Gateway). - Leaky bucket — requests enter a queue (the bucket) and are processed (leak out) at a fixed rate, smoothing bursts into a constant output rate; good when you need to protect a downstream system that can't handle bursts at all.

Q45. Where should rate limiting be implemented — client, server, gateway, or all of them? Ideally at multiple layers for defense in depth: at the API Gateway/edge (cheapest to reject abusive traffic before it reaches your infrastructure, protects overall system capacity), and at the individual service level (protects each service's own dependencies, e.g., a service calling a rate-limited third-party API needs its own internal limiter). Client-side "rate limiting" (like debouncing UI actions) is a UX nicety, not a security control, since a malicious client can simply ignore client-side code — the enforceable limit must live server-side.

A1. HLD Fundamentals (continued)

Proxies

Q46. What is the difference between a forward proxy and a reverse proxy? A forward proxy sits in front of clients and forwards their requests to the internet, masking the client's identity from the server (e.g., a corporate proxy that all employee traffic routes through, or a VPN). A reverse proxy sits in front of servers and forwards client requests to the appropriate backend, masking the server's identity/topology from the client (e.g., Nginx in front of your app servers). Reverse proxies commonly handle load balancing, SSL termination, caching, compression, and request routing — they're the far more common concept in backend system design interviews.

Q47. What responsibilities does Nginx typically handle in a production architecture? SSL/TLS termination (decrypting HTTPS so backend servers only deal with plain HTTP internally), reverse proxying/load balancing across app server instances, serving static files directly (faster than routing through the app layer), request buffering/ compression (gzip), basic rate limiting, and sometimes caching of responses. Terminating SSL at Nginx also centralizes certificate management instead of configuring it on every app instance.

Distributed Systems Concepts

Q48. What is a distributed transaction, and why is two-phase commit (2PC) considered problematic at scale? A distributed transaction spans multiple independent services/databases and must either fully succeed or fully roll back across all of them (atomicity across nodes). Two-phase commit coordinates this: in the prepare phase, a coordinator asks all participants if they can commit; if all say yes, the commit phase tells them all to actually commit. The problem: it's a blocking protocol — if the coordinator crashes after the prepare phase but before sending commit/abort, participants are stuck holding locks indefinitely, unable to know whether to commit or roll back, which severely hurts availability. This is why most large-scale distributed systems avoid 2PC in favor of the Saga pattern or simply designing for eventual consistency.

Q49. What is the Saga pattern, and how does it replace distributed transactions in microservices? A Saga breaks a distributed transaction into a sequence of local transactions, each with a corresponding compensating transaction that undoes its effect if a later step fails. E.g., “place order” saga: reserve inventory → charge payment → schedule shipping. If “charge payment” fails, a compensating action (release inventory) runs. Two coordination styles: choreography (each service publishes events and reacts to others' events, no central coordinator — simpler for few steps, but hard to trace/reason about as steps grow) and orchestration (a central saga orchestrator explicitly calls each step and handles failures — more visible/debuggable, but the orchestrator becomes a critical component). Sagas trade strict atomicity for availability and scalability, accepting that the system passes through visibly inconsistent intermediate states.

Q50. What is the difference between orchestration and choreography in microservices, beyond the saga context? Orchestration — a central service (orchestrator) explicitly directs the interaction sequence between services, calling each one and handling the response/next step. Easier to understand and debug a single workflow, but the orchestrator can become both a bottleneck and a tightly-coupled dependency for every participating service. Choreography — services communicate via events without a central controller; each service knows what to do when it sees a relevant event and emits its own events in turn. More loosely coupled and scalable, but understanding the overall business flow requires piecing it together across many services' event handlers, and can lead to hard-to-trace cyclic dependencies as the system grows.

Q51. What are vector clocks and what problem do they solve? Vector clocks are a mechanism for tracking causality between events across distributed nodes without relying on synchronized physical clocks (which are unreliable across machines — see: clock skew). Each node keeps a vector of counters (one per node); on each event, the node increments its own counter, and when nodes communicate, they merge vectors (taking the max per position). Comparing two vector clocks tells you if one event causally happened-before another, or if they're concurrent (neither happened before

the other) — the latter signaling a potential conflict that needs resolution (e.g., in a multi-master database, to detect concurrent writes to the same key that need reconciling).

Q52. What is the difference between a quorum-based system and simple majority voting, and how is it used for consistency? A quorum system requires a minimum number of nodes (a quorum) to agree before a read or write is considered successful. With N total replicas, if you require W nodes to acknowledge a write and R nodes to agree on a read, and $W + R > N$, you're guaranteed that every read quorum overlaps with every write quorum by at least one node — meaning at least one node in any read will have seen the latest write. This lets systems like Cassandra and DynamoDB tune the consistency/availability trade-off per-operation (e.g., $W=1$ for fast, less consistent writes vs. $W=N$ for strongly consistent but slower writes) rather than picking one fixed point on the CAP spectrum system-wide.

Q53. What is gossip protocol and where is it used? Gossip protocol is a decentralized way for nodes in a distributed system to disseminate information (membership, health status, metadata) by periodically exchanging state with a few random peers, similar to how rumors spread in a social network — information eventually reaches all nodes without a central coordinator, and the protocol is resilient to individual node failures. It's used for cluster membership/failure detection (Cassandra, Consul, Kubernetes' some internals) because it scales well (each node does bounded work regardless of cluster size) and tolerates network partitions gracefully (no single point of coordination to fail).

Q54. What is the difference between horizontal and vertical partitioning of a table? Horizontal partitioning splits a table by rows — different rows live in different partitions (e.g., users A-M in partition 1, N-Z in partition 2), each partition has the same schema. This is what's usually meant by "sharding." Vertical partitioning splits a table by columns — e.g., splitting a wide users table into `user_core` (id, name, email — frequently accessed) and `user_profile_extended` (bio, preferences, rarely-accessed large fields) as separate tables/stores. Vertical partitioning helps when some columns are much larger or much less frequently accessed than others, letting hot data stay compact and cacheable.

Q55. What is leader election, and name one algorithm/protocol used for it. Leader election is the process by which distributed nodes agree on a single node to act as coordinator/primary for some function, so that duties requiring a single authority (like accepting writes) have exactly one owner at a time, avoiding split-brain. Raft is a widely used consensus algorithm that includes leader election as part of its protocol: nodes start as followers, a node that doesn't hear from a leader within a randomized timeout becomes a candidate and requests votes; whichever candidate gets a majority becomes leader for a "term." Systems like etcd, Consul, and CockroachDB use Raft internally; Zookeeper uses a similar but distinct protocol (ZAB).

Q56. What is the Byzantine Generals Problem, and how is it different from a simple crash-fault-tolerant system? The Byzantine Generals Problem describes achieving consensus among distributed nodes when some nodes may not just fail/crash, but actively behave maliciously or arbitrarily (sending conflicting information to

different peers — “Byzantine faults”). This is a strictly harder problem than crash-fault tolerance (where a failed node simply stops responding, and you just need to tolerate its silence). Byzantine fault tolerance requires more nodes to reach consensus safely (classically needing more than $3f+1$ nodes to tolerate f Byzantine nodes) and more complex protocols (PBFT and its descendants). It matters practically in blockchain/decentralized systems where nodes are run by mutually distrusting parties; it’s mostly overkill (and not used) for internal systems within a single trusted organization, where crash-fault-tolerant consensus (Raft/Paxos) is sufficient.

Storage Systems

Q57. What is object storage (like S3) and how does it differ from block storage and file storage? Block storage exposes raw storage as fixed-size blocks, addressed directly by the OS/application (like a virtual hard disk) — used for databases and VM disks where you need low-level, high-performance random access. File storage organizes data in a hierarchical directory/file structure accessed via a filesystem protocol (NFS, SMB) — good for shared file access patterns. Object storage stores data as immutable objects (blob + metadata + a unique key) accessed via a simple HTTP API (GET/PUT/DELETE), with no true directory hierarchy (paths are simulated via key naming conventions) — it scales to essentially unlimited capacity, is highly durable (S3 famously offers 11 nines), and is the default choice for unstructured data like images, videos, backups, and logs, but has higher per-operation latency than block storage and doesn’t support in-place partial-file modification.

Q58. What is a write-ahead log (WAL) and why do most databases use one? A WAL is an append-only log where every change is recorded before it’s applied to the actual data structures on disk. This ensures durability and crash recovery: if the database crashes mid-operation, on restart it can replay the WAL to reconstruct any changes that were logged but not yet fully applied to the main data files, guaranteeing no acknowledged write is lost. It also enables efficient replication (followers can replay the same log) and point-in-time recovery. Postgres, MySQL (InnoDB redo log), and Kafka (its entire storage model is essentially a WAL) all rely on this pattern.

Q59. What is a B-tree and why is it the standard data structure for database indexes? A B-tree is a self-balancing tree structure where each node can have multiple children (not just 2, like a binary tree), keeping the tree shallow even with millions of entries — meaning very few disk reads are needed to find any given key (logarithmic height, but with a much larger branching factor than a binary tree, so the practical depth for millions of rows is often just 3-4 levels). This is important because disk/SSD I/O is the dominant cost, so minimizing the number of reads (not just comparisons) matters far more than in an in-memory context. B+ trees (a common variant) additionally store all actual data in leaf nodes with leaves linked together, making range scans (`WHERE x BETWEEN a AND b`) and `ORDER BY` efficient via a simple leaf traversal.

Q60. What is an LSM tree (Log-Structured Merge tree), and why do write-heavy NoSQL databases (Cassandra, RocksDB) prefer it over B-trees? An LSM tree optimizes for write throughput by buffering writes in memory (a “memtable”) and flushing them sequentially to disk as immutable sorted files (“SSTables”) rather than updating data in place. Sequential writes are far faster than the random writes a B-tree can require (updating a B-tree node in place means seeking to a specific disk location). Background compaction periodically merges SSTables to remove duplicates/deleted entries and control file count. The trade-off: reads can be slower, since a key might need to be checked across multiple SSTables (mitigated with Bloom filters to quickly rule out files that don’t contain a key), and compaction consumes background I/O/CPU. This makes LSM trees the standard choice when write throughput matters more than raw read latency.

Q61. What is data denormalization, and when is it the right call in system design? Denormalization intentionally introduces redundancy (duplicating data across tables/ documents) to optimize read performance by avoiding expensive joins, at the cost of extra storage and more complex write paths (the same piece of data now needs to be updated in multiple places to stay consistent). It’s the right call when: reads vastly outnumber writes, the join pattern is known and stable, and read latency is critical (user-facing paths). Example: storing a denormalized `author_name` directly on a post document instead of joining to a users table on every read, accepting that if a user renames themselves, existing posts show the old name until a (possibly async) backfill updates them.

Microservices & Architecture Patterns

Q62. What are the core trade-offs of microservices vs. a monolith? Monolith: simpler to develop, test, and deploy initially (single codebase, single deploy unit), easier to reason about transactions (everything’s in one process/DB), lower operational overhead (fewer moving parts to monitor). Downsides: harder to scale specific parts independently (you scale the whole thing even if only one feature is hot), a bug in one module can crash the whole app, and it becomes harder to work on as the team/codebase grows (deploy coordination, merge conflicts, slower CI). Microservices: independent deployability and scalability per service, teams can own services end-to-end and choose their own tech stack, failure isolation (one service crashing doesn’t necessarily take down others). Downsides: massive operational complexity (service discovery, distributed tracing, network reliability, data consistency across services), higher latency for flows spanning multiple services, and harder local development/testing. The honest interview answer: most successful companies start as a monolith and extract microservices only where scaling/team boundaries genuinely demand it — “microservices from day one” is a common anti-pattern for early-stage systems.

Q63. What is the strangler fig pattern for migrating a monolith to microservices? Named after the strangler fig vine that gradually envelops and replaces a host tree, this pattern migrates a monolith incrementally rather than via a risky big-bang rewrite: new functionality (or refactored pieces of old functionality) is

built as separate services, while a routing layer (often an API Gateway/reverse proxy) gradually redirects traffic for specific routes/features from the old monolith to the new services. Over time, more and more traffic is “strangled” away from the monolith until it can be fully retired. This keeps the system always in a working, deployable state throughout the migration, versus a long-running parallel rewrite that risks never shipping or diverging from the evolving old system.

Q64. What is the database-per-service pattern in microservices, and what problem does it create that a shared database avoids? Each microservice owns and exclusively accesses its own database, with no other service allowed to query it directly (only through the owning service’s API) — this preserves service independence, letting each team choose their own schema/DB technology and evolve it without coordinating with other teams. The problem it creates: queries that need data spanning multiple services’ domains (e.g., “show orders with customer names and product details”) can no longer be a simple SQL join — they require either the calling service to make multiple API calls and join in application code, an API Gateway that aggregates, or a separate read-optimized view (via CQRS/event-driven materialized views) that’s kept in sync via events.

Q65. What is CQRS (Command Query Responsibility Segregation)? CQRS separates the model used for writes (commands — mutating state) from the model used for reads (queries), which can be entirely different data structures, and often entirely different data stores. This allows independent optimization: the write side can be normalized and optimized for consistency/validation, while the read side can be a denormalized, pre-joined, heavily-cached view optimized for the exact queries the UI needs. It pairs naturally with event sourcing (the write side emits events, and the read side is a materialized projection built by consuming those events) but doesn’t require it. The main cost is added complexity and typically eventual consistency between the write and read sides (there’s a lag while the read projection catches up).

Q66. What is event sourcing, and what are its benefits and costs? Instead of storing just the current state of an entity, event sourcing stores the full sequence of state-changing events that led to that state (e.g., not “balance: \$150” but “deposited \$100”, “deposited \$50”), and the current state is derived by replaying events. Benefits: a complete audit trail for free, the ability to reconstruct state as of any point in time, and the ability to add new read projections later by replaying history (even for logic that didn’t exist when the events were recorded). Costs: querying “current state” requires replaying events (mitigated with periodic snapshots), the event schema becomes a long-term contract that’s hard to change once events are on disk, and it’s a genuinely more complex model to reason about and debug than simple CRUD, so it’s usually reserved for domains where audit trail/history genuinely matters (financial ledgers, order management) rather than applied universally.

Q67. What is a bulkhead pattern in resilience engineering? Borrowed from ship design (where bulkheads are partitions that stop one compartment flooding from sinking the whole ship), the bulkhead pattern isolates resources (thread pools, connection pools) per dependency so that one failing/slow downstream dependency

can't exhaust shared resources and starve calls to unrelated, healthy dependencies. Example: instead of one shared thread pool for all outbound calls, give the "call payment service" and "call recommendation service" paths separate, bounded thread pools — if the recommendation service hangs, it can only exhaust its own pool, and payment calls keep working normally.

Q68. What is backpressure, and why does an unbounded system without it tend to fail badly under load? Backpressure is a mechanism by which a system signals upstream producers to slow down when it can't keep up with the current rate of incoming work, rather than silently accepting unbounded work into memory/queues. Without backpressure, a slow consumer facing a fast producer will keep buffering requests (in an in- memory queue, say), and under sustained overload this queue grows unboundedly — eventually causing out-of-memory crashes, or growing latency so extreme that requests time out anyway (having wasted resources queueing work nobody will use the result of). Backpressure mechanisms include bounded queues with rejection (fail fast once full), reactive streams (explicit demand signaling between producer/consumer), or simply rate- limiting/load-shedding at the edge.

Monitoring, Reliability & Scale Concepts

Q69. What's the difference between SLI, SLO, and SLA? SLI (Service Level Indicator) — an actual measured metric (e.g., "P99 latency" or "% of successful requests"). SLO (Service Level Objective) — an internal target for that SLI (e.g., "P99 latency < 200ms, 99.9% of the time"). SLA (Service Level Agreement) — an external, often contractual, commitment to customers, usually with a consequence (credits/ penalties) for missing it, and typically set looser than the internal SLO to leave margin for error (you don't want to breach a customer contract the moment you miss your internal target). **Q70.** What does "five nines" (99.999%) availability actually mean in terms of allowed downtime, and why is each additional nine dramatically harder to achieve? 99.999% availability allows about 5.26 minutes of downtime per year (compare: 99% allows ~3.65 days/year, 99.9% allows ~8.76 hours/year, 99.99% allows ~52.6 minutes/ year). Each additional nine is dramatically harder because you're no longer just handling "normal" failures — you need to eliminate essentially all single points of failure, automate failover to be near-instant (human-response-time incident handling alone can burn your whole yearly budget in one incident), and invest heavily in testing failure modes proactively (chaos engineering) rather than discovering them in production.

Q71. What is chaos engineering, and why do companies like Netflix intentionally inject failures into production? Chaos engineering is the practice of deliberately injecting controlled failures (killing instances, adding network latency, simulating a dependency outage) into a system — often in production — to verify that the system's resilience mechanisms (retries, circuit breakers, failover) actually work as designed, rather than only being tested in theory. Netflix's Chaos Monkey randomly terminates instances during business hours specifically to force teams to build services that tolerate instance failure as a routine event rather than a rare emergency, since

untested failure-handling code is one of the most common causes of major outages (the failure path is exercised far less often than the happy path, so bugs hide there).

Q72. What is graceful degradation, and give an example of applying it to a real system. Graceful degradation means designing a system to continue providing reduced (but still useful) functionality when a component fails, rather than failing completely. Example: an e-commerce product page that calls a recommendation service for “customers also bought” — if that service is slow/down, the page should render without the recommendations section (perhaps showing nothing or a cached/generic fallback) rather than failing the entire page load. This requires explicitly designing non-critical dependencies to fail independently (with timeouts and fallback values) rather than letting a failure in one component propagate to block the whole response.

Q73. What is the difference between horizontal and vertical auto-scaling, and what metrics commonly trigger auto-scaling? Horizontal auto-scaling adds/removes instances based on load. Vertical auto-scaling resizes an existing instance’s resources (less common in cloud-native systems, since it typically requires a restart). Common trigger metrics: CPU utilization, memory utilization, request queue depth/latency, and custom application metrics (e.g., messages-per-second in a queue a worker pool drains). A common pitfall discussed in interviews: scaling purely on CPU can be misleading for I/O-bound services (a service waiting on a slow downstream call might show low CPU while still being overloaded on concurrent connections) — the best signal is usually a metric close to the actual user-facing symptom, like request latency or queue depth.

Q74. What is the thundering herd problem in the context of auto-scaling / cache expiry / retries (beyond the earlier cache stampede answer), and how do exponential backoff and jitter address it? Thundering herd broadly describes many clients/processes waking up and acting simultaneously in response to the same trigger (a cache expiring, a service coming back online after an outage, a scheduled cron firing), overwhelming a shared resource with a synchronized burst. For retries specifically: if many clients experience a failure at the same moment (e.g., a brief service outage) and all retry with the same fixed delay, they’ll all hit the recovering service again simultaneously, potentially causing it to fail again immediately (a self-sustaining failure loop). Exponential backoff (doubling the wait time on each retry) reduces the total request rate over time. Jitter (adding a random offset to the backoff delay) desynchronizes clients so their retries spread out over a window instead of arriving in a synchronized spike — the combination (“exponential backoff with full jitter”) is the standard, AWS-recommended retry strategy.

Q75. What is a bloom filter, and where is it used in system design? A Bloom filter is a space-efficient probabilistic data structure that tests whether an element is possibly in a set or definitely not in a set — it can have false positives (says “maybe present” for something not actually present) but never false negatives (if it says “definitely not present,” that’s always correct). It works via multiple hash functions setting bits in a bit array; checking membership tests if all corresponding bits are set. Use cases: quickly checking if a key might exist in an LSM-tree SSTable before doing

an expensive disk read (Cassandra, RocksDB use this extensively), checking if a username is already taken before hitting the DB, or preventing “cache penetration” (repeated lookups for keys that don’t exist, which normally bypass the cache and hit the DB every time) by short-circuiting definite non-existence at the cache layer.

A2. HLD Case Studies — Full “Design X” Walkthroughs

Each case study below covers the core Q&A an interviewer would probe: requirements, estimation, high-level design, and the “deep dive” trade-offs. These are written as a sequence of questions and answers that mirror how the actual interview conversation unfolds.

Case Study 1: Design a URL Shortener (like Bit.ly)

Q76. What are the functional and non-functional requirements you’d clarify first? Functional: given a long URL, generate a short unique URL; when a user visits the short URL, redirect to the original long URL; optionally support custom aliases and link expiration. Non-functional: high availability (broken short links are visible and embarrassing), low latency redirects (this is on the critical path of every click), the system is read-heavy (redirects vastly outnumber creations — often 100:1 or more), and uniqueness of short codes must be guaranteed.

Q77. How would you estimate scale for this system? Assume 100M new URLs/day (~1,160 writes/sec average) and a 100:1 read:write ratio, giving ~116,000 redirects/sec average (and design for 2-3x that at peak). Storage: if each record (short code, long URL, metadata) is ~500 bytes, and you retain URLs for 5 years, that’s $100M \times 365 \times 5 \times 500 \text{ bytes} \approx 91\text{TB}$ — large, but well within a sharded relational or key-value store’s capability, especially since old/unused links can be cold-stored or expired. **Q78. How do you generate the short code itself?** Compare at least two approaches. Approach 1 — Base62 encoding of an auto-incrementing ID: use a counter (from a DB sequence or a dedicated ID-generation service) and encode it in base62 (a-z, A-Z, 0-9) to get a short, unique string. Simple and guarantees uniqueness without coordination once you have a unique ID, but a single global counter can become a bottleneck/SPOF at very high write rates (mitigated by pre-allocating ID ranges to different servers). Approach 2 — Hash the long URL (e.g., first 7 chars of an MD5/SHA256 hash): no coordination needed and naturally deterministic (same long URL always maps to the same short code, which can be a feature), but collisions are possible and must be handled (check-and-retry with a salt, or append characters on collision), and it doesn’t naturally support generating multiple different short codes for the same long URL if that’s ever needed.

Q79. Walk through the high-level architecture.

Client -> Load Balancer -> App Servers (write path: POST /shorten, read path: GET / {code}) App Servers -> Cache (Redis, hot short codes -> long URL) -> Database (short_code, long_url, created_at, expiry, owner) -> ID Generator service (if using counter-based codes)

Write path: client submits long URL → app server requests a unique ID/checks for collision → writes mapping to DB → returns short URL. Read path: client hits short URL → app server checks cache first (very high hit rate expected, since traffic is Pareto-distributed) → on miss, reads DB, populates cache, issues an HTTP 301/302 redirect.

Q80. Should the redirect be a 301 (permanent) or 302 (temporary) HTTP redirect, and why does it matter for this system? A 301 gets cached by the browser, meaning subsequent visits to the same short link don't even hit your server again — great for reducing server load, but it means you lose the ability to track click analytics on repeat visits from the same browser, and you can't easily change the destination later since browsers/ some CDNs will remember the old redirect. A 302 always hits your server, letting you track every click and change destinations freely, at the cost of more server load. Most production URL shorteners use 302 specifically to preserve analytics, accepting the extra load (which caching at the server side, not the browser, absorbs instead).

Q81. How would you scale the database as this system grows beyond a single instance's capacity? Shard by short code (e.g., hash-based sharding, or range-based on the first character(s) of the base62 code) so lookups route deterministically to the right shard without a lookup table. Add read replicas per shard for the read-heavy redirect traffic. Since URL mappings are essentially immutable once created (a given short code always points to the same long URL, aside from deletion/expiry), this workload is a great fit for aggressive caching and eventually-consistent replicas — there's no "read your own write must be perfectly fresh" requirement for most use cases.

Q82. How do you handle expired or deleted links, and custom aliases? Store an optional expires_at field; check it on read (return 404/410 if expired) and run a periodic background job to purge/archive expired rows so the hot dataset stays small. For custom aliases, treat the user-requested alias as the short code directly (skip the generator), with a uniqueness check against the DB (and this is a case where a Bloom filter can cheaply short-circuit "definitely available" checks before hitting the DB).

Case Study 2: Design a Rate Limiter

Q83. What are the key requirements for a distributed rate limiter? Accurately limit requests per client (by API key, user ID, or IP) to a configured threshold; work correctly across multiple application server instances (not just limit per-instance, which would let a client multiply their effective limit by hitting different instances); add minimal latency to every request (it's on the hot path); and be resilient (if the rate limiter itself fails, the system should typically fail open — allow traffic — rather than fail closed and block all legitimate traffic, unless the use case is security-critical enough to prefer fail-closed).

Q84. Why can't rate limiting be done with simple in-memory counters on each app server? Because with multiple app server instances behind a load balancer, each instance would track its own independent counter — a client could get N times the intended limit by having requests distributed across N servers, and there's no shared view of the client's true total request rate. This is why a distributed rate limiter needs shared, centralized (or at least synchronized) state, typically backed by Redis.

Q85. Describe a Redis-based implementation of the token bucket algorithm for a distributed rate limiter. Store each client's bucket state as a Redis hash: {tokens: N, last_refill_timestamp: T} . On each request: compute elapsed time since last_refill_timestamp , calculate tokens to add (elapsed × refill_rate, capped at bucket capacity), then check if at least 1 token is available — if so, decrement and allow; if not, reject (HTTP 429). This read-modify-write sequence must be atomic to avoid race conditions under concurrent requests from the same client, implemented via a Redis Lua script (which Redis executes atomically) or Redis's MULTI/EXEC transactions.

Q86. Where in the architecture should the rate limiter live, and what are the trade-offs of each placement? At the API Gateway/edge — centralizes the logic, protects all downstream services uniformly, rejects abusive traffic before it consumes any backend resources; but a gateway-level limiter may not have context about the cost of the specific downstream operation (a cheap read vs. an expensive write might deserve different limits). As middleware within each service — allows per-service, operation-aware limits (e.g., stricter limits on an expensive search endpoint than on a cheap health-check endpoint), but requires each service to implement/maintain its own limiter logic (or a shared library) and means abusive traffic still reaches the service before being rejected. Most production systems use both: a coarse global limit at the gateway, and finer per-endpoint limits within services for expensive operations.

Q87. What HTTP response and headers should a rate-limited request return, and why do the headers matter? Return HTTP 429 (Too Many Requests), along with headers like X-RateLimit-Limit (the total allowed), X-RateLimit-Remaining (how many are left in the current window), and Retry-After (seconds until the client should retry). These headers matter because they let well-behaved clients self-regulate proactively (slow down before hitting the limit) and know exactly when to retry, rather than blindly retrying immediately and making the problem worse. Case Study 3: Design a Distributed Key-Value Store (like DynamoDB/Cassandra, simplified)

Q88. What are the core requirements, and which side of the CAP theorem would you typically favor? Requirements: store/retrieve values by key with low latency at very large scale, high availability (the store is often a foundational dependency other systems build on — if it's down, everything built on it is down), and horizontal scalability of both storage and throughput. Most real-world key-value stores at this scale (Dynamo, Cassandra) favor AP (availability + partition tolerance) over strict consistency, since the target use cases (shopping carts, session stores, user preferences) tolerate brief staleness far better than they'd tolerate unavailability.

Q89. How is data partitioned and replicated across nodes? Data is partitioned using consistent hashing across nodes on a hash ring (see Q23), so each node owns a contiguous range of the hash space. For replication, each key is stored not just on its primary node but on the next $N-1$ nodes clockwise on the ring (the “preference list”), so the failure of any single node doesn’t cause data loss — a standard replication factor is 3.

Q90. How are read/write consistency levels tuned, and what does “quorum” mean in this context (connect to Q52)? Using the W/R/N model from Q52: a write is considered successful once W replicas acknowledge it; a read queries R replicas and returns the most recent version among them (using timestamps or vector clocks to determine “most recent,” and possibly triggering read repair to fix any replica found to be stale). With $N=3$, a common configuration is $W=2$, $R=2$ (satisfying $W+R>N=3$, guaranteeing overlap) — giving strong-ish consistency with tolerance for one node being slow/down on both reads and writes. Clients can tune this per-operation: $W=1$ for very fast, “best effort” writes, or $W=N$ for maximum durability at the cost of write latency/availability.

Q91. How do you resolve conflicts when concurrent writes to the same key happen on different nodes (in an AP-favoring, multi-master system)? Common strategies: Last-write-wins (LWW) using timestamps — simplest, but can silently lose one of the concurrent writes, which is unacceptable for some data (e.g., a shopping cart, where losing an “add item” write is a real bug). Vector clocks (see Q51) — detect that two writes were concurrent (not causally ordered) and surface both versions to the application/client to resolve (this is literally what Amazon’s original Dynamo paper does for shopping carts — merge the two cart versions rather than picking one). CRDTs (Conflict-free Replicated Data Types) — data structures specifically designed so concurrent updates can always be merged deterministically and automatically without conflicts (e.g., a “grow-only counter” or “last-writer-wins register” with well-defined merge semantics) — increasingly the preferred approach where the data type allows it, since it avoids pushing conflict resolution onto the client entirely.

Q92. How would you handle a node failure and recovery in this system? Failure detection typically uses gossip protocol (Q53) combined with heartbeats — peers mark a node as suspected-down if it misses enough heartbeats. While a node is down, its share of writes can be temporarily redirected to another node using “hinted handoff” (the receiving node stores a hint that this data actually belongs to the down node, and replays it once that node recovers) — this preserves availability for writes even during a node outage. On recovery, the node uses anti-entropy (background Merkle-tree comparison with its replica peers) to efficiently identify and repair any data it missed while it was down, without needing to compare every single key individually.

Case Study 4: Design a Unique ID Generator for a Distributed System (like Twitter Snowflake)

Q93. Why can't you just use an auto-incrementing database ID or a UUID for distributed, high-throughput unique ID generation? A DB auto-increment ID requires a single source of truth (the database sequence), which becomes a write bottleneck and SPOF at very high throughput across a sharded/distributed system with multiple write nodes. A standard UUID (v4, random) solves uniqueness without coordination, but UUIDs are not sortable/time-ordered (bad for use cases wanting IDs that roughly reflect creation order, like sorting a feed by recency without a separate timestamp column) and are larger (128 bits vs. 64 bits), which matters at billions of rows for both storage and index size/performance (B-tree indexes on random, non-sequential values suffer worse write amplification and cache locality than on sequential values).

Q94. Explain the structure of a Snowflake-style ID. A 64-bit integer composed of: 1 unused/sign bit, 41 bits for a millisecond timestamp (relative to a custom epoch, giving ~69 years of range), 10 bits for a machine/worker ID (allowing up to 1,024 distinct ID-generating nodes), and 12 bits for a per-millisecond sequence number (allowing up to 4,096 unique IDs per millisecond per machine). This structure gives IDs that are roughly time-sortable (since the timestamp is the most significant bits), globally unique without any coordination between machines (each machine only needs to know its own unique worker ID), and compact (fits in a standard 64-bit integer, unlike a 128-bit UUID).

Q95. How do you assign unique worker/machine IDs without manual configuration, and what happens if the system clock goes backward (clock skew/ NTP adjustment)? Worker IDs are commonly assigned via a coordination service (Zookeeper, etcd) that hands out unique IDs to each generator instance on startup, or derived deterministically from something like the last bits of the machine's MAC address/IP combined with a registration step to detect collisions. For clock rollback: since the algorithm assumes the timestamp is monotonically non-decreasing (to guarantee IDs generated later sort after IDs generated earlier), a common safeguard is for the generator to detect if the current time is behind the last recorded timestamp and, in that case, either refuse to generate IDs until the clock catches up, or throw an error/alert — silently proceeding could produce duplicate or out-of-order IDs.

Case Study 5: Design a Notification System (Push, SMS, Email)

Q96. What are the functional requirements and the main design challenge? Send notifications to users across multiple channels (push notification, SMS, email, in-app) triggered by various events (order shipped, new message, price drop), support both immediate and scheduled sends, and respect user preferences (opted out of marketing emails, quiet hours for push). The main design challenge is that this is inherently a fan-out and integration problem: many different parts of the system need to trigger notifications, each channel has a different third-party provider (APNs/FCM for push, Twilio for SMS, SendGrid/SES for email) with different rate limits/failure

modes, and the system needs to handle huge burst volume gracefully (e.g., a breaking news push to 50M users at once).

Q97. Sketch the high-level architecture.

Event sources (order service, chat service, etc.) -> publish “notification requested” events -> Message Queue (Kafka) -> Notification Service (consumers, one pool per channel or a unified worker that routes by channel) -> checks user preferences / dedup / rate limits -> calls channel-specific provider adapter -> Push: FCM / APNs -> SMS: Twilio -> Email: SES / SendGrid -> Delivery status tracked, retried on failure, DLQ on repeated failure

Decoupling via a message queue is central here: the originating service (e.g., “order shipped”) just publishes an event and moves on — it doesn’t need to know or care about notification delivery mechanics, retry logic, or provider outages, and a burst of events naturally gets buffered and smoothed rather than overwhelming downstream providers synchronously.

Q98. How do you avoid sending duplicate notifications for the same event (e.g., due to at-least-once delivery causing a consumer to reprocess a message)?

Assign each notification request a unique idempotency key (often derived from the triggering event’s ID + channel + user), and before sending, check (and atomically set) that key in a fast store (Redis, with a TTL) — if it’s already present, skip sending. This makes the “process a notification event” step idempotent, so retries/redelivery from the queue (which at-least-once delivery guarantees will happen occasionally) don’t result in the user getting the same push notification three times.

Q99. How do you handle a scenario where a third-party provider (e.g., FCM) is down or heavily rate-limiting you? Wrap the provider call in a circuit breaker (Q43) so persistent failures stop hammering the dead provider and fail fast, giving it room to recover. Failed sends go back onto a retry queue with exponential backoff + jitter (Q74). For strict rate limits from the provider, use a token-bucket limiter (Q44) tuned to the provider’s documented rate, with excess load queued rather than dropped (unless the notification has a very short relevance window, e.g., an OTP that’s useless after 5 minutes, in which case it’s reasonable to expire and drop it rather than deliver it very late).

Q100. How would you design the system to respect user notification preferences and “quiet hours” without checking a database on every single send? Cache user preferences in a fast in-memory/Redis store (preferences change infrequently relative to how often they’re read, making this a great caching candidate — see Q11), invalidated on preference update. For quiet hours specifically (don’t push between 10pm-8am in the user’s local timezone), compute the user’s local time from their stored timezone at send time and either delay the send (re-queue with a scheduled delivery time) or, for a large-scale system, batch-schedule non-urgent notifications into per-timezone delivery windows ahead of time rather than checking per-message.

Case Study 6: Design a Chat/Messaging System (like WhatsApp)

Q101. What are the key functional and non-functional requirements?

Functional: 1:1 and group messaging, message delivery status (sent/delivered/read), online presence, message history/persistence, media sharing. Non-functional: very low latency for real-time delivery, high availability, messages must not be lost (durability), and the system must scale to billions of messages/day with mostly small message payloads (text) but occasional large ones (media).

Q102. Why is a persistent WebSocket connection typically chosen over HTTP polling for real-time chat, and what challenge does it introduce at scale? HTTP polling (repeatedly asking “any new messages?”) wastes bandwidth and adds latency (bounded by poll interval); long polling improves this but still has request overhead per cycle. A WebSocket keeps a single persistent, full-duplex TCP connection open, letting the server push messages to the client instantly with minimal per-message overhead — essential for a responsive chat experience. The challenge: maintaining millions of concurrent open connections requires connection-oriented infrastructure (each server can only hold so many open sockets, unlike stateless HTTP request handling which any server can pick up), so you need dedicated, horizontally-scaled “connection servers” and a way to route a message to whichever specific server holds the recipient’s live connection.

Q103. How do you route a message to the correct recipient when that recipient’s WebSocket connection lives on a different server than the sender’s?

Maintain a connection registry (commonly in Redis) mapping `user_id` -> `server_id` (which connection server currently holds that user’s live socket). When server A receives a message from user X destined for user Y, it looks up which server Y is connected to (say, server B), then forwards the message internally (via a pub/sub channel or direct RPC) to server B, which pushes it down Y’s WebSocket. If Y isn’t currently connected (offline), the message is persisted and delivered via push notification, then delivered over WebSocket on Y’s next connect (with an “undelivered messages” sync step on reconnect).

Q104. How do you implement message delivery status (sent / delivered / read) reliably?

Each message gets a unique ID and status field, updated via explicit acknowledgment events: client sends message → server persists it and marks “sent” → server pushes to recipient’s device, which ACKs receipt → server marks “delivered” → when the recipient actually opens/views the chat, the client sends a “read” event → server marks “read” and (for 1:1 chats where this is shown) notifies the sender. This requires the recipient’s client to explicitly ack at each stage rather than the sender assuming success, since a network drop between “server sent it” and “client received it” is common and must be distinguishable from an actual successful delivery.

Q105. How would you design the message storage schema, and why is a wide-column/NoSQL store (like Cassandra) often preferred over a relational DB here? A common schema partitions by conversation: (`conversation_id`, `message_id`/timestamp) -> {sender, content, status}, so fetching a conversation’s message history

is a single efficient range scan within one partition. Cassandra-like stores are often preferred because: the access pattern is simple and known in advance (fetch recent messages for a conversation, append new ones) rather than needing complex ad hoc joins; write throughput needs to be very high and horizontally scalable (billions of messages/day); and the data naturally partitions cleanly by conversation ID, avoiding cross-shard queries almost entirely — playing directly to a wide-column store's strengths while avoiding its weaknesses (poor support for complex joins/transactions, which this workload doesn't need).

Q106. How do you handle group chats differently from 1:1 chats at the architecture level? For 1:1, a message is delivered to exactly one other recipient. For group chats, a single message must fan out to every group member — for small groups this is a simple write-to-N-connections/N-inboxes operation, but for very large groups (a broadcast channel with millions of members), naive fan-out-on-write becomes far too expensive. Large-scale systems typically switch strategy based on group size: fan-out-on-write for small/medium groups (push a copy to each member's inbox/queue immediately — fast reads, since each recipient's data is precomputed), and fan-out-on-read for very large groups/channels (store the message once, and each reader pulls/computes what's new when they check — avoids an enormous write amplification for one message to millions of recipients, at the cost of a slightly more expensive read).

Case Study 7: Design a News Feed System (like Facebook/ Instagram/ Twitter feed)

Q107. What is the fundamental design tension in a news feed system, and how do the two main strategies address it? The tension is the same fan-out trade-off as Q106, generalized: a feed shows an aggregation of posts from everyone you follow, ordered by recency/relevance. Fan-out-on-write (push model): when a user posts, immediately push that post into the precomputed feed of every follower. Reads are then just "fetch my precomputed feed" — extremely fast. But for a user with millions of followers (a celebrity), a single post triggers millions of writes, and most of that work may be wasted (many followers may never check their feed before the post is stale/irrelevant). Fan-out-on-read (pull model): don't precompute anything; when a user opens their feed, query posts from everyone they follow in real time and merge/rank them. This avoids the celebrity write- amplification problem, but makes every feed read expensive (potentially querying and merging from hundreds of "following" sources), which is a problem since feed reads happen far more often than posts are created.

Q108. What is the hybrid approach most large-scale feed systems actually use? Use fan-out-on-write for the vast majority of users (with normal follower counts), giving them fast, cheap feed reads. For "celebrity" accounts above some follower threshold, skip the write fan-out entirely — their posts are fetched live (fan-out-on-read) and merged into a follower's feed only at read time, alongside their precomputed feed from regular follows. This bounds the worst-case write amplification (a celebrity's post never triggers millions of writes) while keeping reads fast for the common case. Q109. Once you have a merged candidate set of posts for a feed, how is ranking typically

approached at a high level? Rather than pure reverse-chronological order (which many modern feeds have moved away from), candidate posts are scored by a ranking model considering signals like recency, predicted engagement probability (likes/ comments/ shares based on the user's past behavior with similar content/authors), relationship strength (how often the user interacts with this particular author), and content type diversity (avoiding an all-video or all-same-author feed). At real scale this is usually a two-stage process: a cheap, high-recall candidate generation step (pull a broad set of possibly-relevant posts, often using approximate/precomputed signals) followed by a more expensive, high-precision ranking step (an ML model) applied only to that smaller candidate set, since running a full ML ranking model over every possible post for every user would be computationally infeasible at scale.

Q110. How would you design the storage for the precomputed feed (the fan-out- on-write inbox) itself? A common approach: a wide-column/key-value store keyed by `user_id`, storing a bounded, time-ordered list of recent post IDs (not full post content — just references, to keep this structure small and fast to update) — e.g., Redis sorted sets (score = timestamp) work well here, capped to the most recent N entries per user (old entries trimmed) since users rarely scroll back indefinitely. The actual post content is fetched separately (and cacheably) by ID when rendering, keeping the “feed” structure itself lightweight and fast to fan out to millions of followers.

Case Study 8: Design a Video Streaming Service (like YouTube/ Netflix)

Q111. What are the two very different workloads this system must support, and why do they need separate design consideration? Video upload/ingestion (write-heavy but relatively low volume/frequency — creators upload occasionally) requiring transcoding into multiple resolutions/formats, and video playback/streaming (extremely read-heavy, massive scale, latency-and-bandwidth-sensitive, global audience). They're architecturally almost independent systems connected by the processed video asset: upload goes through an async processing pipeline, while playback is a largely separate, CDN-heavy read path — designing them as one tightly coupled system would make each harder to scale on its own terms.

Q112. Walk through the video upload and processing pipeline.

Client uploads raw video -> Upload Service -> Object Storage (raw file) -> publishes “video uploaded” event -> Message Queue -> Transcoding workers (pull job, run e.g. FFmpeg) -> produce multiple resolutions (240p...4K) and formats -> segment into small chunks for adaptive streaming (HLS/DASH) -> write processed segments to Object Storage -> update video metadata DB: status = “ready”, available renditions -> push segments to CDN edge (or CDN pulls on first request) Transcoding is CPU-intensive and slow (can take longer than the video's own runtime for high resolutions), so it's always done asynchronously via a worker pool that can scale horizontally with upload volume, never synchronously blocking the upload response.

Q113. What is adaptive bitrate streaming (ABR), and why is it essential for a video platform at scale? ABR (via protocols like HLS or MPEG-DASH) splits video into short segments (a few seconds each), each encoded at multiple quality/bitrate

levels. The client player continuously monitors its actual network throughput and buffer health, and dynamically requests the next segment at whichever quality level fits current conditions — seamlessly stepping down quality during a network dip (avoiding a hard buffering stall) and stepping back up when bandwidth improves. This is essential because user network conditions vary enormously (mobile on a congested cell tower vs. fiber at home) and a fixed- quality stream would either be unwatchably choppy for slow connections or wastefully low- quality for fast ones.

Q114. How does a CDN specifically help video delivery, beyond the general caching benefit described earlier? Video is by far the largest-bandwidth content most systems serve, so serving it from an origin server directly to a global audience would require enormous, expensive origin bandwidth and would suffer high latency for distant users (bad for video start time and buffering). A CDN caches video segments at edge locations near users, meaning the actual video bytes for a popular video are served from a nearby edge cache after the first request in that region, and the origin only needs to serve each segment once per edge location rather than once per viewer — for viral content this can be a many- orders-of-magnitude reduction in origin load.

Q115. How would you design the “resume watching” / playback position feature at scale, given it requires very frequent small writes? Playback position updates happen frequently (every few seconds while watching) but don’t need strong durability guarantees for every single update (losing the last 5 seconds of position tracking on a crash is a non- issue) — this makes it a good candidate for a fast key-value store (Redis) rather than the primary relational DB, written with a relatively relaxed durability setting, and only periodically/eventually persisted to more durable storage (or simply treated as ephemeral state that’s acceptable to lose entirely on a cache failure, re-derived as “start from a recent checkpoint” rather than exact position). This avoids hammering the primary database with an extremely high-frequency, low-value-per-write workload.

Case Study 9: Design a Ride-Sharing System (like Uber)

Q116. What are the core functional requirements and the central real-time challenge? Riders request rides, drivers accept them, the system tracks live locations of both, matches nearby available drivers to riders, computes ETA and pricing (including surge), and coordinates the trip lifecycle. The central challenge is a real-time, high- frequency geospatial problem: efficiently finding the nearest available drivers to a rider’s location out of potentially millions of drivers, updated continuously as everyone moves, at low latency.

Q117. How do you efficiently query “nearest available drivers” without scanning every driver’s location on every request? Use a geospatial indexing scheme that partitions the map into cells so a query can be limited to nearby cells rather than a full scan. Geohashing encodes latitude/longitude into a string where nearby locations share string prefixes, allowing “find drivers near me” to become a prefix-range query. Quadtrees recursively subdivide space into quadrants, going deeper (smaller cells) in densely populated areas and staying shallow in sparse areas, which

naturally balances cell density. Uber's own published approach (the H3 hexagonal grid system) uses hexagonal cells specifically because hexagons have uniform adjacency (all neighbors are equidistant, unlike a square grid's diagonal neighbors being farther away), which simplifies "expand search radius" logic. In all cases, driver locations are indexed by cell, and a rider's request only needs to look at the rider's cell plus a small ring of surrounding cells, expanding the ring only if too few drivers are found.

Q118. How do driver location updates get propagated at scale, given every active driver's phone is pushing location every few seconds? Driver apps send location pings (every 3-5 seconds while active) via a lightweight write path (not the full request/response API) — often through a message queue (Kafka) to decouple the high-frequency ingestion from downstream consumers, feeding a fast-access geospatial store (Redis with geospatial commands, or a dedicated in-memory index) that's optimized for this write-heavy, read-heavy-but-approximate workload. This location data is inherently ephemeral and doesn't need durable long-term storage at full granularity (though a sampled/aggregated trail might be kept for trip history and ETA-model training) — it just needs to be fast and roughly current, another case where relaxed durability is the correct trade-off (see Q115's reasoning applied to a different domain).

Q119. Once candidate nearby drivers are found, how does the actual matching/assignment work, and what race condition must be handled? The system typically scores candidate drivers (distance/ETA, driver rating, acceptance likelihood) and sends the request to the top choice (or a small batch) with a short timeout to accept; if declined/timed out, it cascades to the next candidate. The key race condition: two different ride requests could simultaneously try to match the same driver. This is handled by treating "assign driver to ride" as an atomic, exclusive operation (e.g., a conditional/compare-and-swap update on the driver's status field — only succeeds if the driver's status is still "available" at the moment of assignment, implemented via a DB transaction, Redis WATCH /optimistic locking, or a distributed lock on the driver ID) so exactly one request can win the assignment even if two requests reach the "assign" step at nearly the same instant.

Q120. How would you design dynamic/surge pricing at a high level?

Continuously compute a supply/demand ratio per geographic cell (same cells used for matching) — number of open ride requests vs. number of available nearby drivers in a recent time window — and derive a surge multiplier from that ratio (often with smoothing/capping to avoid wild price swings from momentary noise, and often gradually decaying rather than snapping instantly). This is fundamentally a streaming aggregation problem (Kafka Streams or a similar stream-processing layer computing rolling per-cell supply/demand windows) feeding a pricing service that both riders (shown the multiplier before requesting) and the matching system consult.

Case Study 10: Design a Search Autocomplete / Typeahead System

Q121. What are the requirements, and why is latency the dominant design constraint? As a user types, the system must return the top-k matching suggestions

after every keystroke, typically within ~100-200ms to feel instant — any slower and the UI feels laggy, breaking the core UX value of autocomplete. This means the query path must avoid expensive operations entirely; nearly all the real work needs to happen ahead of time (offline/async), with the online query path reduced to a cheap lookup.

Q122. What data structure is classically used to serve prefix-based suggestions efficiently, and how does it work? A trie (prefix tree), where each node represents a character and a path from root to a node represents a prefix; each node can also cache its own top-k most popular completions (precomputed) so a query for a given prefix is just a single trie traversal to that node followed by returning its cached top-k list — $O(\text{length of prefix})$ lookup, independent of how many total terms exist in the system. Storing the top-k at each node (rather than computing it at query time by exploring the whole subtree) is what keeps this fast enough for a per-keystroke query.

Q123. How is the trie kept up to date with changing search popularity, given rebuilding it on every single search event would be far too expensive? Don't update the live, query-serving trie in real time. Instead, log raw search queries into a data pipeline, aggregate query frequency counts periodically offline (e.g., hourly or daily batch jobs, often via a system like Spark), rebuild (or incrementally update) the trie's top-k caches from these aggregated counts, then swap in the new trie version — often by building the new trie entirely offline and hot-swapping the reference once ready, so query traffic always hits a complete, consistent structure rather than a partially-updated one.

Q124. How would you shard the trie if it's too large to fit on a single machine? Shard by the first character(s) of the prefix (e.g., all terms starting with 'a' on shard 1, 'b' on shard 2, etc., possibly grouping less-common starting letters together for balance) — since a query always starts with a known prefix, the client/gateway can route directly to the correct shard without needing a lookup step. This is one of the cleaner sharding cases because the natural query pattern (always anchored at a prefix) aligns perfectly with a simple, deterministic sharding key.

Q125. How do you personalize autocomplete suggestions (e.g., showing a user's own recent searches or location-relevant results) without breaking the global- popularity model above? Layer personalization on top rather than baking it into the shared trie: fetch the global top-k suggestions from the trie as the base candidate set, and separately merge in a small number of personalized candidates from a lightweight per-user store (recent searches, location-boosted terms), with a final client-side or edge-side merge/ re-rank step. This keeps the expensive, shared global structure simple and cacheable while still allowing customization, rather than needing a separate massive trie per user (which wouldn't scale). Case Study 11: Design a Web Crawler

Q126. What are the main requirements and inherent challenges of a web crawler at scale? Requirements: discover and download web pages starting from seed URLs, extract new links to crawl further, avoid re-crawling the same page excessively, respect robots.txt and crawl politeness (rate limits per domain so you don't hammer

any single site), and scale to billions of pages. Inherent challenges: the web graph is enormous and has no natural bound, some sites intentionally or unintentionally create infinite crawl spaces (e.g., calendar pages generating a new URL for every possible date indefinitely), and different sites have wildly different crawl-politeness requirements.

Q127. Describe the high-level architecture and data flow.

Seed URLs -> URL Frontier (priority queue of URLs to crawl, partitioned/politeness-aware) -> Fetcher workers pull URLs, download page content -> Duplicate/already-crawled check (URL-seen filter, e.g. Bloom filter) -> Parser extracts links + content -> New links -> back into URL Frontier (after dedup/normalization) -> Content -> Storage (raw HTML) + indexing pipeline

The URL Frontier is the central coordination point — it's essentially a smart, prioritized queue that decides what gets crawled next, balancing freshness, priority (importance of a page/ site), and per-domain politeness.

Q128. How do you enforce crawl politeness (not overwhelming any single website with requests) while still maximizing overall crawler throughput?

Partition the URL Frontier by domain, and maintain a per-domain queue with its own rate limiter/delay (respecting robots.txt Crawl-delay directives where present, or a sensible default like 1 request per few seconds per domain). A common implementation uses many separate FIFO queues (one per domain, or per small set of domains) feeding into a central mapping table, with worker threads pulling from different domain queues round-robin — this lets the overall crawler run many concurrent fetches (high throughput) while guaranteeing no single domain sees requests faster than its politeness policy allows.

Q129. How do you detect and avoid re-crawling duplicate content, given the same content often appears at many different URLs (mirrors, tracking parameters, etc.)? Two separate dedup layers: URL-level dedup — normalize URLs (strip tracking params, resolve relative paths, lowercase the domain) and check against a seen-URL set (a Bloom filter is ideal here — see Q75 — since “possibly a duplicate, double check” false positives are cheap, but the memory savings over an exact set at billions of URLs are massive). Content-level dedup — compute a content hash (or a similarity hash like SimHash/MinHash, which tolerates minor differences like a timestamp in a footer) of the fetched page and check against previously seen content hashes, so byte-identical or near- identical content at different URLs isn't fully reprocessed/reindexed.

Q130. How do you handle crawler traps (e.g., an infinite calendar of auto-generated links)? Common mitigations: cap the maximum crawl depth from a given seed/ domain, cap the total number of pages crawled per domain within a time window, detect URL patterns that look suspiciously repetitive/parameterized (many URLs differing only in a query parameter that keeps incrementing) and throttle or flag them for manual review, and track a page's “usefulness” signal (e.g., content similarity to

prior pages on the same path) to deprioritize a domain that's producing low-value, near-duplicate content indefinitely.

Case Study 12: Design a Distributed File Storage System (like Google File System / simplified S3)

Q131. What are the core requirements, and how does this differ from designing a single-machine filesystem? Store very large files (potentially far larger than any single disk) reliably and with high throughput for large sequential reads/writes (the classic GFS use case was big-data/analytics workloads, not small random-access files), replicate data for durability against machine failure, and scale storage capacity by simply adding more machines. Unlike a single-machine filesystem, every component (where a file's data lives, whether a machine holding a replica is still alive) must be tracked explicitly rather than assumed, because machines will fail at this scale as a routine, expected event rather than a rare exception.

Q132. Describe the classic GFS-style architecture: how are metadata and actual file data separated? A single (logically; in practice made highly-available via consensus/ hot-standby) master/metadata server holds the filesystem namespace, directory structure, and — critically — the mapping of which chunkservers hold which chunks of each file, but not the actual file data itself. Files are split into large fixed-size chunks (64MB in the original GFS design — deliberately large to keep metadata small relative to data volume and to make large sequential I/O efficient). Actual chunk data is stored and served directly by chunkservers, replicated across several of them (typically 3x) for durability. Clients ask the master “which chunkservers have chunk X of this file,” then talk directly to a chunkservers for the actual data transfer — keeping the master out of the (much higher- volume) data path entirely, so it doesn't become a throughput bottleneck.

Q133. Why does keeping the master out of the data path matter so much for scalability, and how is the master itself protected from being a single point of failure? If every byte of file data had to flow through the master, it would immediately become the throughput bottleneck for the entire cluster regardless of how many chunkservers you add — the master's job is deliberately limited to lightweight metadata operations (which are relatively low-volume: “here's where chunk X lives,” not “here is chunk X”), so the system's actual data throughput scales with the number of chunkservers, unconstrained by the master. The master itself is protected via operation log replication to a standby (shadow master) that can take over on failure, plus the design assumption that a brief master outage only blocks new metadata operations (finding files, allocating new chunks) while in-flight data transfers between clients and chunkservers — which don't involve the master — can often continue unaffected.

Q134. How is data consistency maintained when a chunk has multiple replicas, and what happens on a write? Writes are coordinated by a primary replica (one of the chunk's replicas, granted a time-limited lease by the master) which orders and applies the write locally, then forwards it to the other (secondary) replicas in the same

order, and only acknowledges success to the client once enough replicas have applied it — ensuring all replicas apply writes in a consistent order even under concurrent write requests, without needing the master to coordinate every single write (the primary/lease mechanism keeps the master out of the write path too, for the same throughput reasons as Q133).

Case Study 13: Design a Payment Processing System

Q135. What are the defining non-functional requirements that make payment systems different from most other systems in this guide? Correctness and durability are non-negotiable above almost all else — a lost or duplicated payment is a direct financial/ legal/trust problem, not a degraded-UX problem like a stale cache elsewhere in this guide. This pushes the system firmly toward strong consistency and ACID transactions for the core ledger, idempotency everywhere on the write path (Q36), a complete audit trail (often via event sourcing, Q66), and typically favoring correctness/availability trade-offs opposite to what a social feed or chat system would choose — you’d rather briefly reject a payment than risk processing it twice or losing record of it.

Q136. Walk through the high-level flow of processing a payment, including the role of an external payment gateway/processor.

Client -> Payment Service: initiate payment (amount, order_id, idempotency_key)
Payment Service: create payment record (status: PENDING) in DB transaction
Payment Service -> Payment Gateway (Stripe/Adyen/etc.): charge request
Payment Gateway -> external card networks/banks -> authorization result
Payment Gateway -> Payment Service: webhook/response (success/failure)
Payment Service: update payment record status (transactional), publish “payment completed” event
Downstream services (order, inventory, notification) consume the event

Most real systems don’t build direct bank/card-network integration themselves — they integrate with a payment gateway/processor that handles PCI compliance, card network integration, and fraud tooling, and the “system design” work is really about reliably orchestrating around that gateway (idempotency, retries, reconciliation, handling async webhook confirmation) rather than the low-level payment rails themselves.

Q137. Why is idempotency especially critical here, and how is it typically implemented end-to-end? Network failures between the client and payment service, or between the payment service and the gateway, are ambiguous — a client that doesn’t get a response genuinely doesn’t know if the charge succeeded, and a naive retry could cause a double charge. The client generates a unique idempotency key per logical payment attempt (often tied to the order ID) and sends it with the request; the payment service persists this key with the payment record and, on receiving a request with a previously-seen key, returns the stored result instead of processing again. Critically, this idempotency check-and-record must happen atomically with the payment state transition (typically within the same DB transaction) — a check-then-act race with two concurrent requests for the same key must not both slip through.

Q138. Given the payment gateway confirms success asynchronously via a webhook, how do you handle the case where the webhook is delayed, duplicated, or never arrives? Duplicated webhooks: the webhook handler must itself be idempotent (using the gateway’s own event ID to dedupe, since gateways explicitly document that webhooks can be delivered more than once). Delayed/missing webhooks: don’t rely on the webhook as the only path to final status — run a periodic reconciliation job that polls the gateway’s API for the status of any payment still stuck in PENDING beyond an expected window, catching cases where the webhook was lost due to a network issue, an outage on your endpoint, or a misconfiguration — this reconciliation safety net is standard practice specifically because webhook delivery is explicitly best-effort, not guaranteed, from virtually every payment gateway provider.

Q139. How would you design the ledger/accounting data model to make the system auditable and to prevent balance corruption bugs? Use a double-entry bookkeeping model (borrowed from traditional accounting) rather than simply mutating a single “balance” field: every transaction creates at least two ledger entries (a debit and a corresponding credit) that must net to zero, and an account’s balance is always derived (summed) from its ledger entries rather than stored and mutated directly. This makes the system self-auditing (if debits and credits ever don’t balance across the whole ledger, something is provably wrong and can be caught by a routine integrity check) and naturally append-only/immutable (entries are never updated or deleted, only new correcting entries are added for reversals), which is exactly the property regulators and auditors expect and that a simple “balance += amount” mutable field can never really guarantee.

Q140. How do you handle currency and rounding correctly across a system that may operate in multiple countries? Store monetary amounts as integers in the smallest currency unit (cents for USD, not floating-point dollars) to avoid floating-point rounding errors entirely — this is a near-universal rule in financial system design, since binary floating point cannot represent most decimal fractions exactly, and compounding rounding errors across millions of transactions is a real, historically-documented source of bugs and even security exploits (“salami slicing” attacks). Store currency explicitly alongside every amount (never assume a single global currency), and centralize any currency conversion logic (with a clear “as of what exchange rate/ timestamp” record for auditability) rather than scattering conversion math across the codebase.

Case Study 14: Design a Ticket Booking System (like BookMyShow/ Ticketmaster) — Handling Concurrency Hotspots

Q141. What is the central hard problem in this system that doesn’t show up in most other case studies? Extreme write contention on a small number of “hot” rows during a popular event’s on-sale moment — thousands of users simultaneously trying to book the same limited set of seats for a popular concert/movie the second tickets go live. Unlike most systems in this guide where the answer is “scale horizontally and cache reads,” here correctness under contention (never double-selling a seat) on a

small, hot dataset is the actual crux of the problem, and it can't be solved by just adding more read replicas since this is fundamentally a write-conflict problem.

Q142. How do you prevent two users from being sold the same seat (the double-booking problem)? The core mechanism is some form of exclusive lock or atomic conditional update on the specific seat at reservation time: e.g., `UPDATE seats SET status='reserved', user_id=? WHERE seat_id=? AND status='available'` — this is an atomic compare-and-swap at the database level; only one of two concurrent requests for the same seat will actually match the `WHERE` condition and update a row, the other gets zero rows affected and is told the seat is unavailable. This avoids the classic bug of “SELECT to check availability, then separately UPDATE to reserve” (a check-then-act race condition), where two concurrent requests could both pass the check before either commits its update.

Q143. Why is a temporary “hold” or reservation step (rather than immediate purchase) typically part of the flow, and how is it implemented? Users need time to enter payment details after selecting seats; without a hold, another user could grab the same seat during that gap. The system reserves the seat with a short-lived hold (e.g., `status='held', held_until=now()+10min`) atomically at selection time (using the same conditional-update pattern from Q142), and a background expiry process (or a TTL-based mechanism like a Redis key expiring, triggering a callback/cleanup) releases holds that aren't converted to a confirmed purchase within the window, flipping the seat back to available so it re-enters inventory instead of being permanently lost to an abandoned checkout.

Q144. How would you architect the system to survive a massive traffic spike at the exact moment popular tickets go on sale, without the database melting down? Put a virtual waiting room / queueing layer in front of the booking flow itself — when demand vastly exceeds available inventory, admit users into the actual booking flow at a controlled rate (e.g., via a queue token system) rather than letting all of them hit the seat-selection/reservation endpoints simultaneously; this converts an unbounded traffic spike into a bounded, manageable admission rate the backend can actually handle. Behind that: aggressive caching of read-only data (event details, venue seat maps — these don't change during the sale), and, since the actual seat inventory is naturally sharded by event (an event's seats never need to be locked against a different event's seats), the hot contention is naturally isolated per event rather than being one giant global lock across the whole system.

Case Study 15: Design a Centralized Logging and Metrics Monitoring System

Q145. What are the two distinct data types this system handles, and why do they need different storage/query optimizations? Logs are typically high-cardinality, unstructured/semi-structured text events (a specific error message, a request trace) — queried by full-text/keyword search and time range, and often need to preserve raw detail for debugging a specific incident. Metrics are structured, low-cardinality numeric time-series (CPU%, request count, latency) — queried primarily as

aggregations over time (average, percentile, rate of change) across dimensions (per-service, per-host). Logs favor storage optimized for search/text indexing (like Elasticsearch), while metrics favor a time-series database optimized for compact storage of numeric points and fast range/aggregation queries (like Prometheus, InfluxDB) — using one system for both tends to be a poor fit for at least one of the two workloads, which is why production observability stacks (e.g., the “ELK” + Prometheus combination) typically run them as separate specialized systems.

Q146. Describe the high-level data flow for collecting logs/metrics from thousands of distributed hosts/services.

App instances -> local agent (e.g., Fluentd/Filebeat for logs, a metrics exporter for Prometheus-style pull, or push-based via StatsD) -> Buffering/Queue layer (Kafka) — decouples ingestion spikes from processing -> Stream processing / ingestion pipeline (parsing, enrichment, sampling) -> Storage (Elasticsearch for logs, TSDB for metrics) -> Query/Dashboard layer (Kibana/Grafana) + Alerting engine

The message queue buffering layer matters a lot here specifically because observability data volume tends to spike exactly when the underlying system is having problems (an incident generates a flood of error logs and abnormal metrics) — the monitoring pipeline itself must not fall over under that same load, or you lose visibility at precisely the moment you need it most.

Q147. Why is sampling often necessary for high-volume logging/tracing, and how do you sample without losing the events that matter most? At sufficient scale, capturing and storing every single log line/trace from every request becomes prohibitively expensive (both storage cost and ingestion throughput), and most of that volume is low-value routine/successful requests. Naive random sampling (keep 1% of all events) risks losing rare-but-important events (like the one trace that captured an actual error). Better: biased/priority sampling — always keep 100% of error/exception events and traces exceeding a latency threshold (the interesting cases), while sampling down the high-volume “normal, fast, successful” traffic aggressively, since aggregate metrics already capture the overall health signal for the normal-case traffic and you don’t need every single normal request’s full trace detail.

Q148. How does an alerting engine avoid “alert fatigue” from noisy, low-value alerts while still catching real incidents quickly? Techniques: thresholds with a sustained duration rather than instantaneous (“CPU > 90% for 5 minutes,” not a single noisy spike), anomaly-based alerting (alert on deviation from a learned normal baseline/ seasonality rather than a fixed static threshold, since “normal” traffic at 3am vs. 3pm can differ enormously), alert deduplication and grouping (one incident causing 500 downstream alerts should surface as one correlated incident, not 500 separate pages), and escalation policies with severity tiers (a P1 pages someone immediately; a P4 just shows up on a dashboard) so on-call engineers aren’t paged for issues that don’t need immediate human action.

Case Study 16: Design a Distributed Job Scheduler (like a cron-as-a-service system)

Q149. What are the core requirements, and why is a naive single-cron-per-machine approach insufficient at scale? Requirements: schedule jobs to run at specific times or intervals (cron-like), guarantee a scheduled job runs exactly once (not zero times due to the scheduling host being down, not multiple times if you naively run cron on several redundant hosts), scale to potentially millions of distinct scheduled jobs, and handle jobs that take longer than their scheduling interval gracefully. A naive single cron instance is a SPOF (if that machine is down at the scheduled time, the job silently doesn't run), and naively running the same cron config redundantly on multiple machines for "reliability" causes every job to run multiple times instead — neither is acceptable, which is exactly why this needs real distributed-systems design rather than just replicating a Unix cron config.

Q150. How do you guarantee a scheduled job executes exactly once, even with multiple scheduler instances running for redundancy? Use a distributed lock (or leader election, Q55) so that at any given trigger time, only one scheduler instance actually claims and executes a given due job — e.g., using a database row with a conditional update (`UPDATE jobs SET status='claimed', claimed_by=? WHERE job_id=? AND status='pending' AND due_at<=now()`), the same atomic-conditional-update pattern from Q142) so multiple scheduler instances scanning for due jobs simultaneously will only have one of them successfully claim any given job. Combined with a claim timeout/heartbeat (if the claiming instance crashes mid-execution, the claim expires and another instance can retry), this gives effectively-exactly-once execution without needing a single, non-redundant scheduling process.

Q151. How would you store and efficiently query "what jobs are due to run right now" out of potentially millions of scheduled jobs? Index jobs by their next-scheduled- run timestamp (e.g., a database index on `next_run_at` , or a time-ordered structure like a Redis sorted set scored by run time) so "find due jobs" is an efficient range query (`next_run_at <= now()`) rather than a full scan of all jobs. For recurring jobs, store the cron expression and compute/update `next_run_at` immediately after each execution (rather than recomputing the schedule on every scheduler tick), so the "what's due" query stays a cheap, simple range lookup regardless of how complex the underlying recurrence rule is.

Q152. How do you handle a job whose execution takes longer than its scheduling interval (e.g., a job scheduled every minute that sometimes takes 90 seconds to run)? Explicitly decide and enforce a concurrency policy per job rather than letting it happen implicitly: skip (if the previous run is still in progress when the next is due, skip this trigger and wait for the next interval), queue (let the next run wait until the current one finishes, then execute immediately, effectively serializing all runs of this job), or allow concurrent runs (if the job is safe to run in parallel with itself). This must be an explicit design decision because the naive default behavior (just launching a new instance every time the interval fires, regardless of whether the last

one finished) can silently cause an unbounded pile-up of overlapping executions of a slow job, eventually exhausting worker capacity.

Case Study 17: Design a Retrieval-Augmented Generation (RAG) / AI Search System

Q153. What problem does RAG solve, and what are the main components of a RAG system architecture? RAG grounds an LLM's responses in a specific, up-to-date, or private knowledge base (documents, internal wikis, product catalogs) that the model wasn't trained on, by retrieving relevant context at query time and injecting it into the prompt — reducing hallucination and enabling the system to answer questions about content that changes far more often than any model's training/retraining cycle could keep up with. Core components: an ingestion pipeline (chunk documents, generate embeddings, store in a vector index), a retrieval layer (embed the incoming query, perform a similarity search against the vector index to find the most relevant chunks), and a generation layer (an LLM call with the retrieved chunks injected into the prompt as context, plus the original query).

Q154. Why is document chunking strategy important, and what's the trade-off between small and large chunks? Chunking matters because embeddings and retrieval work over fixed pieces of text, and the chunk size directly affects retrieval quality: too small (e.g., single sentences) can lose important surrounding context, causing retrieved snippets that are technically relevant but insufficient to actually answer the question. Too large (e.g., whole documents) dilutes the embedding's specificity (a huge chunk's embedding is an average/blend of many different topics, making it less discriminative for a narrow query) and wastes context-window budget on irrelevant surrounding text once retrieved. Common practice uses moderate chunk sizes (a few hundred tokens) with some overlap between consecutive chunks (so a fact split across a chunk boundary isn't lost entirely) and, where possible, chunking along natural semantic boundaries (paragraphs, sections) rather than a fixed character count blindly.

Q155. How does a vector database perform fast similarity search across potentially billions of high-dimensional embeddings, and what's the key trade-off in the indexing algorithm typically used? Exact nearest-neighbor search (comparing a query vector against every stored vector) doesn't scale past a modest dataset size. Production vector databases use Approximate Nearest Neighbor (ANN) algorithms — most commonly HNSW (Hierarchical Navigable Small World), which builds a multi-layer graph structure where higher layers have fewer, more widely-spaced points (enabling fast coarse navigation) and lower layers are denser (enabling fine-grained accuracy), letting a search traverse from a coarse starting point down to a precise neighborhood in roughly logarithmic steps rather than a linear scan. The core trade-off is recall vs. speed: ANN algorithms can (rarely) miss the true nearest neighbor in exchange for dramatically faster search, and this trade-off is tunable (e.g., HNSW's `ef_search` parameter) — for most RAG use cases, near-exact recall is an acceptable trade for orders-of-magnitude faster retrieval, since the LLM generation

step downstream is far more forgiving of “close enough, highly relevant” context than a use case needing provably exact results.

Q156. What are the main scalability and cost considerations specific to serving LLM-based systems, beyond typical HLD concerns? LLM inference is far more compute-expensive per request than a typical API call (GPU-bound, often hundreds of milliseconds to seconds per generation, with cost scaling by token count in and out), which changes several design defaults: caching becomes even higher-leverage (semantic/exact caching of common queries can avoid re-running expensive inference for repeated or near- duplicate questions), request batching at the inference layer matters a lot for GPU utilization/cost efficiency (grouping concurrent requests to run through the model together, rather than one-at-a-time), and streaming responses (sending generated tokens to the client incrementally as they’re produced) is standard practice specifically to keep perceived latency low despite the genuinely long total generation time, rather than making the user wait for the entire response to complete server-side first.

Case Study 18: Design an E-Commerce Order & Inventory System

Q157b. What is the core consistency challenge in this system, and where does it most often bite in real interviews? Preventing overselling: inventory count must never go negative because two orders raced to buy the last unit of a low-stock item. This is structurally the same conditional-update problem as ticket booking (Q142): `UPDATE inventory SET quantity = quantity - 1 WHERE product_id = ? AND quantity > 0`, an atomic compare-and- decrement, so only one of two concurrent order attempts for the last unit actually succeeds. Interviewers specifically probe whether you reach for this pattern instinctively, since a naive “read quantity, check > 0, then write” is a textbook race condition.

Q158b. How do you architect the order flow so that inventory reservation, payment, and order confirmation don’t need a single giant distributed transaction? Use the Saga pattern (Q49): reserve inventory (local transaction, short-lived hold like Q143) → charge payment (calls payment service) → confirm order (converts the hold to a committed decrement) → on any step failing, run compensating actions for prior steps (release the inventory hold if payment fails). This keeps each service’s local data consistent via its own ACID transaction, while the overall multi-step business process accepts eventual consistency and explicit compensation instead of a fragile cross-service 2PC.

Q159b. How would you design the system to handle flash sales (sudden, massive demand for one product)? Same shape as the ticket-booking hot-seat problem (Q141-Q144): isolate the hot inventory row’s contention from the rest of the system (a flash-sale product shouldn’t slow down unrelated checkouts), consider pre-decrementing a fast in- memory counter (Redis `DECR`, which is atomic) as the first gate before ever touching the relational DB, and put a queueing/waiting-room layer in front if expected demand vastly exceeds stock, so the backend only processes a controlled admission rate rather than every hopeful buyer’s request simultaneously.

Q160b. How is the product catalog (browsing/search) architecturally separated from the order/inventory write path, and why does that separation matter? Catalog browsing is extremely read-heavy, latency-sensitive, and tolerant of slight staleness (seeing a product description update a few seconds late is a non-issue) — so it's served from a heavily cached, denormalized read store (often a search index like Elasticsearch for filtering/search, backed by a CDN-cached product API). Order/inventory is comparatively low-volume but correctness-critical, needing strong consistency. Serving both from the same tightly-consistent transactional store would force the read-heavy catalog traffic to pay the latency/locking cost the write-critical order path actually needs — this is a direct real-world application of the CQRS idea (Q65): separate read and write models, each optimized for its own access pattern. Case Study 19: Design a Distributed Lock Manager

Q161b. What is a distributed lock, and why is naive single-machine locking (e.g., a language-level mutex) insufficient in a distributed system? A distributed lock ensures mutual exclusion for a resource across multiple independent processes/machines — e.g., ensuring only one instance of a horizontally-scaled service processes a given batch job at a time. A language-level mutex only coordinates threads within a single process's memory space; it has no visibility into other machines entirely, so two instances on two different hosts would both happily acquire “the lock” simultaneously since each only sees its own local state.

Q162b. Describe how Redis (via the SETNX/SET-with-NX pattern) is commonly used to implement a simple distributed lock, and the key failure mode to guard against. SET lock_key unique_client_id NX PX 30000 atomically sets the key only if it doesn't already exist (NX), with a 30-second expiry (PX), acquiring the lock. To release, the client checks that the stored value still matches its own unique ID before deleting (via a Lua script for atomicity) — this prevents a client from accidentally releasing a lock it no longer owns (e.g., if its own lock expired and was re-acquired by a different client in the meantime). The key failure mode: if the lock holder is slow (e.g., a long GC pause) and its lock expires before it finishes its critical work, another client can acquire the lock while the first client is still (unknowingly) operating under the false assumption that it still holds exclusive access — a real, documented weakness of simple TTL-based locks that must be mitigated with fencing tokens (see next question) for any use case where this matters.

Q163b. What is a fencing token, and what problem does it solve that a simple lock (even a “correct” one) cannot? A fencing token is a monotonically increasing number issued by the lock service every time the lock is granted. Instead of the protected resource (e.g., a storage service) merely trusting “whoever currently holds the lock,” it requires every write to include the fencing token, and rejects any write with a token lower than the highest it's already seen. This solves the exact failure mode from Q162b: even if a “zombie” client (whose lock actually expired) sends a delayed write after a new client has already acquired the lock and written with a higher token, the resource can detect the zombie's token is stale and reject it — the correctness now

lives in the resource itself, not merely in trusting that only one client believes it holds the lock at a time.

Q164b. Why is Redlock (the multi-Redis-instance distributed lock algorithm) considered controversial in the distributed systems community? Redlock acquires a lock across a majority of N independent Redis instances to tolerate a single instance's failure. The controversy (notably raised by Martin Kleppmann against a rebuttal from Redis's creator) centers on Redlock relying on reasonably-synchronized clocks and bounded process pauses for its safety guarantees — in the real world, clock drift and unpredictable pauses (GC, VM migration, network delays) can violate these assumptions in ways that undermine the "safety" the algorithm claims to provide, especially compared to a lock service built on a proper consensus protocol (Zookeeper, etcd using Raft) which doesn't depend on clock/ timing assumptions in the same way. Practical interview takeaway: know that Redis-based locks are a fast, simple, "good enough for most use cases" solution, but a consensus-based lock service (Zookeeper/ etcd) is the answer when the cost of a rare correctness violation is high enough to justify the added operational complexity. Case Study 20: Design a Real-Time Leaderboard (like a gaming leaderboard)

Q165b. What is the core data structure that makes this problem tractable at scale, and why? A Redis sorted set (ZSET) — a data structure that keeps members ordered by an associated score, backed internally by a skip list, giving $O(\log N)$ insertion/ update and $O(\log N + M)$ range retrieval (e.g., "top 10" or "rank of player X"). This is exactly the shape of the problem (maintain a large set of scores, need fast updates and fast ordered range queries) and avoids needing to re-sort a large dataset on every single score change, which a naive "store scores in a DB table, ORDER BY score on every read" approach would effectively be doing under the hood on every leaderboard view at high traffic.

Q166b. How do you get a specific player's current rank efficiently, given the leaderboard may have tens of millions of entries? Redis's ZREVRANK command returns a member's 0-indexed rank directly in $O(\log N)$ time using the sorted set's internal skip-list structure, without needing to scan or count through the other entries — this is precisely why the sorted-set approach beats a naive relational "count rows with a higher score" query, which would require a full table scan (or at best an index scan) rather than a structure purpose-built for this exact operation.

Q167b. How would you shard a leaderboard that's too large for a single Redis instance, and what complication does sharding introduce for "global top 10" queries? Shard by a natural partition key if one exists (e.g., per-region leaderboards, or per- game-mode) — each shard is then independently a manageable sorted set. The complication: if you need one global top-10 across all shards, you can no longer get it from a single ZREVRANGE call — you'd query the top-K from each shard individually and merge the results in the application layer (since the true global top-10 must be a subset of the union of each shard's own top-10, this merge is correct and bounded in cost regardless of total shard count).

Q168b. How do you handle “seasonal” leaderboards that reset periodically (daily/weekly/all-time) without losing historical data? Use separate sorted-set keys per time period (e.g., `leaderboard:daily:2026-09-18` , `leaderboard:weekly:2026-W38` , `leaderboard:alltime`), updating all relevant period-keys on every score event rather than trying to derive a “daily” view from the all-time set after the fact. Old period-keys are simply left alone (immutable historical record) or archived/expired via TTL once no longer needed, and the “reset” for a new period is really just starting to write to a brand-new key — never an in-place deletion/rewrite of the previous period’s data, which naturally preserves history for free.

Case Study 21: Design an API Gateway (as a system in its own right, not just as a component of another design)

Q169b. Beyond simple routing, what are the core responsibilities an API Gateway must implement, and why centralize them here rather than in each backend service? Authentication/token validation, rate limiting (Q44, Q86), request routing to the correct backend service (often based on path or host), protocol translation (e.g., accepting REST from clients, calling gRPC internally), response aggregation (composing a response from multiple backend calls for a single client request, reducing client-side round trips), and observability (centralized request logging/tracing/metrics for every request entering the system). Centralizing these avoids every one of potentially hundreds of backend microservices needing to correctly re-implement identical auth/rate-limiting/logging logic — both reducing duplicated effort and closing the security gap of a service that “forgot” to implement a check properly.

Q170b. How does an API Gateway avoid becoming both a performance bottleneck and a single point of failure, given every single request in the system passes through it? It must be horizontally scaled statelessly (many identical gateway instances behind a load balancer, no gateway instance holding unique state that would make losing it costly) with any needed shared state (rate-limit counters, session data) pushed out to a fast external store (Redis) rather than kept in-process. Because it sits on literally every request’s critical path, its own logic must be kept extremely lean and low-latency (avoid synchronous calls to slow, non-essential services from within the gateway’s core routing path), and it should fail gracefully (if a non-critical downstream check like detailed logging is slow, that shouldn’t block or fail the primary request).

Q171b. What’s the difference between an API Gateway and a Service Mesh, and are they solving the same problem? An API Gateway manages north-south traffic — external client requests entering the system from outside. A Service Mesh (Istio, Linkerd) manages east-west traffic — service-to-service communication within the system, typically via a sidecar proxy deployed alongside each service instance, handling concerns like mutual TLS between internal services, internal service-to-service retries/circuit breaking, and fine-grained internal traffic routing (canary deployments, traffic splitting). They solve related but distinct problems and are commonly used together in a large microservices architecture: the gateway is the

single front door for the outside world, while the mesh handles the (often far higher volume, and differently-trusted) internal service-to-service traffic behind that door.

A3. Low-Level Design (LLD) — OOP, SOLID & Design Patterns

Object-Oriented Design Basics

Q157. What are the four pillars of OOP, and why do interviewers care about them in an LLD round specifically? Encapsulation — bundling data and the methods that operate on it, hiding internal state behind a controlled interface. Abstraction — exposing only relevant behavior while hiding implementation complexity. Inheritance — a class acquiring behavior/structure from a parent class. Polymorphism — objects of different types responding to the same interface/method call in type-specific ways. LLD interviews care about these because the round is really testing whether you can translate a fuzzy real-world problem (“design a parking lot”) into a clean, extensible class model — and violations of these principles are exactly what produce the rigid, hard-to-extend code that a good LLD answer is judged against.

```
class Account {
    private double balance; // encapsulation: no outside code can set balance directly

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }

    public double getBalance() { return balance; } // controlled read access
}

interface Shape {
    // abstraction: callers only see area(), not how it's computed
    double area();
}

class Circle implements Shape { // inheritance-style contract via interface
    private final double radius;
    Circle(double radius) { this.radius = radius; }
    public double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    private final double w, h;
    Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double area() { return w * h; }
}

double totalArea(List<Shape> shapes) { // polymorphism: same call, different behavior per type
    return shapes.stream().mapToDouble(Shape::area).sum();
}
```

Walking through the code. `Account.balance` is `private`, and the only way to change it is through `deposit()`, which validates its input first — that’s encapsulation: the class protects its own invariant (balance never goes negative from a bad call) instead of trusting every caller to remember to check. `Shape` is an abstraction: `totalArea()` doesn’t know or care whether it’s summing circles or rectangles, only that each one can answer `area()`. `Circle` and `Rectangle` each implement that contract independently — that’s the inheritance-of-contract piece (via `implements`, since these are unrelated shapes with no shared state to inherit, an interface is the right tool, not a base class — see Q158). The polymorphism shows up at the call site in `totalArea()`: `shapes.get(i).area()` dispatches to `Circle.area()` or `Rectangle.area()` depending on the actual runtime type of each element, even though the loop body never branches on type itself. This is exactly the shape (a common interface, no `if (shape instanceof Circle)` checks anywhere) that an LLD interviewer is listening for in every case study in this guide.

Q158. What’s the difference between an abstract class and an interface, and when would you choose one over the other? An interface defines a contract (method signatures) with no implementation (in most languages; some allow default methods), and a class can implement multiple interfaces — used when unrelated classes need to guarantee they support a certain capability (e.g., `Comparable`, `Serializable`). An abstract class can provide partial implementation (shared state or common method logic) alongside abstract methods that subclasses must implement, but a class can typically only extend one abstract class (single inheritance in most OOP languages) — used when there’s a genuine “is-a” hierarchy with meaningful shared behavior to inherit, not just a shared contract. The practical rule of thumb: prefer interfaces for defining capabilities across otherwise-unrelated types; use abstract classes when you have real shared implementation to avoid duplicating across subclasses.

```
abstract class Employee {                                // shared state + shared logic → abstract class
    protected String name;
    protected double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    double monthlyPay() {                                // shared, concrete logic – not repeated per
subtype
        return baseSalary + bonus();
    }

    abstract double bonus();                             // subtype-specific – must be implemented
}

class Engineer extends Employee {
    Engineer(String name, double baseSalary) { super(name, baseSalary); }
    double bonus() { return baseSalary * 0.10; }
}

interface Payable {                                     // pure capability contract, no shared state
```

```

        double amountDue();
    }

    class Invoice implements Payable, Comparable<Invoice> { // multiple interfaces, no relation
        to Employee
        double amount;
        public double amountDue() { return amount; }
        public int compareTo(Invoice other) { return Double.compare(amount, other.amount); }
    }

```

Walking through the code. `Employee` is abstract because every employee genuinely shares state (`name` , `baseSalary`) and a real, non-trivial method (`monthlyPay()`) that every subclass should get for free rather than reimplement — that’s the “real shared implementation” the rule of thumb calls out. `bonus()` stays abstract because the calculation genuinely differs per employee type; `Engineer` is forced to supply it. `Invoice` , by contrast, has nothing in common with `Employee` — it just needs to guarantee it can report an amount due (`Payable`) and be sortable (`Comparable`). Since Java only allows single class inheritance, `Invoice` couldn’t extend two base classes even if that made sense — but it can freely `implements` as many interfaces as it needs, which is precisely why interfaces are the tool for “unrelated types, shared capability” and abstract classes are the tool for “related types, shared implementation.”

Q159. What is composition over inheritance, and why is it generally preferred in modern object-oriented design? Composition builds behavior by including instances of other classes as fields (a `Car` has an `Engine`) rather than inheriting from a base class (a `Car` is a `Vehicle`). It’s generally preferred because inheritance creates tight coupling to a specific parent’s implementation (changes to a base class can silently break subclasses — the “fragile base class” problem) and forces a rigid hierarchy decided upfront, whereas composition allows behavior to be swapped at runtime (inject a different `Engine` implementation) and avoids deep, brittle inheritance chains. The common guidance isn’t “never use inheritance” but “use inheritance only for genuine is-a relationships with stable, well-understood hierarchies, and default to composition otherwise.”

```

interface Engine {
    int horsepower();
}

class ElectricEngine implements Engine {
    public int horsepower() { return 300; }
}

class GasEngine implements Engine {
    public int horsepower() { return 250; }
}

class Car {
    private final Engine engine; // composition: Car HAS-A Engine, not IS-A Engine

    Car(Engine engine) { this.engine = engine; } // injected – swappable at construction or runtime

    int power() { return engine.horsepower(); }
}

```

```

}

Car tesla = new Car(new ElectricEngine()); // same Car class, different behavior via
composition
Car civic = new Car(new GasEngine());

```

Walking through the code. `Car` never extends `Engine` and never extends some `ElectricCar / GasCar` hierarchy — it holds a reference to an `Engine` and delegates to it in `power()`. Swapping `tesla`’s behavior from electric to gas doesn’t require a different class or touching `Car` at all; it just means constructing it with a different `Engine` implementation. Contrast this with an inheritance-based design (`class ElectricCar extends Car` , `class GasCar extends Car`) — that would lock the engine type in at compile time per subclass, and any shared `Car` logic that later needs to change would risk breaking both subclasses simultaneously (the fragile base class problem the answer describes). This is also exactly the shape used for `PaymentStrategy` , `SplitStrategy` , and every other “Strategy” pattern in this guide — composition-over-inheritance is the general principle; Strategy (Q179) is its most common named application.

Q160. What is the difference between association, aggregation, and composition as relationships between classes? Association — the most general relationship: one class uses or references another, with independent lifecycles (e.g., a `Student` and a `Teacher` both existing independently, aware of each other). Aggregation — a “has-a” whole-part relationship where the part can exist independently of the whole (e.g., a `Department` has `Professor` s, but a `Professor` still exists if the department is dissolved). Composition — a stronger “has-a” relationship where the part’s lifecycle is bound to the whole (e.g., a `House` has `Room` s — a room doesn’t meaningfully exist independent of its house; destroying the house destroys the rooms). This distinction matters in LLD interviews for correctly modeling object lifecycles and ownership.

```

class Teacher { String name; }

class Student {                               // ASSOCIATION – independent lifecycles
    List<Teacher> teachers;                     // Student references Teachers, neither owns the other
}

class Professor { String name; }

class Department {                             // AGGREGATION – Professors outlive the Department
    List<Professor> professors;                 // passed in / shared; Department doesn't create them
    void addProfessor(Professor p) { professors.add(p); }
}

class Room { String label; }

class House {                                  // COMPOSITION – Rooms are owned and created by House
    private final List<Room> rooms = new ArrayList<>();
    House(List<String> roomLabels) {
        for (String label : roomLabels) rooms.add(new Room(label)); // House creates its own
        Rooms
    }
    // no addRoom() from outside, no way to hand a Room a separate lifecycle – it dies with

```

```
House
}
```

Walking through the code. The tell in each case is *who constructs the part, and what happens to it if the whole goes away*. `Student.teachers` just holds references to `Teacher` objects that were created elsewhere and will keep existing whether or not this particular `Student` object does — that's association. `Department.professors` is populated via `addProfessor(Professor p)`, taking an already-existing `Professor` from outside — the department doesn't create its professors and firing the department (dropping the `Department` object) doesn't delete the people — that's aggregation. `House`, on the other hand, constructs its own `Room` objects internally in its constructor and exposes no way to hand a `Room` an independent existence outside a `House` — if the `House` object is discarded, nothing else in the program can still be holding a meaningful reference to one of its `Room`s, mirroring how a room doesn't exist independent of the house it's part of. Getting this distinction right in an interview shows up in whether your constructors take fully-formed collaborator objects (aggregation) or build their own internal parts (composition).

SOLID Principles

Q161. Explain the Single Responsibility Principle (SRP) with an example of a violation and its fix. SRP states a class should have only one reason to change — i.e., one responsibility. Violation example: a `User` class that both holds user data and contains `saveToDatabase()` and `sendWelcomeEmail()` methods — now the class changes for three unrelated reasons (a change to user data fields, a change to database persistence logic, a change to email templates). Fix: extract `UserRepository` (handles persistence) and `EmailService` (handles email), leaving `User` as a pure data/behavior model of a user, with the other two collaborating with it rather than being embedded in it.

```
// VIOLATION - three reasons to change bundled into one class
class User {
    String name, email;
    void saveToDatabase() { /* JDBC / ORM logic here */ }
    void sendWelcomeEmail() { /* SMTP / template logic here */ }
}

// FIX - each responsibility isolated in its own class
class User {
    String name, email; // User is now just a data/behavior model
}

class UserRepository {
    void save(User user) { /* JDBC / ORM logic here */ }
}

class EmailService {
    void sendWelcomeEmail(User user) { /* SMTP / template logic here */ }
}

// collaborating code wires them together - User itself stays untouched by either concern
```

```
UserRepository repo = new UserRepository();
EmailService emailService = new EmailService();
User u = new User();
repo.save(u);
emailService.sendWelcomeEmail(u);
```

Walking through the code. In the violation, a change to *how* users are persisted (switching from JDBC to an ORM, say) requires editing `User` — the same class that also defines what a user *is*. So does a change to the welcome email's wording. Three unrelated teams (data modeling, persistence, notifications) would all be editing the same file for unrelated reasons, and a bug introduced while changing the email template risks breaking `saveToDatabase()` through sheer proximity. In the fix, `User` no longer has a `saveToDatabase()` or `sendWelcomeEmail()` method at all — it's reduced to representing user data. `UserRepository` and `EmailService` each take a `User` as a parameter rather than owning one, so `User` never needs to know either of them exists. Each class now has exactly one reason to change: `User` changes only if the shape of user data changes, `UserRepository` only if persistence logic changes, `EmailService` only if email logic changes.

Q162. Explain the Open/Closed Principle (OCP) with a concrete example. A class/module should be open for extension but closed for modification — you should be able to add new behavior without editing existing, tested code. Classic violation: a `PaymentProcessor` with a method containing `if (type == "credit_card") {...} else if (type == "paypal") {...}` — adding a new payment type requires modifying this existing method (and risking breaking the existing cases). Fix: define a `PaymentMethod` interface with a `process()` method, and have `CreditCardPayment`, `PayPalPayment`, etc. each implement it separately — adding a new payment type means adding a new class, never touching the existing, already-tested ones.

```
// VIOLATION - every new payment type means editing this method again
class PaymentProcessor {
    void pay(String type, double amount) {
        if (type.equals("credit_card")) {
            // charge credit card
        } else if (type.equals("paypal")) {
            // charge via PayPal
        }
        // adding "apple_pay" means adding another else-if HERE, risking the existing
        // branches
    }
}

// FIX - new payment types are new classes, this method never changes again
interface PaymentMethod {
    void process(double amount);
}

class CreditCardPayment implements PaymentMethod {
    public void process(double amount) { /* charge credit card */ }
}

class PayPalPayment implements PaymentMethod {
    public void process(double amount) { /* charge via PayPal */ }
```

```

}

class ApplePayPayment implements PaymentMethod { // NEW type – zero changes to existing
code
    public void process(double amount) { /* charge via Apple Pay */ }
}

class PaymentProcessor {
    void pay(PaymentMethod method, double amount) {
        method.process(amount); // never branches on type – closed for modification
    }
}

```

Walking through the code. The violation’s `pay()` method is the single place every payment type’s logic lives, so every new type is a new branch added to an already-tested method — and a typo or logic slip in the new branch sits right next to, and can accidentally affect, the working branches above it. In the fix, `pay(PaymentMethod method, double amount)` doesn’t know or care which concrete implementation it received — it just calls `process()` polymorphically. Adding `ApplePayPayment` is purely additive: a brand-new class that implements the existing interface, with the `PaymentProcessor` class itself never touched, recompiled logic-wise, or re-tested for regressions in the existing credit-card and PayPal paths. That’s “open for extension” (new classes are welcome) and “closed for modification” (the dispatching class’s source doesn’t change) at the same time.

Q163. Explain the Liskov Substitution Principle (LSP), and give the classic example of a violation. Subtypes must be substitutable for their base type without altering the correctness of the program — i.e., any code that works with a base class should work correctly if handed a subclass instance instead, without needing to know the difference. The classic violation is `Square` extends `Rectangle`: if `Rectangle` has independent `setWidth()` / `setHeight()` methods, a `Square` subclass that overrides both to keep `width=height` together breaks any code that sets width and height independently expecting rectangle-like behavior (e.g., code that sets width to 5 and height to 10 expecting area 50 would get area 100 for a `Square` instance) — the subclass looks like an “is-a” relationship geometrically, but violates the behavioral contract the base class’s clients depend on.

```

// VIOLATION
class Rectangle {
    protected int width, height;
    void setWidth(int w) { this.width = w; }
    void setHeight(int h) { this.height = h; }
    int area() { return width * height; }
}

class Square extends Rectangle {
    @Override void setWidth(int w) { width = w; height = w; } // silently changes height
too
    @Override void setHeight(int h) { width = h; height = h; } // silently changes width too
}

void testArea(Rectangle r) {
    r.setWidth(5);
}

```

```

    r.setHeight(10);
    assert r.area() == 50;    // passes for Rectangle, FAILS for Square (area is 100)
}

// FIX – don't force an is-a relationship the behavioral contract can't honor
interface Shape {
    int area();
}

class RectangleShape implements Shape {
    private final int width, height;           // immutable – no independent-setter
    contract to violate
    RectangleShape(int width, int height) { this.width = width; this.height = height; }
    public int area() { return width * height; }
}

class SquareShape implements Shape {
    private final int side;
    SquareShape(int side) { this.side = side; }
    public int area() { return side * side; }
}

```

Walking through the code. `testArea()` is written against `Rectangle`'s documented contract: `setWidth` and `setHeight` are independent, so setting them to 5 and 10 should always yield area 50 regardless of which subtype was actually passed in — that's what "substitutable" means. `Square.setWidth()` breaks that contract by mutating a field the caller didn't touch, so the exact same test that passes for a plain `Rectangle` silently produces the wrong answer for a `Square`, even though `Square` compiles fine as a `Rectangle` and satisfies `instanceof Rectangle`. The fix doesn't try to patch `Square`'s setters — it removes the false "is-a" relationship entirely. `RectangleShape` and `SquareShape` both implement `Shape` and are immutable (constructed once, no setters at all), so there's no independent-setter contract to accidentally violate in the first place; each type only promises what it can actually deliver.

Q164. Explain the Interface Segregation Principle (ISP), and why "fat interfaces" are a problem. Clients shouldn't be forced to depend on methods they don't use — many small, specific interfaces are better than one large, general-purpose one. Violation example: a single `Worker` interface with `work()` and `eat()` methods, implemented by both `HumanWorker` and `RobotWorker` — the robot is forced to implement `eat()` meaningfully even though it's nonsensical for it (leading to an empty or exception-throwing method, a code smell signaling the interface is wrong). Fix: split into separate `Workable` and `Eatable` interfaces, and have `RobotWorker` implement only `Workable`.

```

// VIOLATION – fat interface forces RobotWorker to implement a method that makes no sense for it
interface Worker {
    void work();
    void eat();
}

class RobotWorker implements Worker {
    public void work() { /* do work */ }
    public void eat() {

```

```

        throw new UnsupportedOperationException("Robots don't eat"); // code smell: forced
no-op / exception
    }
}

// FIX – segregated, capability-specific interfaces
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class HumanWorker implements Workable, Eatable {
    public void work() { /* do work */ }
    public void eat() { /* take a lunch break */ }
}

class RobotWorker implements Workable { // only implements what actually applies
    public void work() { /* do work */ }
}

```

Walking through the code. In the violation, any code that just wants “something that can `work()`” is forced to depend on the full `Worker` interface — including `eat()` — even though a caller iterating over a list of `Workers` to call `work()` has no legitimate use for `eat()` on a robot. The `throw new UnsupportedOperationException` inside `RobotWorker.eat()` is the tell: the class is being forced to promise something it can’t honor, purely because the interface bundled two unrelated capabilities together. After segregating into `Workable` and `Eatable`, `RobotWorker` implements `Workable` only — there’s no method to stub out, no exception to throw, and no way for calling code to even attempt `robotWorker.eat()` since that method doesn’t exist on the type at all. `HumanWorker` implements both, because both genuinely apply to it.

Q165. Explain the Dependency Inversion Principle (DIP), and how it differs from simple dependency injection. High-level modules shouldn’t depend on low-level modules directly — both should depend on abstractions (interfaces), and the abstraction shouldn’t depend on implementation details. Violation: a `NotificationService` class that directly instantiates and calls `new EmailSender()` internally — it’s now tightly coupled to that one concrete implementation, hard to test (can’t substitute a mock) and hard to extend (adding SMS notification means modifying this class). Fix: `NotificationService` depends on an abstract `MessageSender` interface, injected in from outside (via constructor, typically). DIP is the principle (depend on abstractions); dependency injection is one common technique used to satisfy it (supplying the concrete implementation from outside rather than the class constructing its own dependency) — DI is how you achieve DIP in practice, but DIP itself is the broader architectural rule.

```

// VIOLATION – high-level NotificationService depends directly on a concrete low-level class
class EmailSender {
    void send(String to, String message) { /* SMTP logic */ }
}

```

```

class NotificationService {
    private final EmailSender sender = new EmailSender(); // hardcoded – can't swap, can't
    mock
    void notify(String user, String message) { sender.send(user, message); }
}

// FIX – both depend on an abstraction; concrete type is supplied from outside
interface MessageSender {
    void send(String to, String message);
}

class EmailSender implements MessageSender {
    public void send(String to, String message) { /* SMTP logic */ }
}

class SmsSender implements MessageSender { // new capability – no change to
    NotificationService
    public void send(String to, String message) { /* SMS gateway logic */ }
}

class NotificationService {
    private final MessageSender sender; // depends on the interface, not a
    concrete class

    NotificationService(MessageSender sender) { // dependency injection: supplied from
    outside
        this.sender = sender;
    }

    void notify(String user, String message) { sender.send(user, message); }
}

NotificationService emailNotifier = new NotificationService(new EmailSender());
NotificationService smsNotifier = new NotificationService(new SmsSender());

```

Walking through the code. In the violation, `new EmailSender()` appears inside `NotificationService` itself — the high-level policy class (deciding *when* to notify) is welded to one specific low-level mechanism (*how* — via SMTP email). A unit test for `NotificationService` can't substitute a fake sender to verify `notify()` was called correctly without actually hitting SMTP infrastructure, and adding SMS support means opening up and editing `NotificationService`'s source. In the fix, `NotificationService`'s constructor takes a `MessageSender` — it depends only on the interface, never on `EmailSender` or `SmsSender` by name. The concrete choice is made once, outside the class, at the call site (`new NotificationService(new EmailSender())`), which is the dependency injection technique. Note the distinction the answer draws: DIP is satisfied the moment `NotificationService` stops referencing a concrete class internally and depends on `MessageSender` instead; the constructor parameter is simply the specific mechanism (DI) used to hand it a concrete instance to fulfill that abstraction at runtime.

Design Patterns — Creational

Q166. Explain the Singleton pattern, and what problems can it cause if overused? Singleton ensures a class has exactly one instance and provides a global

access point to it — used for shared resources like a configuration manager, logging service, or connection pool where having multiple instances would be wasteful or incorrect. Problems with overuse: it introduces global mutable state (hard to reason about, hard to test in isolation since tests can leak state between them via the shared singleton), hides dependencies (a class silently reaching for `Singleton.getInstance()` internally is less honest/testable than the dependency being explicitly passed in via DI), and can become a concurrency bottleneck/bug source if not made properly thread-safe. Modern guidance often prefers dependency injection frameworks managing a single instance's lifecycle (effectively achieving the same “one instance” goal) over the classic self-enforcing Singleton pattern with its global static access.

```
class ConfigManager {
    private static ConfigManager instance;           // the one shared instance, held
statically
    private Map<String, String> settings;

    private ConfigManager()
    {
        // private constructor - can't be `new`'d from outside
        settings = loadFromDisk();
    }

    public static ConfigManager getInstance() {      // global access point
        if (instance == null) {
            instance = new ConfigManager();
        }
        return instance;
    }

    public String get(String key) { return settings.get(key); }
}

// anywhere in the codebase:
String dbUrl = ConfigManager.getInstance().get("db.url");
```

Walking through the code. The constructor is `private`, so the only way to obtain a `ConfigManager` anywhere in the program is through `getInstance()` — there is no `new ConfigManager()` available outside the class itself. `getInstance()` lazily creates the instance the first time it's called (if `(instance == null)`) and returns that same object reference on every subsequent call, which is what “exactly one instance” means in practice: two different parts of the codebase calling `getInstance()` get back the identical object, sharing the same `settings` map. That sharing is also exactly the risk the answer flags — a unit test that mutates `settings` via one code path can leave state that a completely unrelated test observes later, since both tests are silently operating on the same global object rather than fresh, isolated instances.

Q167. How do you implement a thread-safe Singleton, and what is “double-checked locking”? A naive lazy singleton (`if (instance == null) instance = new Singleton();`) is not thread-safe — two threads could both pass the null check before either finishes construction, creating two instances. A simple fix is synchronizing the entire method, but that adds unnecessary lock contention on every call after the instance already exists. Double-checked locking checks null, then acquires a lock, then

checks null again inside the lock before constructing — the second check prevents duplicate construction from a race, while the outer unsynchronized check avoids paying the lock's cost on every subsequent call once the instance already exists (the common case). The instance field must be declared `volatile` (in Java, for example) to prevent instruction reordering issues where another thread could see a partially-constructed object.

```
class ConfigManager {
    private static volatile ConfigManager instance;    // volatile - prevents reordering/
                                                       // stale-read bugs
    private Map<String, String> settings;

    private ConfigManager() { settings = loadFromDisk(); }

    public static ConfigManager getInstance() {
        if (instance == null) {                        // first check - no lock, fast path
                                                       // once initialized
            synchronized (ConfigManager.class) {      // lock only taken during the rare
                                                       // init race window
                if (instance == null) {                // second check - inside the lock,
                                                       // prevents double-create
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}
```

Walking through the code. The outer `if (instance == null)` runs on every single call to `getInstance()`, uncontended and lock-free — once `instance` is set, this check fails fast and the method returns immediately, which is why this pattern avoids the performance cost of synchronizing the entire method on every call. The `synchronized` block is only entered on the rare occasion the outer check finds `instance` still null (i.e., during the brief window before the first construction completes). Inside the lock, the check runs again — this second check is what actually prevents two threads racing: if Thread A already constructed the instance while Thread B was waiting to acquire the lock, Thread B's inner check now sees a non-null `instance` and skips constructing a second one. The `volatile` keyword matters because, without it, the JVM (or CPU) is allowed to reorder the instructions inside `new ConfigManager()` such that `instance` could be made non-null *before* the constructor has fully finished populating `settings` — a second thread could then read a non-null but only partially-constructed object. `volatile` establishes a happens-before relationship that rules this out.

Q168. Explain the Factory Method pattern and how it differs from a simple constructor call. Factory Method defines an interface/abstract method for creating an object, letting subclasses decide which concrete class to instantiate — the calling code depends only on the abstract product type, not the concrete class. This differs from calling `new ConcreteClass()` directly because it decouples the client from knowing which specific subclass to instantiate (that decision can depend on configuration, input parameters, or subclass-specific logic), satisfying the Open/Closed Principle — adding a

new product type means adding a new factory subclass, not modifying client code that creates objects.

```
interface Notification {
    void send(String message);
}

class EmailNotification implements Notification {
    public void send(String message) { /* send via email */ }
}

class SmsNotification implements Notification {
    public void send(String message) { /* send via SMS */ }
}

abstract class NotificationFactory {
    abstract Notification
    createNotification();           // the "factory method" – subclasses decide the type

    void notify(String message) {           // shared logic, doesn't know the
        concrete type
        Notification n = createNotification();
        n.send(message);
    }
}

class EmailNotificationFactory extends NotificationFactory {
    Notification createNotification() { return new EmailNotification(); }
}

class SmsNotificationFactory extends NotificationFactory {
    Notification createNotification() { return new SmsNotification(); }
}

NotificationFactory factory = new EmailNotificationFactory(); // client picks the factory
once
factory.notify("Your order has shipped");                       // never mentions
EmailNotification directly
```

Walking through the code. `NotificationFactory.notify()` is written entirely against the `Notification` interface — it calls `createNotification()` and then `n.send(message)`, and never references `EmailNotification` or `SmsNotification` by name. The decision of *which* concrete class gets instantiated is deferred to `createNotification()`, an abstract method each subclass overrides. `EmailNotificationFactory` overrides it to return a `new EmailNotification()`; a hypothetical `PushNotificationFactory` could be added later, overriding `createNotification()` to return a `new PushNotification()`, without a single line of `notify()` or `NotificationFactory` changing. Contrast this with client code that simply calls `new EmailNotification()` directly wherever a notification is needed — that client is now hardwired to that one concrete class at every call site, whereas here the concrete choice is centralized in one small overridden method per factory subclass.

Q169. Explain the Abstract Factory pattern, and how it differs from Factory Method. Abstract Factory provides an interface for creating families of related objects without specifying their concrete classes — e.g., a `UIFactory` interface with `createButton()` and `createCheckbox()` methods, implemented by `WindowsUIFactory` and

`MacUIFactory`, guaranteeing that all UI elements produced by a single factory instance are visually/behaviorally consistent with each other (you never accidentally mix a Windows button with a Mac checkbox). The difference from Factory Method: Factory Method is about creating one product via a single overridden method (usually via subclassing); Abstract Factory is about creating a whole family of related products via a factory object with multiple creation methods, typically using composition (an Abstract Factory implementation often uses several Factory Methods internally).

```
interface Button { void render(); }
interface Checkbox { void render(); }

class WindowsButton implements Button { public void render() { /* draw Windows-style button */ } }
class WindowsCheckbox implements Checkbox { public void render() { /* draw Windows-style checkbox */ } }
class MacButton implements Button { public void render() { /* draw Mac-style button */ } }
class MacCheckbox implements Checkbox { public void render() { /* draw Mac-style checkbox */ } }

interface UIFactory {                                // one factory, multiple related products
    Button createButton();
    Checkbox createCheckbox();
}

class WindowsUIFactory implements UIFactory {
    public Button createButton() { return new WindowsButton(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}

class MacUIFactory implements UIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}

void renderForm(UIFactory factory) {
    Button b = factory.createButton();                // guaranteed to match the same family as...
    Checkbox c = factory.createCheckbox();            // ...this checkbox – never mixed
    b.render();
    c.render();
}
```

Walking through the code. `UIFactory` bundles two creation methods, `createButton()` and `createCheckbox()`, rather than one — that bundling is the entire point: whichever concrete `UIFactory` implementation `renderForm()` is handed, both the button and checkbox it produces come from that same implementation, so they're guaranteed to be visually consistent. If `renderForm()` is called with a `WindowsUIFactory`, it's structurally impossible to end up with a `WindowsButton` alongside a `MacCheckbox`, because both objects came from calling methods on the exact same factory instance. Compare this to Factory Method (Q168), where `NotificationFactory` only ever produced one kind of product per factory subclass — Abstract Factory's defining feature is that a single factory object is responsible for an entire *family* of related products that need to stay consistent with each other.

Q170. Explain the Builder pattern, and when it's preferable to a large constructor with many parameters ("telescoping constructor" problem). Builder separates the construction of a complex object from its representation, using a step-by-step fluent interface to set only the parameters you care about, then a final `build()` call to produce the finished, often immutable object. It's preferable when a class has many optional parameters/configuration — a constructor with 10 parameters (many optional, easy to mix up positionally, e.g., accidentally swapping two same-typed arguments) is error-prone and unreadable at the call site, whereas `new PizzaBuilder().size(LARGE).addTopping(CHEESE).addTopping(PEPPERONI).build()` is self-documenting and lets callers specify only what's relevant to them, with sensible defaults for everything else.

```
class Pizza {
    private final String size;           // required
    private final List<String> toppings; // optional, defaults to empty
    private final boolean extraCheese;   // optional, defaults to false

    private Pizza(Builder b) {           // constructor is private – only Builder can
call it
        this.size = b.size;
        this.toppings = b.toppings;
        this.extraCheese = b.extraCheese;
    }

    static class Builder {
        private String size;
        private List<String> toppings = new ArrayList<>();
        private boolean extraCheese = false;

        Builder size(String size) { this.size = size; return this; } // returns
`this` – fluent chaining
        Builder addTopping(String topping) { toppings.add(topping); return this; }
        Builder extraCheese(boolean value) { this.extraCheese = value; return this; }

        Pizza build() {
            if (size == null) throw new IllegalStateException("size is required");
            return new Pizza(this); // validated once, here, not scattered
across callers
        }
    }
}

Pizza order = new Pizza.Builder()
    .size("LARGE")
    .addTopping("CHEESE")
    .addTopping("PEPPERONI")
    .extraCheese(true)
    .build();
```

Walking through the code. `Pizza`'s own constructor is `private`, so the only way to create one is through `Builder` — this is what forces every `Pizza` to go through validation in `build()` rather than allowing a caller to construct a half-configured `Pizza` directly. Each setter method on `Builder` (`size()`, `addTopping()`, `extraCheese()`) returns `this`, which is what enables the fluent chaining syntax (`.size(...).addTopping(...).build()`) — without that `return this`, each call would need

to be a separate statement reassigning the builder variable. The caller only calls the methods relevant to their order — nobody has to pass `null` or a default value positionally for toppings they don't want, the way a 6-parameter constructor `new Pizza("LARGE", null, false, null, 0, false)` would require. `build()` is the one place that checks required fields (`size == null`) before construction, centralizing validation that would otherwise need to be duplicated in every constructor overload if you tried to solve the same problem with telescoping constructors.

Q171. Explain the Prototype pattern, and give a real use case. Prototype creates new objects by cloning an existing instance (a “prototype”) rather than constructing from scratch, useful when object creation is expensive (e.g., involves an expensive database call, complex computation, or loading a large resource) and many similar instances are needed with minor variations. Real use case: a game engine spawning many enemy objects that share most configuration (base stats, model, AI behavior) but need individual instance state (position, current health) — cloning a pre-configured prototype and then tweaking the few per-instance fields is far cheaper than fully re-initializing each one from raw configuration every time.

```
class Enemy implements Cloneable {
    String modelName;
    int baseHealth;
    List<String> aiScript; // expensive to load – parsed once, reused across clones
    int positionX, positionY; // per-instance – varies for every spawned enemy

    @Override
    public Enemy clone() {
        try {
            Enemy copy = (Enemy) super.clone(); // shallow copy first
            copy.aiScript = new ArrayList<>(this.aiScript); // then deep-copy the mutable
            parts (Q172)
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e); // Enemy implements Cloneable, so this can't
            actually happen
        }
    }
}

Enemy goblinPrototype =
    loadExpensiveGoblinConfig(); // slow: model load, script parse – done ONCE

Enemy goblin1 = goblinPrototype.clone(); // cheap: no reload, no reparse
goblin1.positionX = 10; goblin1.positionY = 5;

Enemy goblin2 = goblinPrototype.clone(); // cheap
goblin2.positionX = 50; goblin2.positionY = 12;
```

Walking through the code. `loadExpensiveGoblinConfig()` — the operation the answer describes as expensive (model loading, AI script parsing) — runs exactly once, to produce `goblinPrototype`. Every subsequent enemy is produced via `clone()` instead of calling that expensive loader again: `goblin1` and `goblin2` both skip straight past the costly initialization and only pay for copying already-loaded data plus setting the two fields that actually vary per instance (`positionX`, `positionY`). The `clone()` method itself

does the shallow-then-deep-copy dance described in Q172: `super.clone()` copies all fields bit-for-bit first (cheap, but `aiScript` in the copy would otherwise point at the *same* list object as the original), and then `copy.aiScript = new ArrayList<>(this.aiScript)` explicitly deep-copies that one mutable reference field so that mutating one enemy's script list later can't accidentally mutate every other clone's list too.

Q172. What's the difference between shallow copy and deep copy, and why does it matter for the Prototype pattern? A shallow copy duplicates an object's top-level fields, but if a field is itself a reference to another object (or a collection/array), the copy shares that same nested reference with the original — mutating the nested object through the copy also affects the original, which is usually not what's intended when "cloning" something. A deep copy recursively clones nested objects too, so the copy is fully independent. This matters for Prototype specifically because a clone that's meant to be independently mutable (the whole point of cloning something to then customize it) will have subtle, hard-to-debug shared-state bugs if the clone operation is only shallow when the object graph contains mutable nested references.

```
class Inventory {
    List<String> items;
    Inventory(List<String> items) { this.items = items; }
}

class Player implements Cloneable {
    String name;
    Inventory inventory; // reference field – the crux of shallow vs. deep

    Player shallowClone() {
        Player copy = new Player();
        copy.name = this.name;
        copy.inventory = this.inventory; // BUG: copy.inventory IS this.inventory
        (same object)
        return copy;
    }

    Player deepClone() {
        Player copy = new Player();
        copy.name = this.name;
        copy.inventory = new Inventory(new ArrayList<>(this.inventory.items)); // new,
        independent object
        return copy;
    }
}

Player original = new Player();
original.inventory = new Inventory(new ArrayList<>(List.of("sword")));

Player shallow = original.shallowClone();
shallow.inventory.items.add("shield");
// original.inventory.items now ALSO contains "shield" – unintended shared mutation

Player deep = original.deepClone();
deep.inventory.items.add("bow");
// original.inventory.items is unaffected – fully independent
```

Walking through the code. In `shallowClone()`, `copy.inventory = this.inventory` copies the *reference* — after this line, `copy.inventory` and `original.inventory` point at the exact same `Inventory` object in memory, not two separate objects that happen to look alike. That's why `shallow.inventory.items.add("shield")` also changes what `original.inventory.items` contains: there was only ever one `Inventory` object, being read through two different `Player` variables. `deepClone()` avoids this by explicitly constructing a brand-new `Inventory` with a brand-new `ArrayList` copy of the items — after this, `deep.inventory` and `original.inventory` are two distinct objects, so mutating one's item list has no effect on the other's. For the Prototype pattern specifically, this is exactly the bug that would appear if `Enemy.clone()` in Q171 had simply done `copy.aiScript = this.aiScript` instead of `new ArrayList<>(this.aiScript)` — every cloned enemy would silently share and corrupt the same AI script list.

Design Patterns — Structural

Q173. Explain the Adapter pattern with a real-world example. Adapter converts the interface of a class into another interface that a client expects, letting otherwise-incompatible interfaces work together without modifying either side's existing code. Real example: integrating a third-party payment SDK whose method is `chargeCard(cardNum, amount)` into a system whose internal `PaymentProcessor` interface expects a `process(PaymentRequest)` method — you write an adapter class implementing your internal interface, which internally translates the call into the third-party SDK's expected format. This is exactly the pattern used constantly when integrating any external library/API that wasn't designed with your internal interfaces in mind.

```
// third-party SDK - you cannot modify this class
class ThirdPartyPaymentSdk {
    void chargeCard(String cardNumber, double amountCents) { /* their charging logic */ }
}

// your internal interface - the rest of your codebase depends on THIS shape
interface PaymentProcessor {
    void process(PaymentRequest request);
}

class PaymentRequest {
    String cardNumber;
    double amountDollars;
}

class ThirdPartySdkAdapter implements PaymentProcessor {    // bridges the two shapes
    private final ThirdPartyPaymentSdk sdk;

    ThirdPartySdkAdapter(ThirdPartyPaymentSdk sdk) { this.sdk = sdk; }

    public void process(PaymentRequest request) {
        sdk.chargeCard(request.cardNumber, request.amountDollars * 100); // translate
dollars→cents, method name
    }
}

PaymentProcessor processor = new ThirdPartySdkAdapter(new ThirdPartyPaymentSdk());
```

```
processor.process(new PaymentRequest()); // rest of the codebase never sees
ThirdPartyPaymentSdk directly
```

Walking through the code. `ThirdPartyPaymentSdk` is a class you don't control and can't change — its method is named `chargeCard`, takes a cents amount, and doesn't know anything about your `PaymentRequest` type. `ThirdPartySdkAdapter` is the only class in the codebase that mentions `ThirdPartyPaymentSdk` at all: it implements your own `PaymentProcessor` interface, and inside `process()` it does the actual translation — unpacking `PaymentRequest`'s fields and converting dollars to cents before calling the SDK's differently-shaped method. Every other class in the system depends only on `PaymentProcessor`, never on `ThirdPartyPaymentSdk` — so if you later switch payment providers, you write one new adapter class and nothing else in the codebase changes.

Q174. Explain the Decorator pattern, and how it differs from simply subclassing to add behavior. Decorator attaches additional responsibilities to an object dynamically by wrapping it in another object implementing the same interface, which delegates to the wrapped object and adds its own behavior before/after. Unlike subclassing (which fixes the combination of behaviors at compile time — you'd need a new subclass for every combination, e.g., `CoffeeWithMilkAndSugar`), decorators can be composed at runtime in any combination (`new SugarDecorator(new MilkDecorator(new Coffee()))`), avoiding a combinatorial explosion of subclasses for every possible feature combination. Java's `InputStream` wrapping (`new BufferedInputStream(new FileInputStream(...))`) is a canonical real-world example.

```
interface Coffee {
    double cost();
    String description();
}

class PlainCoffee implements Coffee {
    public double cost() { return 2.00; }
    public String description() { return "Coffee"; }
}

abstract class CoffeeDecorator implements Coffee { // itself a Coffee – that's what
    makes stacking work
    protected final Coffee wrapped;
    CoffeeDecorator(Coffee wrapped) { this.wrapped = wrapped; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee wrapped) { super(wrapped); }
    public double cost() { return wrapped.cost() + 0.50; } // delegate, then add
    public String description() { return wrapped.description() + " + Milk"; }
}

class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee wrapped) { super(wrapped); }
    public double cost() { return wrapped.cost() + 0.25; }
    public String description() { return wrapped.description() + " + Sugar"; }
}

Coffee order = new SugarDecorator(new MilkDecorator(new PlainCoffee()));
```

```
order.cost();           // 2.00 + 0.50 + 0.25 = 2.75
order.description();    // "Coffee + Milk + Sugar"
```

Walking through the code. `CoffeeDecorator` implements `Coffee` and *also* holds a reference to another `Coffee` (wrapped) — that dual role (being a `Coffee` while wrapping a `Coffee`) is exactly what lets decorators stack arbitrarily.

`MilkDecorator.cost()` doesn't reimplement pricing from scratch; it calls `wrapped.cost()` and adds its own `0.50` on top, delegating to whatever it's wrapping before adding its own piece. Because `SugarDecorator` also wraps a `Coffee` (not specifically a `PlainCoffee`), it doesn't care whether that inner `Coffee` is a plain one or already itself a `MilkDecorator`-wrapped one — `new SugarDecorator(new MilkDecorator(new PlainCoffee()))` builds a chain where calling `cost()` on the outermost object cascades the call inward through every layer, each adding its own charge, before returning back out. To get "coffee with sugar only" instead, you'd construct `new SugarDecorator(new PlainCoffee())` — no `CoffeeWithSugarOnly` subclass needed, and no combinatorial subclass explosion as more add-ons are introduced.

Q175. Explain the Facade pattern, and what problem it solves. Facade provides a simplified, higher-level interface to a complex subsystem of many interacting classes, hiding the subsystem's internal complexity from client code. It solves the problem of tight coupling between client code and many low-level subsystem classes — instead of a client needing to know how to correctly orchestrate calls across a dozen internal classes in the right order, it calls one simple method on the facade, which handles the orchestration internally. Example: a `VideoConverterFacade.convert(file, format)` method that internally coordinates codec selection, bitrate calculation, audio processing, and file writing classes that a client shouldn't need to understand or touch directly.

```
// subsystem – several low-level classes a client shouldn't have to orchestrate directly
class CodecSelector { String selectCodec(String format) { return "H264"; } }
class BitrateCalculator { int calculate(String file) { return 5000; } }
class AudioProcessor { void process(String file) { /* ... */ } }
class FileWriter { void write(String file, String codec, int bitrate) { /* ... */ } }

class VideoConverterFacade {                               // single simplified entry point
    private final CodecSelector codecSelector = new CodecSelector();
    private final BitrateCalculator bitrateCalculator = new BitrateCalculator();
    private final AudioProcessor audioProcessor = new AudioProcessor();
    private final FileWriter fileWriter = new FileWriter();

    void convert(String file, String format) {
        String codec = codecSelector.selectCodec(format);    // orchestration order
        matters –
        int bitrate =
        bitrateCalculator.calculate(file);                  // the client shouldn't need to know it
        audioProcessor.process(file);
        fileWriter.write(file, codec, bitrate);
    }
}
```

```
new VideoConverterFacade().convert("clip.mov", "mp4"); // client: one call, no subsystem
knowledge needed
```

Walking through the code. Without the facade, a client wanting to convert a video would need to know that codec selection must happen before bitrate calculation, that audio processing is a separate step, and that the file writer needs both the codec and bitrate as inputs — four classes, called in a specific order, with knowledge of how they relate. `VideoConverterFacade.convert()` is the only method the client actually calls; it owns instances of all four subsystem classes internally and calls them in the correct sequence. The calling code at the bottom is reduced to one line with no visibility into `CodecSelector`, `BitrateCalculator`, `AudioProcessor`, or `FileWriter` at all — if the internal orchestration order ever needs to change (say, audio processing needs to happen before bitrate calculation), that change happens entirely inside `VideoConverterFacade`, and every caller is unaffected.

Q176. Explain the Proxy pattern and its common variants (virtual, protection, remote proxy). Proxy provides a surrogate/placeholder for another object, controlling access to it. Virtual proxy — delays expensive object creation until it's actually needed (lazy loading), e.g., a proxy for a large image that only loads the actual image data on first access. Protection proxy — controls access based on permissions, e.g., checking authorization before delegating a call to the real object. Remote proxy — represents an object that lives in a different address space/machine, hiding the network call behind a local-looking interface (this is essentially what RPC/gRPC client stubs are). All variants share the same shape: the proxy implements the same interface as the real object, so client code interacts with it identically, unaware whether it's talking to the real object or a proxy controlling access to it.

```
interface Image {
    void display();
}

class RealImage implements Image { // the expensive real object
    private final String filename;
    RealImage(String filename) {
        this.filename = filename;
        loadFromDisk(); // expensive - happens at construction
    }
    private void loadFromDisk() { /* slow I/O */ }
    public void display() { /* render pixels */ }
}

class VirtualProxyImage implements Image { // same interface - client can't tell the
    difference
    private final String filename;
    private RealImage realImage; // null until actually needed

    VirtualProxyImage(String filename) { this.filename = filename; } // cheap - no loading
    yet

    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename); // expensive load deferred to first real
            use
        }
        realImage.display();
    }
}
```

```

    }
    realImage.display();
}
}

Image image = new VirtualProxyImage("large_photo.jpg"); // instant – no disk I/O yet
// ... later, only if actually needed ...
image.display(); // NOW the expensive load happens

```

Walking through the code. `RealImage`’s constructor does the expensive work (`loadFromDisk()`) immediately, so `new RealImage(...)` is costly the moment it’s called. `VirtualProxyImage` implements the exact same `Image` interface but its constructor does nothing expensive — it just stores the filename. The `realImage` field starts `null`, and `display()` checks for that: only the *first* call to `display()` triggers `new RealImage(filename)` (and therefore the expensive disk load); every subsequent call reuses the already-loaded `realImage`. Client code holding an `Image` reference calls `.display()` identically whether it’s holding a `RealImage` or a `VirtualProxyImage` — the laziness is entirely invisible from the calling side, which is the “surrogate that controls access” idea: the proxy decides *when* (or whether) the real, expensive object actually gets created.

Q177. What’s the difference between Decorator and Proxy, given both wrap another object behind the same interface? The intent differs: Decorator’s purpose is to add new behavior/responsibilities to an object (and decorators are typically meant to be freely stacked/composed), while Proxy’s purpose is to control access to an object (lazy instantiation, permission checks, remote delegation) without changing what the object fundamentally does — a proxy for an image doesn’t add new image-editing capability, it just controls when/how the real image object is created or accessed. In practice the structural code often looks nearly identical (both wrap and implement the same interface), so the distinguishing factor in an interview answer is the intent behind the wrapping, not the mechanics.

```

// Same structural shape as both MilkDecorator (Q174) and VirtualProxyImage (Q176) – compare
// the INTENT:

class MilkDecorator extends CoffeeDecorator { // DECORATOR: adds a new capability
    (extra cost/description)
    public double cost() { return wrapped.cost() + 0.50; } // genuinely changes what the
    object produces
}

class VirtualProxyImage implements Image { // PROXY: controls access, doesn't
    change what display() means
    public void display() {
        if (realImage == null) realImage = new RealImage(filename); // gatekeeping, not new
        behavior
        realImage.display(); // delegates unchanged
    }
}

```

Walking through the code. Both classes wrap another object behind the same interface their wrapped object implements — structurally, `MilkDecorator` and

`VirtualProxyImage` are shaped almost identically (a field holding the wrapped object, a method that calls into it). The difference is what each one is *for*: `MilkDecorator.cost()` genuinely changes the answer — a coffee with milk costs more than a plain coffee, and that’s new information the decorator adds. `VirtualProxyImage.display()`, once `realImage` is loaded, produces exactly the same visual result `RealImage.display()` would have produced on its own — the proxy hasn’t added any new capability to image display, it has only controlled *when* the real object gets constructed. If an interviewer asks you to tell the two apart in a design you’re presenting, point at what the wrapper method actually returns or does differently: a changed result (new cost, new description, new formatted output) signals Decorator; an unchanged result reached via gatekeeping (lazy creation, a permission check, a network hop) signals Proxy.

Q178. Explain the Composite pattern, and what problem it solves for tree-like structures. Composite lets you treat individual objects (leaves) and compositions of objects (composites/containers) uniformly through a common interface, so client code can operate on a whole tree structure without needing to distinguish between “is this a single item or a group of items” at every point. Classic example: a filesystem where a `File` and a `Folder` (which contains `File`s and other `Folder`s) both implement a common `FileSystemItem` interface with a `getSize()` method — calling `getSize()` on a folder recursively sums its children’s sizes, and client code calling `getSize()` on the root doesn’t need to know or care whether it’s dealing with a single file or a deeply nested folder tree.

```
interface FileSystemItem {
    long getSize();
}

class File implements FileSystemItem {           // LEAF – no children
    private final long sizeBytes;
    File(long sizeBytes) { this.sizeBytes = sizeBytes; }
    public long getSize() { return sizeBytes; }
}

class Folder implements FileSystemItem {          // COMPOSITE – contains other
    FileSystemItems (leaves or folders)
    private final List<FileSystemItem> children = new ArrayList<>();

    void add(FileSystemItem item) { children.add(item); }

    public long getSize() {
        long total = 0;
        for (FileSystemItem child : children) {
            total +=
child.getSize();           // recurses – works whether child is a File or another Folder
        }
        return total;
    }
}

Folder root = new Folder();
root.add(new File(100));
Folder subfolder = new Folder();
subfolder.add(new File(200));
subfolder.add(new File(50));
```

```
root.add(subfolder);

root.getSize();    // 350 – client called ONE method, didn't need to know about the nested
structure
```

Walking through the code. `File` and `Folder` both implement `FileSystemItem`, so both respond to `getSize()` — but the implementations differ in a way the caller never has to think about. `File.getSize()` is the base case: it just returns its own stored size. `Folder.getSize()` is the recursive case: it loops over its `children` (which are typed as `FileSystemItem`, not specifically `File`) and calls `getSize()` on each one — and because `subfolder` is itself a `Folder` added as a child of `root`, calling `root.getSize()` recurses into `subfolder.getSize()`, which recurses into its own children's `getSize()` calls, before the sums bubble back up. The calling code at the bottom calls `root.getSize()` exactly once and gets the correct total (350) without ever writing an `if (item instanceof File) ... else if (item instanceof Folder) ...` check — that uniform treatment of leaves and composites through the identical interface is the entire value Composite provides for tree-shaped data.

A3. LLD Fundamentals (continued)

Design Patterns — Behavioral

Q179. Explain the Strategy pattern, and how it directly implements the Open/Closed Principle. Strategy defines a family of interchangeable algorithms behind a common interface, letting the algorithm used vary independently of the client that uses it. Example: a `SortStrategy` interface implemented by `QuickSort`, `MergeSort`, etc., with a `Sorter` class holding a reference to a `SortStrategy` and delegating to it. This is essentially the direct pattern-level implementation of OCP (Q162) — adding a new sorting algorithm means adding a new class implementing the interface, never modifying the `Sorter` class or any existing strategy. It's also the standard replacement for large if/else or switch blocks that pick behavior based on a type flag.

```
interface SortStrategy {
    void sort(int[] data);
}

class QuickSort implements SortStrategy {
    public void sort(int[] data) { /* quicksort logic */ }
}

class MergeSort implements SortStrategy {
    public void sort(int[] data) { /* mergesort logic */ }
}

class Sorter {
    private SortStrategy strategy;                // holds a reference to the interface,
    not a concrete class

    Sorter(SortStrategy strategy) { this.strategy = strategy; }
```

```

    void setStrategy(SortStrategy strategy) { this.strategy = strategy; } // swappable at
    RUNTIME too

    void sort(int[] data) { strategy.sort(data); } // never branches on which algorithm –
    just delegates
}

Sorter sorter = new Sorter(new QuickSort());
sorter.sort(myArray); // uses QuickSort
sorter.setStrategy(new MergeSort());
sorter.sort(myArray); // now uses MergeSort – same Sorter object, no new class
needed

```

Walking through the code. `Sorter` never contains an `if (algorithm == "quicksort")` branch anywhere — its `sort()` method is a single line that delegates to whatever `SortStrategy` it currently holds. The constructor and `setStrategy()` are the only two places a concrete algorithm is ever mentioned, and both take the abstract `SortStrategy` type as their parameter, not a specific class. This is what lets the same `sorter` object switch from `QuickSort` to `MergeSort` between two calls without being reconstructed — and it's why adding a hypothetical `HeapSort` implements `SortStrategy` later requires writing exactly one new class and touching nothing inside `Sorter`, which is the Open/Closed Principle (Q162) made concrete at the pattern level.

Q180. Explain the Observer pattern, and where you've likely already used it without naming it. Observer defines a one-to-many dependency where a subject maintains a list of observers and notifies them automatically of any state changes, without needing to know concrete details about who its observers are (just that they implement a common `Observer` interface with an `update()` method). Most developers use this constantly without naming it: DOM event listeners (`addEventListener`), pub/sub systems, and reactive state management libraries (RxJS Observables, even React's re-render-on-state-change model) are all Observer pattern applications — a subject (an event target, a state store) notifying interested subscribers when something changes.

```

interface Observer {
    void update(double newPrice);
}

class StockTicker { // the SUBJECT
    private final List<Observer> observers = new ArrayList<>();
    private double price;

    void subscribe(Observer o) { observers.add(o); }
    void unsubscribe(Observer o) { observers.remove(o); }

    void setPrice(double newPrice) {
        this.price = newPrice;
        notifyAll(newPrice); // subject doesn't know WHO its observers are, only
        that they implement Observer
    }

    private void notifyAll(double newPrice) {
        for (Observer o : observers) o.update(newPrice);
    }
}

```

```

class PriceDisplay implements Observer {
    public void update(double newPrice) { /* refresh UI with newPrice */ }
}

class PriceAlertService implements Observer {
    public void update(double newPrice) { /* check thresholds, maybe send an alert */ }
}

StockTicker ticker = new StockTicker();
ticker.subscribe(new PriceDisplay());
ticker.subscribe(new PriceAlertService());
ticker.setPrice(150.25); // BOTH observers get notified – StockTicker never mentions either
                           class by name

```

Walking through the code. `StockTicker` holds a `List<Observer>`, never a `List<PriceDisplay>` or anything concrete — `subscribe()` accepts anything implementing `Observer`. When `setPrice()` is called, `notifyAll()` loops over every subscribed observer and calls `update()` on each, without any conditional logic distinguishing a `PriceDisplay` from a `PriceAlertService`; it treats them uniformly through the interface. Both concrete observer classes were added to the list independently and can be added or removed at runtime (`subscribe` / `unsubscribe`) without `StockTicker` itself ever being modified — which is exactly the shape of `element.addEventListener('click', handler)` in the browser: the DOM element (subject) doesn't know or care what your handler function does, it just calls it when the click event (the state change) occurs.

Q181. Explain the Command pattern, and give a real use case beyond “undo/redo.” Command encapsulates a request (an action plus its parameters) as an object, decoupling the object that invokes an operation from the object that knows how to perform it. Beyond undo/redo (where each command object stores enough info to reverse itself), real use cases: a job queue system where each queued job is a `Command` object (encapsulating exactly what needs to happen, serializable, executable later by any worker), or a UI toolkit where button click handlers are `Command` objects that can be configured/reassigned without the button needing to know what specific action it triggers.

```

interface Command {
    void execute();
    void undo();
}

class Document {
    StringBuilder content = new StringBuilder();
}

class InsertTextCommand implements Command { // the request AND its parameters,
bundled as an object
    private final Document doc;
    private final String text;
    private final int position;

    InsertTextCommand(Document doc, String text, int position) {
        this.doc = doc; this.text = text; this.position = position;
    }
}

```

```

    }

    public void execute() { doc.content.insert(position, text); }
    public void undo() { doc.content.delete(position, position + text.length()); }
}

class CommandHistory {                                // the INVOKER — doesn't know what
each command does
    private final Deque<Command> history = new ArrayDeque<>();

    void executeAndTrack(Command cmd) {
        cmd.execute();
        history.push(cmd);
    }

    void undoLast() {
        if (!history.isEmpty()) history.pop().undo(); // calls undo() without knowing the
concrete command type
    }
}

CommandHistory history = new CommandHistory();
history.executeAndTrack(new InsertTextCommand(doc, "Hello", 0));
history.undoLast(); // reverses it, via the same generic Command interface

```

Walking through the code. `InsertTextCommand` bundles everything needed to perform (and later reverse) one specific action — the target document, the text to insert, and where — into a single object, rather than the caller passing three separate arguments to some `insertText(doc, text, position)` method directly.

`CommandHistory.executeAndTrack()` calls `cmd.execute()` on whatever `Command` it's handed without knowing it's specifically an `InsertTextCommand` — it could just as easily be a `DeleteTextCommand` or `FormatTextCommand` implementing the same interface. That's the decoupling: `CommandHistory` is the invoker (it knows *when* to run commands and how to track them for undo) while each concrete `Command` implementation is the only thing that knows *how* to actually perform (and reverse) its specific action. Pushing executed commands onto a stack (`history.push(cmd)`) and popping-and-undoing (`history.pop().undo()`) is what turns this into working undo/redo — but the same shape (bundle a request as an object, hand it to a generic invoker) is exactly what a job queue does with serialized job objects, with “execute later, possibly on a different worker” replacing “execute now, possibly undo later.”

Q182. Explain the State pattern, and how it's a cleaner alternative to a large state-machine switch statement. State allows an object to alter its behavior when its internal state changes, appearing as if the object changed its class — implemented by encapsulating state-specific behavior into separate state classes, with the context object delegating behavior to its current state object and the state objects themselves handling transitions to the next state. This is cleaner than a giant `switch(state) { case A: ... case B: ... }` scattered across every method of a class, because adding a new state means adding a new class (again, satisfying OCP) rather than touching a large, growing switch statement in multiple places throughout the codebase, and each state's logic is cleanly isolated rather than interleaved with every other state's logic in the same method bodies.

```

interface OrderState {
    void next(OrderContext order);    // each state decides what "next" means, and what
    String name();                    state follows
}

class PlacedState implements OrderState {
    public void next(OrderContext order) { order.setState(new ShippedState()); }
    public String name() { return "PLACED"; }
}

class ShippedState implements OrderState {
    public void next(OrderContext order) { order.setState(new DeliveredState()); }
    public String name() { return "SHIPPED"; }
}

class DeliveredState implements OrderState {
    public void next(OrderContext order) {
        throw new IllegalStateException("Order already delivered – no further transitions");
    }
    public String name() { return "DELIVERED"; }
}

class OrderContext {                // the CONTEXT – delegates to whatever state it
    currently holds
    private OrderState state = new PlacedState();

    void setState(OrderState state) { this.state = state; }
    void advance() {
        state.next(this);            // context doesn't know the transition rules – the state does
        String currentStatus() { return state.name(); }
    }

    OrderContext order = new OrderContext();
    order.currentStatus(); // "PLACED"
    order.advance();
    order.currentStatus(); // "SHIPPED"
    order.advance();
    order.currentStatus(); // "DELIVERED"
    order.advance();       // throws – DeliveredState refuses to go further
}

```

Walking through the code. `OrderContext` doesn't contain a `switch (status) { case PLACED: ...; case SHIPPED: ... }` anywhere — its `advance()` method just calls `state.next(this)`, delegating the actual decision of “what state comes after this one” to whichever `OrderState` object it currently holds. Each state class encodes its own transition: `PlacedState.next()` moves to a new `ShippedState`, `ShippedState.next()` moves to `DeliveredState`, and `DeliveredState.next()` refuses (throwing, since there's nowhere further to go). Adding a new state — say, a `CancelledState` reachable from `PlacedState` — means writing one new class and changing `PlacedState`'s `next()` logic in exactly one place, rather than finding and updating every method in `OrderContext` that happened to have a case for the old set of statuses. Each state's behavior is fully contained in its own small class instead of interleaved with every other state's logic inside shared `OrderContext` methods.

Q183. Explain the Chain of Responsibility pattern, and a real-world system that uses it. Chain of Responsibility passes a request along a chain of potential handlers,

where each handler decides either to process the request or pass it to the next handler in the chain, without the sender needing to know which handler will ultimately process it (or how many handlers exist). Real-world example: HTTP middleware in Express.js or similar frameworks — each middleware function can handle (or partially handle and pass along) a request, and the request moves down the chain until something sends a response, with the caller (whoever sent the HTTP request) unaware of and unconcerned with how many middleware functions exist in between.

```
abstract class Handler {
    protected Handler next; // reference to the next link in the chain

    Handler setNext(Handler next) { this.next = next; return this; }

    abstract void handle(Request request);

    protected void passToNext(Request request) {
        if (next != null) next.handle(request); // hand off if this handler doesn't fully
        handle it
    }
}

class AuthHandler extends Handler {
    void handle(Request request) {
        if (!request.isAuthenticated()) { request.reject("401"); return; } // stops the
        chain here
        passToNext(request); // otherwise
        continue
    }
}

class RateLimitHandler extends Handler {
    void handle(Request request) {
        if (request.exceedsRateLimit()) { request.reject("429"); return; }
        passToNext(request);
    }
}

class BusinessLogicHandler extends Handler {
    void handle(Request request) { request.respond("200 OK"); } // end of the chain –
    actually does the work
}

Handler chain = new AuthHandler();
chain.setNext(new RateLimitHandler()).setNext(new BusinessLogicHandler());
chain.handle(incomingRequest); // caller doesn't know or care how many handlers exist in
between
```

Walking through the code. Each `Handler` subclass makes an independent decision: `AuthHandler.handle()` either rejects the request outright (`request.reject("401")`), then returns — stopping the chain right there) or calls `passToNext(request)` to hand off to whatever comes after it. `RateLimitHandler` does the same for its own concern. Neither handler knows what handler comes after it by name — `passToNext()` just calls `next.handle(request)` on whatever was wired in via `setNext()`. The chain is assembled once, outside any individual handler (`chain.setNext(new RateLimitHandler()).setNext(...)`), and the calling code at the bottom only ever talks to

the first handler in the chain — it doesn't know or need to know that there are two more handlers after `AuthHandler`, exactly as the answer describes for HTTP middleware: the code that issued the original request has no idea how many `app.use(...)` middleware functions sit between it and the final route handler.

Q184. Explain the Template Method pattern, and how it differs from Strategy.

Template Method defines the skeleton of an algorithm in a base class method, with certain steps deferred to subclasses via abstract/overridable methods — the overall algorithm structure/sequence is fixed, but specific steps are customizable. The key difference from Strategy: Template Method uses inheritance (subclasses override specific steps of a fixed algorithm shape defined in the parent), while Strategy uses composition (the whole algorithm is swapped out as an interchangeable object, with no fixed shared skeleton). Example: an abstract `DataProcessor` class with a final `process()` method calling `readData()` → `transformData()` → `saveData()` in that fixed order, where subclasses like `CsvDataProcessor` and `JsonDataProcessor` only override the individual step methods, not the overall sequence.

```
abstract class DataProcessor {
    final void process() {                // final – the SEQUENCE itself can never be
    overridden
        List<String> raw = readData();
        List<String> transformed = transformData(raw);
        saveData(transformed);
    }

    abstract List<String> readData();      // steps subclasses MUST customize
    abstract List<String> transformData(List<String> raw);
    void saveData(List<String> data) {     // a step with a sensible DEFAULT –
    override only if needed
        System.out.println("Saving " + data.size() + " records to default store");
    }
}

class CsvDataProcessor extends DataProcessor {
    List<String> readData() { /* read from a .csv file */ return List.of("a,1", "b,2"); }
    List<String> transformData(List<String> raw) { /* CSV-specific parsing */ return raw; }
}

class JsonDataProcessor extends DataProcessor {
    List<String> readData() { /* read from a .json file */ return List.of("{}"); }
    List<String> transformData(List<String> raw) { /* JSON-specific parsing */ return raw; }
    @Override void saveData(List<String> data) { /* override the default – save to a
    different store */ }
}

new CsvDataProcessor().process(); // readData → transformData → saveData, in that exact
fixed order
```

Walking through the code. `process()` is declared `final` — no subclass can override the *order* in which `readData()`, `transformData()`, and `saveData()` run, only what each individual step does. `CsvDataProcessor` and `JsonDataProcessor` both override `readData()` and `transformData()` with format-specific logic, but neither touches the sequencing in `process()` itself; `JsonDataProcessor` additionally overrides the optional `saveData()` step

(which has a default implementation), while `CsvDataProcessor` is happy with that default. Compare this to Strategy (Q179): `Sorter` there held a `SortStrategy` object and could swap the *entire* algorithm at runtime by assigning a different strategy instance — there was no fixed multi-step skeleton shared across strategies, each `SortStrategy` implementation was free to structure its internals however it liked. Here, by contrast, every `DataProcessor` subclass is locked into the same three-step shape defined once in the parent class; only the content of each step varies, via inheritance and method overriding rather than object composition.

Q185. Explain the Iterator pattern, and why it's fundamental enough that most languages build it into the language itself. Iterator provides a way to sequentially access elements of a collection without exposing the collection's underlying representation (array, linked list, tree) — the client just calls `hasNext()` / `next()` (or uses a `for...of` loop, Python's iterator protocol, etc.) without needing to know or care how the collection stores its elements internally. It's built into most modern languages (Java's `Iterable`, JavaScript's iterator protocol/ `Symbol.iterator`, Python's `__iter__`) because traversal is such a universally needed operation across every collection type that baking a standard protocol into the language lets any collection interoperate with any language construct (for-loops, spread syntax, destructuring) that expects iterability.

```
class LinkedListNode<T> { T value; LinkedListNode<T> next; }

class SimpleLinkedList<T> implements Iterable<T> { // exposing a custom collection through
the standard protocol
    private LinkedListNode<T> head;

    void add(T value) {
        LinkedListNode<T> node = new LinkedListNode<>();
        node.value = value;
        node.next = head;
        head = node;
    }

    public Iterator<T> iterator() { // returns something that knows how to
walk THIS structure
        return new Iterator<T>() {
            private LinkedListNode<T> current = head;

            public boolean hasNext() { return current != null; }
            public T next() {
                T value = current.value;
                current = current.next; // internal representation only known
HERE
                return value;
            }
        };
    }
}

SimpleLinkedList<String> list = new SimpleLinkedList<>();
list.add("c"); list.add("b"); list.add("a");

for (String item : list) { // works because SimpleLinkedList implements Iterable —
no linked-list knowledge needed
```

```

        System.out.println(item);
    }

```

Walking through the code. `SimpleLinkedList` is backed by hand-linked `LinkedListNode` objects internally — a representation completely different from, say, an array-backed list — but `implements Iterable<T>` means it only needs to supply one method, `iterator()`, returning an object with `hasNext()` / `next()`. That inner `Iterator` is the only piece of code that ever touches `current.next` directly; the `for (String item : list)` loop at the bottom calls `hasNext()` / `next()` through the standard interface without knowing or caring that nodes and `next` pointers exist underneath. This is exactly why the pattern is baked into the language: any class that `implements Iterable` — whether it's backed by an array, a linked list, a tree walked in some traversal order, or even a lazily-computed sequence — automatically works with `for-each` loops, `stream()`, and every other language construct that expects an `Iterable`, without those constructs needing separate code paths per underlying data structure.

Q186. Explain the Mediator pattern, and how it reduces coupling in a system with many interacting objects. Mediator defines an object that encapsulates how a set of other objects interact, so those objects don't reference each other directly (avoiding a tangled many-to-many web of direct dependencies) — instead, each object only knows about the mediator, and communicates through it. Example: an air traffic control tower as the mediator — individual planes don't communicate directly with every other plane to coordinate landing order (which would be an unmanageable N^2 web of dependencies as planes are added/removed); they each only talk to the tower, which coordinates all of them. In software, a chat room implementation is a classic example: users don't hold direct references to every other user, they send messages through a central `ChatRoom` mediator that routes them.

```

interface ChatMediator {
    void sendMessage(String message, User sender);
    void addUser(User user);
}

class ChatRoom implements ChatMediator { // the MEDIATOR
    private final List<User> users = new ArrayList<>();

    public void addUser(User user) { users.add(user); }

    public void sendMessage(String message, User sender) {
        for (User u : users) {
            if (u != sender) u.receive(message, sender); // routes on everyone's behalf –
no direct user-to-user links
        }
    }
}

class User {
    private final String name;
    private final ChatMediator mediator; // only knows about the mediator,
not other Users
}

```

```

    User(String name, ChatMediator mediator) {
        this.name = name;
        this.mediator = mediator;
        mediator.addUser(this);
    }

    void send(String message) { mediator.sendMessage(message, this); }
    void receive(String message, User from) { System.out.println(name + " got: " + message +
" from " + from.name); }
}

ChatMediator room = new ChatRoom();
User alice = new User("Alice", room);
User bob = new User("Bob", room);
alice.send("Hey Bob!");    // Alice never holds a reference to Bob directly

```

Walking through the code. `User` never stores a list of other `User` objects — its only stored reference is `mediator`, the shared `ChatMediator`. When `alice.send(...)` is called, it calls `mediator.sendMessage(message, this)`, handing the message and itself to the room; `alice` never looks up `bob` or calls anything on him directly.

`ChatRoom.sendMessage()` is the one place that knows about every user in the room (users) and does the actual fan-out, looping through and calling `receive()` on each user except the sender. Adding a third user (Charlie) requires no changes to `User`, `alice`, or `bob` at all — Charlie just registers with the same `ChatRoom`, and `sendMessage()`'s existing loop automatically includes him. Without the mediator, every `User` would need a direct reference to every other `User` to send messages point-to-point, and that reference list would need updating on every user join/leave — exactly the unmanageable N^2 web the air traffic control tower analogy is warning about.

Q187. Explain the Visitor pattern, and why it's considered one of the harder patterns to justify using. Visitor lets you define a new operation on a set of classes without modifying those classes, by having each class accept a "visitor" object and call back a type-specific method on it (`accept(Visitor v) { v.visit(this); }`), with the actual operation logic living in the visitor implementation instead of the visited classes. It's considered hard to justify because it fundamentally inverts the Open/Closed trade-off compared to most patterns: adding a new operation is easy (just write a new visitor), but adding a new type to visit requires modifying every existing visitor implementation to handle it — the opposite trade-off from something like Strategy. This makes it a good fit only when the set of types is stable/rarely changes but new operations over those types are added frequently (e.g., compiler AST node types, where new "operations" like type-checking, optimization passes, and code generation are added far more often than genuinely new syntax node types).

```

interface ExprVisitor<R> {                                // one method PER type in the (stable)
    hierarchy
    R visitNumber(NumberExpr n);
    R visitAdd(AddExpr a);
}

interface Expr {
    <R> R accept(ExprVisitor<R> visitor);                // each Expr type knows how to call back into
    the visitor
}

```

```

}

class NumberExpr implements Expr {
    double value;
    NumberExpr(double value) { this.value = value; }
    public <R> R accept(ExprVisitor<R> visitor) { return visitor.visitNumber(this); }
}

class AddExpr implements Expr {
    Expr left, right;
    AddExpr(Expr left, Expr right) { this.left = left; this.right = right; }
    public <R> R accept(ExprVisitor<R> visitor) { return visitor.visitAdd(this); }
}

class EvaluateVisitor implements ExprVisitor<Double> {           // operation #1 – evaluate the
expression
    public Double visitNumber(NumberExpr n) { return n.value; }
    public Double visitAdd(AddExpr a) { return a.left.accept(this) + a.right.accept(this); }
}

class PrintVisitor implements ExprVisitor<String> {              // operation #2 – added WITHOUT
touching NumberExpr/AddExpr
    public String visitNumber(NumberExpr n) { return String.valueOf(n.value); }
    public String visitAdd(AddExpr a) { return "(" + a.left.accept(this) + " + " + a.right.ac
cept(this) + ")"; }
}

Expr expr = new AddExpr(new NumberExpr(2), new NumberExpr(3));
expr.accept(new EvaluateVisitor());    // 5.0
expr.accept(new PrintVisitor());       // "(2.0 + 3.0)"

```

Walking through the code. `NumberExpr.accept()` and `AddExpr.accept()` both just call back into whichever visitor they’re handed (`visitor.visitNumber(this)` / `visitor.visitAdd(this)`) — neither `Expr` subtype contains any evaluation or printing logic itself. `EvaluateVisitor` and `PrintVisitor` each implement `ExprVisitor<R>` completely independently, and adding `PrintVisitor` as a second operation required writing one new class with zero changes to `NumberExpr`, `AddExpr`, or `EvaluateVisitor` — that’s the “adding a new operation is easy” side of the trade-off. The cost shows up if a third expression type is added later — say, `MultiplyExpr` — because `ExprVisitor<R>` would need a new `visitMultiply()` method, and *every existing visitor implementation* (`EvaluateVisitor`, `PrintVisitor`, and any others) would need to be updated to implement that new method or the code won’t compile. That’s the inverted trade-off the answer describes: stable type hierarchy, frequently added operations, is where this earns its complexity; a type hierarchy that changes often makes Visitor actively painful.

UML & Modeling Basics

Q188. What is a UML class diagram used for in an LLD interview, and what do the different arrow types mean? A class diagram visually represents classes, their attributes/methods, and the relationships between them — used in interviews to quickly communicate your object model without writing full code. Key arrow types: a solid line with a hollow triangle arrowhead = inheritance/generalization (points to the parent/interface); a dashed line with a hollow triangle = interface implementation; a

solid line with a diamond = aggregation (hollow diamond) or composition (filled diamond) at the “whole” end; a plain solid line = simple association; a dashed line with an open arrowhead = dependency (one class uses another, typically as a method parameter or local variable, without holding a persistent reference).

Q189. What is the difference between a class diagram and a sequence diagram, and when would you use each in an interview? A class diagram shows the static structure — what classes exist, their attributes/methods, and the relationships between them (good for communicating your overall object model). A sequence diagram shows dynamic behavior — the sequence of messages/method calls exchanged between objects over time for a specific scenario/use case (good for walking through a specific flow, like “what happens step by step when a user places an order,” showing exactly which objects call which methods on which other objects in what order). In an LLD interview, you typically present the class diagram (or just describe the classes) to establish the structure, then may walk through a sequence diagram (or narrate the equivalent verbally) for a key use case to demonstrate the design actually works end-to-end for a real scenario, not just looks reasonable in the abstract.

Q190. What does it mean to “design to an interface, not an implementation,” and how does this show up concretely in an LLD interview answer? It means client code and collaborating classes should depend on abstract interfaces/contracts rather than concrete classes, so implementations can be swapped, mocked (for testing), or extended without touching the dependent code — this is the practical, everyday application of DIP (Q165) and is arguably the single most consistently rewarded habit across LLD interviews. Concretely, this shows up as: a

`NotificationService` constructor taking a `MessageSender` interface parameter rather than internally `new`-ing a concrete `EmailSender`; a `PaymentProcessor` accepting a `PaymentStrategy` interface rather than hardcoding logic per payment type; a repository class exposing an interface (`UserRepository`) separate from its concrete implementation (`PostgresUserRepository`) so the concrete storage technology could be swapped later without touching business logic that depends only on the interface.

A4. LLD Case Studies — Machine Coding Walkthroughs

These are the “design and code the classes for X” problems asked in machine-coding/LLD rounds. Each answer covers the core entities, key relationships, and the design decisions an interviewer will probe — with representative code, and a line-by-line walkthrough of that code, where it clarifies the design.

Case Study 1: Design a Parking Lot System

Q191. What are the core entities and their relationships? `ParkingLot` (contains multiple `Level` s) → `Level` (contains multiple `ParkingSpot` s) → `ParkingSpot` (has a type:

MOTORCYCLE / COMPACT / LARGE , and a status: free/occupied) → `Vehicle` (abstract base, with `Motorcycle` / `Car` / `Truck` subtypes, each knowing which spot size(s) it can fit in) → `Ticket` (created on entry, links a `Vehicle` to an assigned `ParkingSpot` and an entry timestamp) → `Payment` / `Pricing` strategy (computed on exit based on duration and spot type).

Q192. Why should spot assignment logic live in its own class rather than inside `ParkingLot` or `Vehicle` directly? This is a direct SRP (Q161) application: `ParkingLot` should manage the structure of levels/spots, `Vehicle` should represent vehicle properties, and neither should also contain the algorithm for finding a suitable free spot (which is a distinct, independently-changeable concern — you might want to switch from “nearest available spot” to “spread vehicles across levels for balance” without touching the `ParkingLot` or `Vehicle` classes at all). A separate `SpotAssignmentStrategy` interface (Strategy pattern, Q179) with implementations like `NearestSpotStrategy` keeps this cleanly swappable.

```
enum SpotSize { MOTORCYCLE, COMPACT, LARGE }

interface ParkingSpot {
    boolean isFree();
    SpotSize getSize();
    void assignVehicle(Vehicle v);
    void removeVehicle();
}

class SimpleParkingSpot implements ParkingSpot {
    private final SpotSize size;
    private Vehicle occupant;

    SimpleParkingSpot(SpotSize size) { this.size = size; }

    public boolean isFree() { return occupant == null; }
    public SpotSize getSize() { return size; }
    public void assignVehicle(Vehicle v) { this.occupant = v; }
    public void removeVehicle() { this.occupant = null; }
}

abstract class Vehicle {
    protected String licensePlate;
    abstract SpotSize[] compatibleSpotSizes(); // e.g. Motorcycle fits any size
}

class Motorcycle extends Vehicle {
    SpotSize[] compatibleSpotSizes() {
        return new SpotSize[]{SpotSize.MOTORCYCLE, SpotSize.COMPACT, SpotSize.LARGE};
    }
}

class Car extends Vehicle {
    SpotSize[] compatibleSpotSizes() {
        return new SpotSize[]{SpotSize.COMPACT, SpotSize.LARGE};
    }
}

class Truck extends Vehicle {
    SpotSize[] compatibleSpotSizes() {
        return new SpotSize[]{SpotSize.LARGE};
    }
}
```

```

    }
}

class Level {
    int floor;
    List<ParkingSpot> spots;
}

interface SpotAssignmentStrategy {
    Optional<ParkingSpot> findSpot(Vehicle vehicle, List<Level> levels);
}

class NearestSpotStrategy implements SpotAssignmentStrategy { // simplest strategy: first
    free compatible spot
    public Optional<ParkingSpot> findSpot(Vehicle vehicle, List<Level> levels) {
        Set<SpotSize> compatible = new HashSet<>(Arrays.asList(vehicle.compatibleSpotSizes()))
    };
    for (Level level : levels) {
        for (ParkingSpot spot : level.spots) {
            if (spot.isFree() && compatible.contains(spot.getSize())) {
                return Optional.of(spot);
            }
        }
    }
    return Optional.empty();
}
}

```

Walking through the code. `ParkingSpot` is an interface, not a concrete class — `SimpleParkingSpot` is one implementation, holding its `size` (fixed at construction) and its current `occupant` (null means free). `Vehicle.compatibleSpotSizes()` is `abstract`, so every concrete vehicle type is forced to declare which spot sizes it can legally use: a `Truck` can only use `LARGE`, while a `Motorcycle` can squeeze into any size — this list is what `NearestSpotStrategy` checks against. `NearestSpotStrategy.findSpot()` builds a `Set<SpotSize>` from the vehicle's compatible sizes (a set, so the `contains()` check is $O(1)$ rather than scanning an array per spot), then walks levels top to bottom and spots within each level in order, returning the first spot that's both `isFree()` and in the compatible set. Notice `findSpot()` takes the `Vehicle` and `List<Level>` as parameters rather than being a method on `ParkingLot` or `Vehicle` itself — that's the SRP point the answer makes: swapping in a `BalancedSpotStrategy` later (spreading vehicles across levels instead of always filling the nearest one first) means writing a new class implementing this same interface, with zero changes to `Level`, `ParkingSpot`, or `Vehicle`.

Q193. How would you design the pricing/billing so that different pricing models (hourly, flat-rate, per-vehicle-type) can be supported without modifying the core ticketing flow? Define a `PricingStrategy` interface with a `calculateFee(Ticket ticket)` method, injected into the class that finalizes a `Ticket` on exit (Strategy pattern again). `HourlyPricingStrategy` computes based on duration; `VehicleTypePricingStrategy` might apply a different hourly rate depending on `Vehicle` subtype. The exit/checkout flow calls `pricingStrategy.calculateFee(ticket)` without knowing or caring which concrete strategy is configured — satisfying OCP, since a new pricing model is a new class, not a modification to the checkout logic.

```

class Ticket {
    Vehicle vehicle;
    ParkingSpot spot;
    long entryTimestampMillis;
    Long exitTimestampMillis; // null until the vehicle actually exits
}

interface PricingStrategy {
    double calculateFee(Ticket ticket);
}

class HourlyPricingStrategy implements PricingStrategy {
    private static final double RATE_PER_HOUR = 2.0;

    public double calculateFee(Ticket ticket) {
        long durationMillis = ticket.exitTimestampMillis - ticket.entryTimestampMillis;
        double hours = Math.ceil(durationMillis / (1000.0 * 60 * 60)); // round UP –
partial hour still billed
        return hours * RATE_PER_HOUR;
    }
}

class VehicleTypePricingStrategy implements PricingStrategy {
    public double calculateFee(Ticket ticket) {
        long durationMillis = ticket.exitTimestampMillis - ticket.entryTimestampMillis;
        double hours = Math.ceil(durationMillis / (1000.0 * 60 * 60));
        double rate = (ticket.vehicle instanceof Truck) ? 4.0 : 2.0; // trucks cost more
per hour
        return hours * rate;
    }
}

class CheckoutService {
    private final PricingStrategy pricingStrategy; // injected – CheckoutService
never hardcodes a rate

    CheckoutService(PricingStrategy pricingStrategy) { this.pricingStrategy =
pricingStrategy; }

    double checkout(Ticket ticket) {
        ticket.exitTimestampMillis = System.currentTimeMillis();
        double fee = pricingStrategy.calculateFee(ticket); // delegates – doesn't know
which strategy is plugged in
        ticket.spot.removeVehicle();
        return fee;
    }
}

```

Walking through the code. `Ticket` records the raw data (`entryTimestampMillis`, and `exitTimestampMillis` once set) but contains no pricing logic itself — that separation is what lets multiple `PricingStrategy` implementations operate on the same `Ticket` shape. `HourlyPricingStrategy.calculateFee()` computes elapsed duration and rounds hours *up* with `Math.ceil` (a deliberate business rule: 61 minutes bills as 2 hours, not 1.02), then multiplies by a flat rate. `VehicleTypePricingStrategy` does the same duration math but picks a different `rate` depending on `ticket.vehicle`'s runtime type. `CheckoutService`'s constructor takes a `PricingStrategy` — it's constructed once with whichever strategy the parking lot operator has configured, and `checkout()` calls

`pricingStrategy.calculateFee(ticket)` without an `if/else` on which strategy is active. Swapping from hourly to per-vehicle-type pricing is a one-line change at the point `CheckoutService` is constructed (`new CheckoutService(new VehicleTypePricingStrategy())`), never inside `checkout()` itself.

Q194. How do you handle the concurrency problem of two vehicles

simultaneously being assigned the same spot? Exactly the same atomic-conditional-update pattern from the HLD ticket-booking case study (Q142): spot assignment must be an atomic compare-and-swap (“assign this spot only if it’s still free”), whether implemented as a synchronized method/lock around the assignment operation in a single-process system, or as a DB-level conditional update in a distributed one. This is a good moment in an interview to explicitly connect your LLD answer back to the concurrency-correctness principles from the HLD portion — interviewers notice when a candidate treats the two rounds as genuinely connected rather than entirely separate skill sets.

```
class SafeParkingSpot implements ParkingSpot {
    private final SpotSize size;
    private volatile Vehicle occupant;
    private final Object lock = new Object();    // one lock per spot – not a global lock
    (Q216 explains why)

    SafeParkingSpot(SpotSize size) { this.size = size; }

    public boolean isFree() { return occupant == null; }
    public SpotSize getSize() { return size; }

    // atomic "assign only if still free" – the compare-and-swap the answer describes
    boolean tryAssign(Vehicle v) {
        synchronized (lock) {
            if (occupant != null) return false;    // already taken between the caller's
            check and this call
            occupant = v;
            return true;
        }
    }

    public void assignVehicle(Vehicle v) {
        tryAssign(v);    // interface method delegates to the safe version
    }
    public void removeVehicle() { synchronized (lock) { occupant = null; } }
}
```

Walking through the code. The unsafe version of this class would have callers do `if (spot.isFree()) spot.assignVehicle(vehicle)` as two separate steps — and between the `isFree()` check and the `assignVehicle()` call, another thread (say, a second entry gate processing a different car at the same instant) could have already assigned that exact spot, so both vehicles end up believing they own it. `tryAssign()` collapses the check and the assignment into one `synchronized` block, so no other thread can observe or act on `occupant` while this check-then-set sequence is in progress — it either returns `true` (this call won the race and now owns the spot) or `false` (someone else got there first, entirely inside the same atomic operation). Note the lock is a field *on the spot itself* (`this.lock`), not a single lock shared by the whole `ParkingLot` — locking per-spot

means two different gates assigning two different spots never contend with each other at all, only genuinely concurrent attempts on the *same* spot ever block.

Q195. How would you extend this design to support multiple entry/exit gates with real-time spot availability display? Each `Level` maintains a count of free spots per `SpotSize` (updated atomically on assignment/removal rather than recounted by scanning all spots every time — an important performance detail), and a `DisplayBoard` (Observer pattern, Q180) subscribes to spot-availability-changed events from each level, updating a shared read-optimized view that gate displays query — this way, an expensive full-scan-and-count operation isn't repeated on every single display refresh across every gate.

```
interface AvailabilityObserver {
    void onAvailabilityChanged(int levelNumber, SpotSize size, int newFreeCount);
}

class Level {
    int floor;
    List<ParkingSpot> spots;
    Map<SpotSize, AtomicInteger> freeCountBySize = new EnumMap<>(SpotSize.class); //
    maintained incrementally
    private final List<AvailabilityObserver> observers = new ArrayList<>();

    void subscribe(AvailabilityObserver o) { observers.add(o); }

    void onSpotFreed(SpotSize size) {
        int newCount = freeCountBySize.get(size).incrementAndGet(); // O(1) update, no re-
        scan of all spots
        notifyObservers(size, newCount);
    }

    void onSpotTaken(SpotSize size) {
        int newCount = freeCountBySize.get(size).decrementAndGet();
        notifyObservers(size, newCount);
    }

    private void notifyObservers(SpotSize size, int newCount) {
        for (AvailabilityObserver o : observers) o.onAvailabilityChanged(floor, size,
        newCount);
    }
}

class DisplayBoard implements AvailabilityObserver { // one shared, read-optimized
    view every gate queries
    private final Map<Integer, Map<SpotSize, Integer>> availability = new ConcurrentHashMap<>
    ();

    public void onAvailabilityChanged(int levelNumber, SpotSize size, int newFreeCount) {
        availability.computeIfAbsent(levelNumber, k -> new EnumMap<>(SpotSize.class))
        .put(size, newFreeCount);
    }

    int getFreeCount(int levelNumber, SpotSize size) {
        return availability.getOrDefault(levelNumber, Map.of()).getOrDefault(size, 0);
    }
}
```

Walking through the code. `Level.freeCountBySize` is a running counter, not something recomputed by iterating `spots` on every query — `onSpotFreed()` and `onSpotTaken()` each perform a single `incrementAndGet()` / `decrementAndGet()` (atomic, so concurrent gates updating the same level's counts don't race each other) and immediately push the new count out via `notifyObservers()`. `DisplayBoard` implements `AvailabilityObserver` and subscribes to every `Level`; its `onAvailabilityChanged()` callback just updates one entry in its own in-memory `availability` map — an $O(1)$ write triggered only when something actually changes. Every gate display then calls `displayBoard.getFreeCount(level, size)`, which is a cheap map lookup against `DisplayBoard`'s already-current data, rather than each of potentially dozens of gate screens independently scanning every spot on every level on every refresh — exactly the expensive-full-scan-avoided-via-push-updates trade-off the answer is pointing at, and the same Observer shape as `StockTicker / Observer` in Q180.

Case Study 2: Design an Elevator System

Q196. What are the core entities, and what is the central algorithmic challenge? `Elevator` (has current floor, direction, current state, a queue/set of requested floors), `ElevatorController / ElevatorSystem` (manages multiple elevators, decides which elevator services a given request), `Floor` (has up/down call buttons), `Request` (external floor call with direction, or internal floor selection from within a car). The central challenge is the dispatch algorithm: given N elevators at various floors/directions and a new pickup request, which elevator should service it — this is the part interviewers spend the most time probing.

Q197. Describe a reasonable elevator dispatch algorithm and its scoring logic. A common approach scores each elevator for a given request and picks the lowest-cost one: an elevator already moving toward the requested floor in the same direction as the request scores very cheaply (it can pick up the request “on the way” with minimal detour); an idle elevator scores based on simple distance; an elevator moving away from the request (would need to finish its current trip, reverse, then come back) scores expensively. This is essentially a variant of the classic SCAN/elevator disk-scheduling algorithm from operating systems, applied to physical elevators instead of disk arm movement — a nice detail to mention if you know it, since it shows the pattern-recognition depth interviewers are probing for.

```
enum Direction { UP, DOWN, IDLE }
enum ElevatorState { IDLE, MOVING, DOORS_OPEN, OUT_OF_SERVICE }

class Elevator {
    int id;
    int currentFloor;
    Direction direction = Direction.IDLE;
    ElevatorState state = ElevatorState.IDLE;
}

class ElevatorController {
    List<Elevator> elevators;
```

```

Elevator selectElevator(int requestFloor, Direction requestDir) {
    return elevators.stream()
        .filter(e -> e.state !=
ElevatorState.OUT_OF_SERVICE) // never dispatch a broken elevator
        .min(Comparator.comparingInt(e -> cost(e, requestFloor, requestDir)))
        .orElseThrow();
}

int cost(Elevator e, int floor, Direction dir) {
    if (e.direction == Direction.IDLE) {
        return Math.abs(e.currentFloor - floor); // idle: pure
distance
    }
    boolean movingToward =
        (e.direction == Direction.UP && floor >= e.currentFloor && dir == Direction.UP) |
|
        (e.direction == Direction.DOWN && floor <= e.currentFloor && dir == Direction.DOW
N);
    return movingToward ? Math.abs(e.currentFloor - floor)
        : Integer.MAX_VALUE / 2; // heavy penalty, not infinite -
avoids overflow on addition
    }
}

```

Walking through the code. `selectElevator()` first filters out any `OUT_OF_SERVICE` elevator (never worth scoring, since it can never be dispatched), then uses `Comparator.comparingInt` with `cost()` as the key extractor to find the single elevator with the lowest cost — `min()` over a `Stream` is exactly “score everyone, pick the cheapest.” `cost()` itself implements the three-tier logic the answer describes: an `IDLE` elevator’s cost is just the floor distance (`Math.abs`), since there’s no existing trip to consider. A `MOVING` elevator’s cost depends on `movingToward` — a boolean computed from whether the elevator’s current direction matches the request’s direction *and* the requested floor is actually ahead of the elevator in that direction (an elevator moving `UP` from floor 3 can pick up an `UP` request at floor 7, but not one at floor 1, since it would have already passed it going the wrong way to detour back). If `movingToward` is true, the cost is the same simple distance; if false, the cost is `Integer.MAX_VALUE / 2` — deliberately not `Integer.MAX_VALUE` itself, so that if this value is ever added to another cost elsewhere in a more elaborate scoring formula, it won’t silently overflow into a negative number and corrupt the comparison.

Q198. How does an individual elevator decide the order to service multiple queued floor requests (both external calls it picked up and internal button presses)? Each elevator maintains its requested floors typically as two ordered sets/priority queues — “floors to visit while going up” and “floors to visit while going down” — and services all pending requests in its current direction before reversing (the SCAN algorithm behavior again), rather than naively servicing requests in the order they arrived, which would cause erratic, inefficient back-and-forth movement (e.g., go to floor 10, then immediately back down to floor 2, then back up to floor 8, because that’s the order requests happened to arrive in).

```

class Elevator {
    int id;

```

```

int currentFloor;
Direction direction = Direction.IDLE;
ElevatorState state = ElevatorState.IDLE;

private final TreeSet<Integer> upStops = new TreeSet<>(); // sorted ascending –
natural order for going up
private final TreeSet<Integer> downStops = new TreeSet<>(Comparator.reverseOrder()); //
sorted descending

void requestFloor(int floor) {
    if (floor > currentFloor) upStops.add(floor);
    else if (floor < currentFloor) downStops.add(floor);
    if (direction == Direction.IDLE) {
        direction = floor > currentFloor ? Direction.UP : Direction.DOWN;
    }
}

Integer nextStop() {
    if (direction == Direction.UP) {
        if (!upStops.isEmpty()) return upStops.first(); // nearest floor ABOVE,
in current direction
        if (!downStops.isEmpty()) { direction = Direction.DOWN; return
downStops.first(); } // reverse – SCAN
        direction = Direction.IDLE;
        return null;
    } else if (direction == Direction.DOWN) {
        if (!downStops.isEmpty()) return downStops.first(); // nearest floor BELOW,
in current direction
        if (!upStops.isEmpty()) { direction = Direction.UP; return upStops.first(); }
        direction = Direction.IDLE;
        return null;
    }
    return null;
}

void arriveAt(int floor) {
    currentFloor = floor;
    upStops.remove(floor);
    downStops.remove(floor);
}
}

```

Walking through the code. `upStops` is a `TreeSet` in natural (ascending) order, and `downStops` is a `TreeSet` in reverse (descending) order — both are kept sorted automatically by the data structure itself, which is what makes `.first()` always return “the nearest stop in the direction currently being serviced” in $O(\log n)$ rather than needing to scan and compare every pending floor on each call. `requestFloor()` routes a newly requested floor into whichever set is relevant based on whether it’s above or below the elevator’s current position. `nextStop()` is where the SCAN behavior lives: while `direction == UP`, it keeps pulling from `upStops` (the closest floor above, since the set is ascending-sorted) until that set is empty — only then does it check `downStops` and flip `direction` to `DOWN`, rather than ever jumping to a downward stop while upward stops still remain. This is exactly what prevents the erratic floor-10-then-floor-2-then-floor-8 behavior the answer warns about: all upward requests get serviced together, in floor order, before the elevator ever reverses.

Q199. How would you model the elevator’s state machine, and why is the State pattern (Q182) a natural fit here? States: `IDLE`, `MOVING_UP`, `MOVING_DOWN`, `DOORS_OPEN`, `OUT_OF_SERVICE` — each state has different valid transitions and different behavior for the same triggering event (e.g., a new floor request while `DOORS_OPEN` behaves differently than while `MOVING_UP`). The State pattern is a strong fit because it cleanly avoids a sprawling conditional (`if state == MOVING_UP and event == ... else if state == DOORS_OPEN and event == ...`) by encapsulating each state’s specific behavior in its own class, and makes adding a new state (like a `MAINTENANCE_MODE`) or new transition logic isolated and low-risk to the rest of the system.

```
interface ElevatorStateHandler {
    void onFloorRequest(ElevatorContext ctx, int floor);
    void onArrival(ElevatorContext ctx);
}

class DoorsOpenHandler implements ElevatorStateHandler {
    public void onFloorRequest(ElevatorContext ctx, int floor) {
        ctx.queueFloor(floor); // just queue it – doors must close before
        moving
    }
    public void onArrival(ElevatorContext ctx) { /* not applicable while doors are open */ }
}

class MovingHandler implements ElevatorStateHandler {
    public void onFloorRequest(ElevatorContext ctx, int floor) {
        ctx.queueFloor(floor); // same call, but no doors to worry about
        closing
    }
    public void onArrival(ElevatorContext ctx) {
        ctx.openDoors();
        ctx.setHandler(new DoorsOpenHandler()); // transition happens HERE, inside the
        state that knows it's next
    }
}

class ElevatorContext {
    private ElevatorStateHandler handler = new IdleHandler();
    private final Elevator elevator;

    ElevatorContext(Elevator elevator) { this.elevator = elevator; }

    void setHandler(ElevatorStateHandler handler) { this.handler = handler; }
    void queueFloor(int floor) { elevator.requestFloor(floor); }
    void openDoors() { elevator.state = ElevatorState.DOORS_OPEN; }

    void requestFloor(int floor) { handler.onFloorRequest(this,
    floor); } // delegates – no switch on state
    void arrive() { handler.onArrival(this); }
}

class IdleHandler implements ElevatorStateHandler {
    public void onFloorRequest(ElevatorContext ctx, int floor) { ctx.queueFloor(floor); }
    public void onArrival(ElevatorContext ctx) { /* n/a */ }
}
```

Walking through the code. `ElevatorContext.requestFloor()` doesn’t inspect what state the elevator is in and branch on it — it calls `handler.onFloorRequest(this, floor)`,

and whichever concrete handler is currently installed decides what that means. `onFloorRequest()` happens to do the same thing (`ctx.queueFloor(floor)`) in both `DoorsOpenHandler` and `MovingHandler` here, but the point is that it *doesn't have to* — a hypothetical `OutOfServiceHandler.onFloorRequest()` could simply do nothing, silently rejecting requests, without any other class needing to know that special case exists. The transition itself is visible in `MovingHandler.onArrival()`: after opening the doors (`ctx.openDoors()`), it calls `ctx.setHandler(new DoorsOpenHandler())` — the state that just finished is the one that decides what state comes next, exactly mirroring `PlacedState.next()` deciding to move to `ShippedState` in Q182. Adding `MAINTENANCE_MODE` later means writing one new `ElevatorStateHandler` implementation and wiring in the transitions that lead to and from it, with `ElevatorContext` itself needing no changes at all.

Q200. How would the design change to support “scale to hundreds of elevators across a large building” efficiently, given the naive dispatch algorithm scores every elevator for every request? At real scale, don't score every elevator against every request globally — partition elevators into zones/banks (as real skyscrapers already do physically — express vs. local banks serving different floor ranges) so a request's candidate elevator set is bounded rather than growing linearly with total building size, and consider maintaining each elevator's current cost/availability in a lightweight priority structure (updated incrementally as elevators move, similar in spirit to the leaderboard sorted-set idea from the HLD section) rather than recomputing every elevator's cost from scratch on every single new request.

Case Study 3: Design a Tic-Tac-Toe / Generic Board Game Framework

Q201. What's the key design insight that lets this generalize cleanly to other board games (Connect Four, etc.) rather than being hardcoded to a 3x3 grid? Abstract the board size and win-condition logic away from being hardcoded assumptions baked into game flow control. A `Board` class parameterized by dimensions, a `Player` class holding a symbol, and — critically — a separate `WinningStrategy` interface (Strategy pattern again) responsible only for checking whether the current board state satisfies a win condition, decoupled from how moves are made or whose turn it is. This means supporting Connect Four's “4 in a row including diagonals on a taller board” is a new `WinningStrategy` implementation and a resized `Board`, not a rewrite of the game loop.

```
class Board {
    private final char[][] grid;
    private final int size;

    Board(int size) {
        this.size = size;
        this.grid = new char[size][size];
        for (char[] row : grid) Arrays.fill(row, '-');
    }
}
```

```

void placeSymbol(int row, int col, char symbol) { grid[row][col] = symbol; }
char get(int row, int col) { return grid[row][col]; }
int getSize() { return size; }
boolean isFull() {
    for (char[] row : grid) for (char c : row) if (c == '-') return false;
    return true;
}
}

class Player {
    String name;
    char symbol;
}

class Move {
    int row, col;
    Player player;
    Move(int row, int col, Player player) { this.row = row; this.col = col; this.player =
player; }
}

class GameResult {
    enum Type { WIN, DRAW, ONGOING }
    Type type;
    Player winner;
    static GameResult win(Player p) { GameResult r = new GameResult(); r.type = Type.WIN; r.w
inner = p; return r; }
    static GameResult draw() { GameResult r = new GameResult(); r.type = Type.DRAW; return
r; }
    static GameResult ongoing() { GameResult r = new GameResult(); r.type = Type.ONGOING; ret
urn r; }
}

interface WinningStrategy {
    boolean checkWinner(Board board, Move lastMove);
}

class ThreeInARowStrategy implements WinningStrategy { // standard tic-tac-toe win
condition
    public boolean checkWinner(Board board, Move lastMove) {
        int n = board.getSize();
        char symbol = lastMove.player.symbol;
        // check the row and column the last move was placed in
        boolean rowWin = true, colWin = true;
        for (int i = 0; i < n; i++) {
            if (board.get(lastMove.row, i) != symbol) rowWin = false;
            if (board.get(i, lastMove.col) != symbol) colWin = false;
        }
        if (rowWin || colWin) return true;
        // check both diagonals only if the last move actually sits on one
        if (lastMove.row == lastMove.col) {
            boolean diagWin = true;
            for (int i = 0; i < n; i++) if (board.get(i, i) != symbol) diagWin = false;
            if (diagWin) return true;
        }
        if (lastMove.row + lastMove.col == n - 1) {
            boolean antiDiagWin = true;
            for (int i = 0; i < n; i++) if (board.get(i, n - 1 - i) != symbol) antiDiagWin =
false;
            if (antiDiagWin) return true;
        }
        return false;
    }
}

```

```

}

class Game {
    Board board;
    List<Player> players;
    WinningStrategy winStrategy;
    int currentPlayerIndex = 0;

    GameResult makeMove(int row, int col) {
        Player current = players.get(currentPlayerIndex);
        board.placeSymbol(row, col, current.symbol);
        Move lastMove = new Move(row, col, current);
        if (winStrategy.checkWinner(board, lastMove)) {
            return GameResult.win(current);
        }
        if (board.isFull()) return GameResult.draw();
        currentPlayerIndex = (currentPlayerIndex + 1) % players.size();
        return GameResult.ongoing();
    }
}

```

Walking through the code. Board is constructed with a `size` parameter rather than a hardcoded `3` — the same class works for a 3×3 tic-tac-toe grid or a larger board, with no special-casing inside Board itself. `ThreeInARowStrategy.checkWinner()` only checks the row, column, and (conditionally) diagonals that the *just-played* move (`lastMove`) actually touches — it doesn't rescan the entire board looking for any winning line anywhere, which would be wasteful after every single move; a win, if one now exists, must run through the square that was just played. The diagonal checks are additionally guarded (`lastMove.row == lastMove.col` for the main diagonal, `lastMove.row + lastMove.col == n - 1` for the anti-diagonal) so a move that doesn't sit on a diagonal at all skips that check entirely. `Game.makeMove()` is the orchestrator: it places the symbol, builds a `Move` record of what just happened, and hands both the `board` and `lastMove` to `winStrategy.checkWinner()` — it never contains win-condition logic itself. Swapping to Connect Four's "4 in a row, taller board, gravity-drop columns" would mean writing a `FourInARowStrategy` implementing the same `WinningStrategy` interface (plus adjusting how a move's `col` maps to a `row` under gravity) — `Game.makeMove()` doesn't change.

Q202. How do you validate a move efficiently (is the cell empty, is it this player's turn, is the game already over) without scattering these checks across the UI/controller layer? Centralize all move validation inside `Game.makeMove()` (or a dedicated `MoveValidator` if the rules grow complex enough to deserve their own class) rather than trusting the caller (UI layer) to only ever send legal moves — this is a defensive-design principle: the domain model should enforce its own invariants and never rely on callers behaving correctly, since UI code, test code, and future callers you haven't written yet should all be equally unable to corrupt game state.

```

class InvalidMoveException extends RuntimeException {
    InvalidMoveException(String message) { super(message); }
}

class Game {

```

```

Board board;
List<Player> players;
WinningStrategy winStrategy;
int currentPlayerIndex = 0;
boolean isOver = false;

GameResult makeMove(int row, int col) {
    validateMove(row, col); // single gate – every caller
    goes through this
    Player current = players.get(currentPlayerIndex);
    board.placeSymbol(row, col, current.symbol);
    Move lastMove = new Move(row, col, current);
    if (winStrategy.checkWinner(board, lastMove)) {
        isOver = true;
        return GameResult.win(current);
    }
    if (board.isFull()) { isOver = true; return GameResult.draw(); }
    currentPlayerIndex = (currentPlayerIndex + 1) % players.size();
    return GameResult.ongoing();
}

private void validateMove(int row, int col) {
    if (isOver) throw new InvalidMoveException("Game is already over");
    if (row < 0 || row >= board.getSize() || col < 0 || col >= board.getSize()) {
        throw new InvalidMoveException("Move is outside the board");
    }
    if (board.get(row, col) != '-') throw new InvalidMoveException("Cell is already
occupied");
    // note: no separate "is it this player's turn" check needed – makeMove always
    // uses players.get(currentPlayerIndex) internally, so a caller has no way to
    // move on someone else's behalf even if it wanted to
}
}

```

Walking through the code. `validateMove()` is a private method called as the very first line of `makeMove()` — every single call path into the game, whether from a REST controller, a test, or a future WebSocket handler, goes through the exact same three checks (game not already over, coordinates in bounds, cell not occupied) because there is no other way to place a move; `board.placeSymbol()` is never called directly by anything outside `Game`. Note the comment about whose turn it is: rather than needing a fourth explicit check, the design sidesteps the problem entirely — `makeMove()` always looks up `players.get(currentPlayerIndex)` itself rather than accepting a `Player` argument from the caller, so there is no parameter through which a caller could even attempt to claim “it’s my turn” when it isn’t; the class simply never asks the caller who they are. This is the defensive-design point made concrete: instead of trusting the UI to only enable the current player’s button, the domain model makes an illegal call (wrong turn, occupied cell, out-of-bounds, game already decided) throw, no matter which layer above it made the mistake.

Q203. How would you extend the design to support an AI opponent, without modifying the `Game` or `Board` classes? Introduce a `Player` abstraction with two implementations: `HumanPlayer` (move comes from external UI input) and `AIPlayer` (move is computed internally, e.g., via minimax with alpha-beta pruning for a game like tic-tac-toe/Connect Four). The `Game` class only ever calls a generic

`player.getNextMove(board)` method and doesn't know or care whether the implementation behind it is a human-driven UI callback or an AI search algorithm — again, depending on an interface/abstraction rather than a concrete implementation (DIP, Q165), which is the single idea that recurs across almost every one of these LLD case studies.

```
interface Player {
    char getSymbol();
    Move getNextMove(Board board);    // the single method Game actually calls –
    implementation-agnostic
}

class HumanPlayer implements Player {
    private final char symbol;
    private final InputSource input;    // e.g., wraps a UI event queue or console
    reader

    HumanPlayer(char symbol, InputSource input) { this.symbol = symbol; this.input = input; }

    public char getSymbol() { return symbol; }
    public Move getNextMove(Board board) {
        int[] coords = input.waitForNextClick();    // blocks until a human actually clicks a
        cell
        return new Move(coords[0], coords[1], this);
    }
}

class AIPlayer implements Player {
    private final char symbol;

    AIPlayer(char symbol) { this.symbol = symbol; }

    public char getSymbol() { return symbol; }
    public Move getNextMove(Board board) {
        return minimax(board, symbol);    // computed internally – no external
        input needed
    }

    private Move minimax(Board board, char symbol)
    { /* alpha-beta search over possible moves */ return null; }
}

class Game {
    Board board;
    List<Player> players;    // typed as the interface – could be
    Human, AI, or a mix
    WinningStrategy winStrategy;
    int currentPlayerIndex = 0;

    GameResult playNextTurn() {
        Player current = players.get(currentPlayerIndex);
        Move move =
        current.getNextMove(board);    // Game has no idea whether this took milliseconds or a
        human click
        return makeMove(move.row, move.col);
    }

    GameResult makeMove(int row, int col) { /* unchanged from Q202 */ return null; }
}
```

Walking through the code. `Player` is now an interface with one method the `Game` actually depends on: `getNextMove(Board board)`. `HumanPlayer.getNextMove()` blocks on `input.waitForNextClick()` — it's driven by an external event, taking however long a real person takes to click. `AIPlayer.getNextMove()` instead runs `minimax()` internally and returns immediately (relative to a human, at least) with a computed move — no external input source involved at all. `Game.playNextTurn()` calls `current.getNextMove(board)` identically regardless of which concrete `Player` is in `players.get(currentPlayerIndex)` — it has no branch checking “is this player human or AI.” A game with one `HumanPlayer` and one `AIPlayer` in the same `players` list works without `Game` ever needing to know which slot holds which kind of player; `Board` and `makeMove()` are completely untouched by this addition, exactly satisfying the “without modifying `Game` or `Board`” constraint in the question.

Case Study 4: Design a Library Management System

Q204. What are the core entities and the central relationship that most candidates model incorrectly at first? `Book` (a title/ISBN-level catalog entry — metadata like author, genre) vs. `BookCopy` / `BookItem` (a specific physical copy the library owns, with its own status: available/checked-out/reserved/lost) — the mistake most candidates make is modeling only `Book` and treating “checking out a book” as decrementing some count on it, rather than recognizing that a library owns multiple physical copies of the same title, each with independent state (copy #3 of “Dune” can be checked out while copy #1 is on the shelf). `Member` (a library patron, with borrowing limits/history), `Loan` / `Checkout` (links a specific `BookCopy` to a `Member` with due date), and `Reservation` (a member waiting for a currently-unavailable copy to become free).

```
class Book {                                     // title-level catalog entry – metadata only, no
    availability state
    String isbn;
    String title;
    String author;
}

enum CopyStatus { AVAILABLE, CHECKED_OUT, RESERVED, LOST }

class BookCopy {                                // ONE PHYSICAL COPY – this is what actually has a
    status
    String copyId;
    Book book;                                  // which title this copy belongs to
    CopyStatus status = CopyStatus.AVAILABLE;
}

class Member {
    String memberId;
    List<Loan> activeLoans = new ArrayList<>();
    int maxAllowed = 5;
    boolean hasOverdueBooks() {
        return activeLoans.stream().anyMatch(l -> l.dueDate.isBefore(LocalDate.now()));
    }
}

class Loan {
```

```

    BookCopy copy;
    Member member;
    LocalDate dueDate;
}

```

Walking through the code. `Book` holds only title-level metadata — `isbn`, `title`, `author` — and critically has no `status` field and no count of “how many are available.” `BookCopy` is the class that actually has a `status`, and it holds a reference to a `Book` (`book`) rather than being the same object — meaning a library with five physical copies of “Dune” has one `Book` object and five separate `BookCopy` objects, each independently `AVAILABLE`, `CHECKED_OUT`, etc. Checking out copy #3 flips `status` on that one `BookCopy` instance; copy #1’s `status` is completely unaffected, since they’re distinct objects rather than a shared counter on `Book`. This is the exact distinction the question flags as the common modeling mistake: a design that instead put an `availableCount` integer directly on `Book` couldn’t represent “which specific copy is where” (a copy that’s lost vs. one that’s merely checked out, or a copy sitting at a different branch — see Q207), because it collapsed several independently-stateful physical objects into a single number.

Q205. How do you enforce business rules like “a member can have at most 5 books checked out at once” or “overdue members can’t check out new books” cleanly? Encapsulate these as validation rules inside a dedicated `CheckoutService` (or a chain of validator objects, echoing Chain of Responsibility, Q183) that runs before a checkout is permitted, rather than scattering `if member.currentLoans.size() >= 5` checks across multiple entry points (UI, API, batch import) — every path that can initiate a checkout should go through the same validation gate, so a rule added once is enforced everywhere, and rules can be composed/reordered without touching unrelated code.

```

class CheckoutException extends RuntimeException {
    CheckoutException(String message) { super(message); }
}

interface CheckoutRule {
    void validate(Member member, BookCopy copy) throws CheckoutException;
}

class MaxBooksRule implements CheckoutRule {
    public void validate(Member member, BookCopy copy) {
        if (member.activeLoans.size() >= member.maxAllowed) {
            throw new CheckoutException("Checkout limit reached");
        }
    }
}

class OverdueRule implements CheckoutRule {
    public void validate(Member member, BookCopy copy) {
        if (member.hasOverdueBooks()) {
            throw new CheckoutException("Member has overdue books");
        }
    }
}

```

```

class CopyAvailableRule implements CheckoutRule {
    public void validate(Member member, BookCopy copy) {
        if (copy.status != CopyStatus.AVAILABLE) {
            throw new CheckoutException("This copy is not available");
        }
    }
}

class CheckoutService {
    private final List<CheckoutRule>
rules; // the chain – order and membership configured ONCE, here

    CheckoutService(List<CheckoutRule> rules) { this.rules = rules; }

    Loan checkout(Member member, BookCopy copy) {
        for (CheckoutRule rule : rules) {
            rule.validate(member, copy); // every rule runs – first
            // failure throws and stops checkout
        }
        copy.status = CopyStatus.CHECKED_OUT;
        Loan loan = new Loan();
        loan.copy = copy; loan.member = member; loan.dueDate = LocalDate.now().plusWeeks(3);
        member.activeLoans.add(loan);
        return loan;
    }
}

CheckoutService service = new CheckoutService(
    List.of(new CopyAvailableRule(), new OverdueRule(), new MaxBooksRule())
);

```

Walking through the code. `CheckoutRule` is a small interface with one method, `validate()`, that either does nothing (rule satisfied) or throws a `CheckoutException` (rule violated). `CheckoutService` holds a `List<CheckoutRule>` rather than hardcoding any specific checks inline, and `checkout()` loops through calling `rule.validate()` on each one in order — the loop itself has no idea how many rules there are or what any individual rule checks for. Every single caller — a UI button handler, a REST endpoint, a batch import script — is expected to go through `checkoutService.checkout(member, copy)` rather than mutating `copy.status` or `member.activeLoans` directly, which is what guarantees a rule enforced here is enforced everywhere, with no code path able to bypass it. Adding a new rule (say, a `MembershipExpiredRule`) means writing one new class and adding it to the `List.of(...)` at construction time — `CheckoutService.checkout()` itself never changes, and the rules can be freely reordered in that same list (checking cheap conditions like copy availability before more expensive ones) without touching any rule’s own code.

Q206. How would you design the reservation/hold queue so that when a checked-out book is returned, the right waiting member is notified fairly? Each `Book` (title-level) maintains an ordered queue (FIFO for fairness) of `Reservations`. On `BookCopy` return, the `LibraryService` checks if any reservation exists for that title; if so, it assigns the returning copy to the head of the queue (transitioning it to a “held for pickup” status with an expiry, rather than immediately checked-out, to give that member a window to actually come get it) instead of returning the copy to general

availability — and only if that hold also expires unclaimed does it cascade to the next member in the queue or finally return to open shelf availability.

```
class Reservation {
    Member member;
    LocalDateTime requestedAt;
}

class Book {
    String isbn;
    String title;
    String author;
    Deque<Reservation> reservationQueue = new
ArrayDeque<>(); // FIFO – offer at tail, poll from head
}

class LibraryService {
    void returnCopy(BookCopy copy) {
        Book book = copy.book;
        if (!book.reservationQueue.isEmpty()) {
            Reservation next = book.reservationQueue.poll(); // FIFO: earliest
            requester served first
            copy.status = CopyStatus.RESERVED; // held, not yet fully
            checked out
            scheduleHoldExpiry(copy, next.member); // e.g. 48-hour pickup
            window
            notifyMember(next.member, copy);
        } else {
            copy.status = CopyStatus.AVAILABLE; // no one waiting – back
            to open shelf
        }
    }

    void onHoldExpired(BookCopy copy, Member heldFor) {
        Book book = copy.book;
        if (!book.reservationQueue.isEmpty()) {
            returnCopy(copy); // cascade to the NEXT
            member in line
        } else {
            copy.status = CopyStatus.AVAILABLE;
        }
    }

    private void scheduleHoldExpiry(BookCopy copy, Member member)
{ /* schedule a delayed job */ }
    private void notifyMember(Member member, BookCopy copy) { /* send notification */ }
}
```

Walking through the code. `Book.reservationQueue` is a `Deque` used strictly FIFO: `returnCopy()` calls `.poll()` (removes and returns the head — the *earliest* reservation) rather than treating the queue as a stack or picking arbitrarily, which is exactly what “fairness” means here. When a copy is returned and someone is waiting, `returnCopy()` does not set the copy back to `AVAILABLE` — it sets it to `RESERVED` and calls `scheduleHoldExpiry()`, deliberately giving the specific member at the head of the queue an exclusive window to claim it rather than opening it to whoever checks the catalog first. `onHoldExpired()` is the callback for when that window lapses unclaimed: it checks the queue again and, if someone else is now waiting, calls `returnCopy()` recursively —

which naturally repeats the exact same “poll the next reservation, set a fresh hold” logic for the next person in line, cascading through the queue one member at a time until either someone claims the hold or the queue empties out, at which point the copy finally becomes genuinely AVAILABLE to anyone.

Q207. How does this design change if the library has multiple branches sharing a catalog, with inter-branch book transfers? Add a Branch entity that owns BookCopy instances (each copy belongs to exactly one branch at a time), and introduce a TransferRequest concept for moving a copy between branches (e.g., to fulfill a hold placed by a member at a branch that doesn’t have that title in stock) — this is essentially a small internal workflow/state machine on top of the existing BookCopy status field (add IN_TRANSIT as a status), and demonstrates to an interviewer that you can extend an existing model incrementally rather than needing to redesign it from scratch when requirements grow.

```
enum CopyStatus { AVAILABLE, CHECKED_OUT, RESERVED, LOST, IN_TRANSIT } // one new value
added – nothing removed

class Branch {
    String branchId;
    String name;
}

class BookCopy {
    String copyId;
    Book book;
    Branch currentBranch; // NEW – which branch physically holds this copy
                           right now
    CopyStatus status = CopyStatus.AVAILABLE;
}

class TransferRequest {
    BookCopy copy;
    Branch fromBranch;
    Branch toBranch;
    LocalDateTime requestedAt;
}

class TransferService {
    TransferRequest initiateTransfer(BookCopy copy, Branch toBranch) {
        if (copy.status != CopyStatus.AVAILABLE) {
            throw new IllegalStateException("Only available copies can be transferred");
        }
        copy.status = CopyStatus.IN_TRANSIT; // reuses the SAME status field
        every other flow checks
        TransferRequest request = new TransferRequest();
        request.copy = copy; request.fromBranch = copy.currentBranch; request.toBranch =
toBranch;
        return request;
    }

    void completeTransfer(TransferRequest request) {
        request.copy.currentBranch = request.toBranch;
        request.copy.status =
CopyStatus.AVAILABLE; // arrives, becomes available again – AT the new branch
    }
}
```

```
}  
}
```

Walking through the code. The change is deliberately additive rather than a rewrite: `BookCopy` gains one new field (`currentBranch`) and `CopyStatus` gains one new value (`IN_TRANSIT`) — the existing `CheckoutRule.validate()` logic from Q205 that checks `copy.status != CopyStatus.AVAILABLE` continues to work unmodified, since a copy that's `IN_TRANSIT` simply fails that same existing check the same way a `CHECKED_OUT` copy would, correctly preventing it from being checked out mid-transfer without anyone needing to add a new rule. `TransferService.initiateTransfer()` guards that only an `AVAILABLE` copy can begin a transfer (you can't transfer a copy someone currently has checked out), flips its status to `IN_TRANSIT`, and records a `TransferRequest`. `completeTransfer()` — called once the physical copy has actually arrived — updates `currentBranch` to the destination and flips status back to `AVAILABLE`, at which point it's checkable again, but now at the new branch. This is the incremental-extension point the answer is making: none of `Loan`, `Member`, `CheckoutService`, or the reservation-queue logic in Q206 needed to change at all to support this entirely new capability.

Case Study 5: Design an Expense-Sharing App (like Splitwise)

Q208. What are the core entities, and what's the central computational problem this system solves? `User`, `Group` (a set of users sharing expenses, e.g., roommates or a trip), `Expense` (paid by one user, split among a set of users in some proportion), and critically a balance/settlement computation: given a history of expenses, determine the minimal, simplified set of “who owes whom how much” transactions — this simplification (minimizing the number of actual payments needed to settle all debts within a group) is the classic algorithmic centerpiece interviewers probe for.

Q209. Why shouldn't split logic be hardcoded as “equal split” inside the `Expense` class, and how do you generalize it? Real systems need equal splits, exact-amount splits, and percentage splits — hardcoding one means the class needs modification (violating OCP) every time a new split type is needed. Use a `SplitStrategy` interface (Strategy pattern, again the recurring theme) with `EqualSplitStrategy`, `ExactAmountSplitStrategy`, and `PercentageSplitStrategy` implementations, each responsible for computing each participant's owed share given the total expense amount and the relevant input (nothing for equal split, explicit amounts for exact split, percentages for percentage split) — with validation (e.g., exact amounts must sum to the total; percentages must sum to 100) living inside each concrete strategy, since the validation rule is itself specific to the split type.

```
class User {  
    String userId;  
    String name;  
}  
  
class Share {
```

```

        User user;
        double amountOwed;
    }

    interface SplitStrategy {
        List<Share> computeShares(double totalAmount, List<User> participants, Map<User, Double>
input);
    }

    class EqualSplitStrategy implements SplitStrategy {
        public List<Share> computeShares(double totalAmount, List<User> participants, Map<User, D
ouble> input) {
            double perPerson = totalAmount / participants.size();
            List<Share> shares = new ArrayList<>();
            for (User u : participants) {
                Share s = new Share(); s.user = u; s.amountOwed = perPerson;
                shares.add(s);
            }
            return shares;
        }
    }

    class ExactAmountSplitStrategy implements SplitStrategy {
        public List<Share> computeShares(double totalAmount, List<User> participants, Map<User, D
ouble> input) {
            double sum = input.values().stream().mapToDouble(Double::doubleValue).sum();
            if (Math.abs(sum - totalAmount) > 0.01) {
                throw new IllegalArgumentException("Exact amounts must sum to the total expense")
            ;
            }
            List<Share> shares = new ArrayList<>();
            for (User u : participants) {
                Share s = new Share(); s.user = u; s.amountOwed = input.get(u);
                shares.add(s);
            }
            return shares;
        }
    }

    class PercentageSplitStrategy implements SplitStrategy {
        public List<Share> computeShares(double totalAmount, List<User> participants, Map<User, D
ouble> input) {
            double sumPercent = input.values().stream().mapToDouble(Double::doubleValue).sum();
            if (Math.abs(sumPercent - 100.0) > 0.01) {
                throw new IllegalArgumentException("Percentages must sum to 100");
            }
            List<Share> shares = new ArrayList<>();
            for (User u : participants) {
                Share s = new Share(); s.user = u; s.amountOwed = totalAmount * (input.get(u) / 1
00.0);
                shares.add(s);
            }
            return shares;
        }
    }

    class Expense {
        User paidBy;
        double totalAmount;
        List<Share> shares; // computed once, at creation, by whichever
SplitStrategy was used

        Expense(User paidBy, double totalAmount, List<User> participants,

```

```

        Map<User, Double> input, SplitStrategy strategy) {
    this.paidBy = paidBy;
    this.totalAmount = totalAmount;
    this.shares = strategy.computeShares(totalAmount, participants, input);    //
    delegates – no split math here
    }
}

```

Walking through the code. `SplitStrategy.computeShares()` takes the total amount, the participants, and a generic `Map<User, Double> input` — the meaning of that map’s values differs entirely by strategy (nothing meaningful for equal split, dollar amounts for exact split, percentages for percentage split), but the method signature stays uniform, which is what lets `Expense`’s constructor call `strategy.computeShares(...)` without knowing which concrete strategy it received. `EqualSplitStrategy` ignores `input` entirely and just divides evenly. `ExactAmountSplitStrategy` validates that the dollar amounts in `input` actually sum to `totalAmount` (using a small epsilon, `0.01`, to tolerate floating-point rounding rather than requiring exact equality) before trusting them — that validation lives *inside* this strategy specifically, because “must sum to the total” only makes sense for this split type; `PercentageSplitStrategy` has its own, different validation (“must sum to 100”). `Expense`’s constructor contains zero arithmetic itself — it just calls `strategy.computeShares(...)` and stores the result. Adding a fourth split type (say, “by shares/weights,” common for roommates splitting rent unevenly by room size) means writing one new class implementing `SplitStrategy`, with `Expense` untouched.

Q210. Describe the algorithm for simplifying group debts into the minimum number of settling transactions. First compute each member’s net balance (total paid minus total owed share, across all expenses) — members end up either net creditors (positive, owed money) or net debtors (negative, owe money). Then greedily match the largest creditor against the largest debtor repeatedly: the debtor pays the creditor `min(|debtor's balance|, creditor's balance)`, both balances update, and whichever hits zero drops out of consideration; repeat until all balances are zero. This greedy max-vs-max matching provably minimizes the total number of transactions (a known result — it’s optimal, not just a heuristic), and is far better UX than settling every individual pairwise expense relationship separately, which could require far more transactions than necessary.

```

class Transaction {
    User from;    // the debtor – pays
    User to;      // the creditor – receives
    double amount;
    Transaction(User from, User to, double amount) { this.from = from; this.to = to; this.amount = amount; }
}

List<Transaction> simplifyDebts(Map<User, Double> netBalances) {
    PriorityQueue<Map.Entry<User, Double>> creditors = // max-heap by balance – largest creditor first
    new PriorityQueue<>((a, b) -> Double.compare(b.getValue(), a.getValue()));
    PriorityQueue<Map.Entry<User, Double>> debtors = // min-heap by balance – most-negative (largest debtor) first
}

```

```

    new PriorityQueue<>((a, b) -> Double.compare(a.getValue(), b.getValue()));

    for (Map.Entry<User, Double> entry : netBalances.entrySet()) {
        if (entry.getValue() > 0.01) creditors.add(entry);
        else if (entry.getValue() < -0.01) debtors.add(entry);
        // balances within epsilon of zero are already settled – skip
    }

    List<Transaction> result = new ArrayList<>();
    while (!creditors.isEmpty() && !debtors.isEmpty()) {
        var creditor = creditors.poll();    // largest remaining creditor
        var debtor = debtors.poll();        // largest remaining debtor

        double amount = Math.min(creditor.getValue(), -
        debtor.getValue()); // never overpay either side
        result.add(new Transaction(debtor.getKey(), creditor.getKey(), amount));

        double remainingCredit = creditor.getValue() - amount;
        double remainingDebt = debtor.getValue() + amount;

        if (remainingCredit > 0.01) creditors.add(Map.entry(creditor.getKey(),
        remainingCredit)); // still owed – re-queue
        if (remainingDebt < -0.01) debtors.add(Map.entry(debtor.getKey(),
        remainingDebt));   // still owes – re-queue
    }
    return result;
}

```

Walking through the code. `creditors` is a max-heap (largest positive balance popped first) and `debtors` is a min-heap over negative balances (most negative — i.e., owes the most — popped first); building these from `netBalances` also filters out anyone already settled (within a small epsilon of zero), so the loop never wastes a transaction on someone who owes/is owed effectively nothing. Each iteration pops the single largest creditor and single largest debtor and settles `amount = min(creditor.getValue(), -debtor.getValue())` between them directly — this greedy “biggest vs. biggest” pairing is what the theory behind this algorithm guarantees minimizes the total transaction count, rather than, say, settling debts in the order expenses happened to occur. After the payment, `remainingCredit` and `remainingDebt` are recalculated; whichever side didn’t fully zero out (the larger of the two balances, since `amount` was the smaller one) gets pushed back onto its heap to be matched again in a later iteration, while the side that hit (approximately) zero simply isn’t re-added and drops out of consideration. The loop terminates once either heap empties, at which point every balance has been reduced to zero by the minimum number of `Transaction`s the greedy approach can produce.

Q211. How would you handle an expense being edited or deleted after other expenses/settlements have already occurred on top of it? Rather than mutating balances in place (error-prone and loses history), treat expenses as an append-only event log (a smaller-scale application of event sourcing, Q66) and recompute affected balances from the event history — an edit is really “the original expense event plus a correction event,” and a deletion is a reversal event, never an in-place mutation of history. This keeps the system auditable (a user can always see exactly why their

balance is what it is) and avoids subtle bugs from partially-applied in-place balance updates when edits happen concurrently with other activity.

```
abstract class ExpenseEvent {
    String eventId;
    LocalDateTime timestamp;
}

class ExpenseCreatedEvent extends ExpenseEvent {
    Expense expense;
}

class ExpenseEditedEvent extends ExpenseEvent {
    String originalExpenseId;
    Expense revisedExpense;           // the NEW correct version – original event is
never overwritten
}

class ExpenseDeletedEvent extends ExpenseEvent {
    String originalExpenseId;
}

class GroupLedger {
    private final List<ExpenseEvent> events = new
ArrayList<>(); // append-only – nothing is ever removed or edited in place

    void recordCreation(Expense expense) {
        ExpenseCreatedEvent e = new ExpenseCreatedEvent();
        e.expense = expense; e.timestamp = LocalDateTime.now();
        events.add(e);
    }

    void recordEdit(String originalExpenseId, Expense revisedExpense) {
        ExpenseEditedEvent e = new ExpenseEditedEvent();
        e.originalExpenseId = originalExpenseId; e.revisedExpense = revisedExpense; e.timestamp = LocalDateTime.now();
        events.add(e);           // the ORIGINAL ExpenseCreatedEvent stays in
the log, untouched
    }

    void recordDeletion(String originalExpenseId) {
        ExpenseDeletedEvent e = new ExpenseDeletedEvent();
        e.originalExpenseId = originalExpenseId; e.timestamp = LocalDateTime.now();
        events.add(e);
    }

    Map<User, Double> computeCurrentBalances()
    {
        // recomputed from the log, never stored+mutated
        Map<String, Expense> currentState = new LinkedHashMap<>(); // expenseId -> latest
active version
        for (ExpenseEvent event : events) {
            if (event instanceof ExpenseCreatedEvent e) {
                currentState.put(e.expense.expenseId, e.expense);
            } else if (event instanceof ExpenseEditedEvent e) {
                currentState.put(e.originalExpenseId, e.revisedExpense); // replace, don't
mutate the old one
            } else if (event instanceof ExpenseDeletedEvent e) {
                currentState.remove(e.originalExpenseId);           // reversal –
drops out of the balance calc
            }
        }
    }
}
```

```

        return sumSharesIntoBalances(currentState.values());
    }

    private Map<User, Double> sumSharesIntoBalances(Collection<Expense> activeExpenses) { /*
sums paidBy vs. shares */ return null; }
}

```

Walking through the code. `GroupLedger.events` only ever grows via `add()` — there is no method anywhere that reaches into a past `ExpenseCreatedEvent` and changes its fields. `recordEdit()` doesn't touch the original event; it appends a brand-new `ExpenseEditedEvent` that references the `originalExpenseId` and carries the corrected `revisedExpense` — the full history of “what this expense used to say, and what it was changed to” is preserved in the log itself. `computeCurrentBalances()` is where the actual current state gets derived: it replays every event from the start, building up a `currentState` map keyed by expense ID. A `Created` event inserts an entry; an `Edited` event for the same ID *replaces* that entry (not merges or patches it — the whole revised `Expense` object supersedes the old one in the map); a `Deleted` event removes the entry from `currentState` entirely, so that expense's shares simply don't contribute to `sumSharesIntoBalances()` at all. Because balances are recomputed fresh from this replay every time rather than being a running total incrementally patched in place, there's no way for a concurrent edit and an unrelated new expense to interleave into a corrupted partial-update state — each call to `computeCurrentBalances()` either sees an event or it doesn't, with no half-applied intermediate.

Case Study 6: Design an In-Memory LRU Cache

Q212. What two data structures combine to give $O(1)$ `get` and `put` operations, and why does each alone fall short? A hash map (key $\rightarrow$ node reference) alone gives $O(1)$ lookup but no way to efficiently track/update recency order. A doubly linked list alone can maintain order (move a node to the front on access) in $O(1)$ once you have a reference to the node, but finding that node by key would be $O(n)$ without the hash map. Combined: the hash map gives $O(1)$ access to any node's location, and the doubly linked list (most-recently-used at head, least-recently-used at tail) gives $O(1)$ removal/reinsertion at any position once you have that node reference — together, both `get` and `put` are $O(1)$.

```

class LRUCache<K, V> {
    private class Node {
        K key; V value; Node prev, next;
    }

    private final int capacity;
    private final Map<K, Node> map = new HashMap<>(); // O(1) lookup: key -> node
reference
    private final Node head = new Node(), tail = new Node(); // dummy sentinels (Q213)

    LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }
}

```

```

    }

    V get(K key) {
        Node node = map.get(key);
        if (node == null) return null;
        moveToFront(node);           // accessing a key makes it most-recently-used
        return node.value;
    }

    void put(K key, V value) {
        if (map.containsKey(key)) {
            Node node = map.get(key);
            node.value = value;
            moveToFront(node);
            return;
        }
        if (map.size() == capacity) {
            Node lru = tail.prev;     // the node right before the tail sentinel =
            // least recently used
            remove(lru);
            map.remove(lru.key);
        }
        Node node = new Node();
        node.key = key; node.value = value;
        map.put(key, node);
        addToFront(node);
    }

    private void moveToFront(Node node) { remove(node); addToFront(node); }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addToFront(Node node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }
}

```

Walking through the code. `map` gives $O(1)$ lookup from a `key` straight to its `Node` — without it, finding “the node for this key” to update its position would require walking the linked list, an $O(n)$ operation. The linked list itself (`head` / `tail` sentinels, and each `Node`’s `prev` / `next`) maintains recency order: `head.next` is always the most-recently-used entry, `tail.prev` is always the least-recently-used. `remove(node)` re-links `node.prev` and `node.next` around the node being removed — this works in $O(1)$ *given a direct reference to the node*, which is exactly what `map.get(key)` supplied. `addToFront(node)` inserts a node right after `head`. `moveToFront()` composes the two (`remove` then `addToFront`), and both `get()` and `put()` call it on every access — a `get()` doesn’t just read a value, it also promotes that entry to most-recently-used, which is the defining behavior of an LRU cache. `put()`’s eviction branch (`if (map.size() == capacity)`) reads `tail.prev` — the node immediately before the tail sentinel — as the least-recently-used entry to evict, removes it from both the list and the map, and only

then inserts the new entry; every one of these steps (map lookup, list removal, list insertion) is $O(1)$, which is what makes the overall operation $O(1)$.

Q213. Why use dummy head/tail sentinel nodes instead of tracking head / tail as nullable references directly? Sentinels eliminate a large class of null-check edge cases (empty list, removing the only element, inserting into an empty list) that would otherwise need special-cased handling in `addToFront` / `remove` — with sentinels, every real node always has a valid `prev` and `next` to link against (even the first real node's `prev` is the head sentinel, never `null`), which meaningfully simplifies the code and removes a common source of off-by-one/null-pointer bugs in a live-coding setting under time pressure.

```
// WITHOUT sentinels – every operation needs explicit null-checks for the empty-list /
single-node cases
class NullableHeadTailList<K, V> {
    Node head, tail; // nullable – the empty list is head == null && tail == null

    void addToFront(Node node) {
        if (head == null) { // special case #1: list was empty
            head = tail = node;
            node.prev = node.next = null;
        } else { // the "normal" case
            node.next = head;
            head.prev = node;
            head = node;
        }
    }

    void remove(Node node) {
        if (node.prev == null && node.next == null) { head = tail =
null; } // special case: only node
        else if (node.prev == null) { head = node.next; head.prev =
null; } // special case: removing head
        else if (node.next == null) { tail = node.prev; tail.next =
null; } // special case: removing tail
        else { node.prev.next = node.next; node.next.prev = node.prev; } // the
"normal" case
    }
}

// WITH sentinels – from Q212 – no special cases at all:
private void addToFront(Node node) {
    node.next = head.next; // head is NEVER null – it's a permanent sentinel object
    node.prev = head;
    head.next.prev = node; // head.next is NEVER null either – even on an "empty" list
    it points at tail
    head.next = node;
}
```

Walking through the code. The sentinel-free version needs `addToFront()` to check `if (head == null)` as a genuinely separate code path, because on an empty list there's no existing `head` node to link the new node in front of — the new node *becomes* both `head` and `tail` simultaneously, a case with different logic from every subsequent insertion. `remove()` is worse: it needs three separate special cases (removing the only node, removing the current head, removing the current tail) on top of the normal middle-of-

list case, since each of those boundary positions has a `null` neighbor that would otherwise cause a `NullPointerException` if treated uniformly. In the sentinel version from Q212, `head` and `tail` are permanent dummy `Node` objects that never represent real cache entries and are never `null` — even when the cache is genuinely empty, `head.next` still points at `tail` (set up once in the constructor), so `addToFront()`’s line `node.next = head.next` and `head.next.prev = node` work identically whether the cache currently has zero entries or a thousand. There is exactly one code path for insertion and one for removal, with no `if (head == null)` anywhere, which is precisely the reduction in edge cases the answer describes — valuable specifically because an LLD interview is live coding under time pressure, where an unnecessary special case is an unnecessary chance to introduce a bug live.

Q214. How would you make this LRU cache thread-safe for concurrent access, and what’s the performance trade-off of the simplest approach? The simplest approach wraps `get / put` in a single lock (e.g., synchronized methods, or a single `ReentrantLock`) — correct, but serializes all access, so under high concurrency this cache becomes a bottleneck (only one thread can read or write at a time, even though logically most `get` calls don’t need to block each other). A more advanced approach uses a `ReadWriteLock` (multiple concurrent reads, exclusive writes) — but note that even a “pure read” `get()` here isn’t really read-only, since it mutates the linked-list order as a side effect of tracking recency, which undermines a naive read-write lock’s benefit unless you either accept slightly relaxed recency tracking under concurrency or use a more sophisticated concurrent structure (e.g., sharding the cache into multiple independently-locked segments, similar in spirit to Java’s `ConcurrentHashMap` internals).

```
class ThreadSafeLRUCache<K, V> {
    private final LRUCache<K, V> delegate;           // wraps the plain LRUCache from
    Q212 – no logic duplicated
    private final ReentrantLock lock = new ReentrantLock(); // ONE lock for the whole cache
    – the simplest approach

    ThreadSafeLRUCache(int capacity) { delegate = new LRUCache<>(capacity); }

    V get(K key) {
        lock.lock();
        try {
            return
            delegate.get(key);           // get() ALSO mutates list order – so it needs the lock too, not just
            put()
        } finally {
            lock.unlock();               // unlock in finally – always releases, even if
            get() throws
        }
    }

    void put(K key, V value) {
        lock.lock();
        try {
            delegate.put(key, value);
        } finally {
            lock.unlock();
        }
    }
}
```

```
}
}
```

Walking through the code. `ThreadSafeLRUCache` doesn't reimplement the LRU logic — it wraps the plain `LRUCache` from Q212 and adds exactly one `ReentrantLock` shared by both `get()` and `put()`. The critical detail the answer flags is that `get()` needs the lock too, not just `put()` — because `delegate.get(key)` internally calls `moveToFront()`, which mutates the linked list's `prev / next` pointers. If `get()` weren't locked, two threads calling `get()` on different keys “simultaneously” could corrupt the shared linked-list structure (one thread's `remove()` interleaving with another's `addToFront()` mid-pointer-update), even though neither thread is doing what looks like a “write” from the caller's perspective. The `try / finally` around each locked section guarantees `unlock()` runs even if `delegate.get()` or `delegate.put()` throws, which matters because a `ReentrantLock` left locked by a thread that exited abnormally would permanently block every other thread from ever acquiring it. The trade-off the answer names is visible directly in this code: every `get()` call from every thread, even ones touching completely unrelated keys, funnels through the same single `lock` — under high concurrency, that serialization is the bottleneck, which is what motivates the sharded/segmented alternative mentioned for more advanced designs.

Q215. How would you extend this to an LFU (Least Frequently Used) cache instead, and why is $O(1)$ LFU harder to achieve than $O(1)$ LRU? LFU needs to evict the least-frequently-accessed item, with ties broken by recency. The $O(1)$ approach uses a hash map of `key -> node` (as before) plus a hash map of `frequency -> doubly-linked-list of nodes at that frequency` plus tracking the current minimum frequency — on access, a node moves from its current frequency's list to the next frequency's list (updating the min-frequency tracker if its old list becomes empty and it was the minimum). It's harder than LRU because you're now maintaining an ordered collection of lists (one per frequency level) rather than a single list, and must correctly maintain the “current minimum frequency” pointer as items move between frequency buckets — meaningfully more bookkeeping than LRU's single-list-reordering, which is exactly why interviewers consider $O(1)$ LFU a noticeably harder follow-up than LRU.

```
class LFUCache<K, V> {
    private class Node {
        K key; V value; int freq = 1;
    }

    private final int capacity;
    private int minFreq = 0;
    private final Map<K, Node> keyToNode = new HashMap<>(); // O(1):
    key -> node
    private final Map<Integer, LinkedHashSet<Node>> freqToNodes = new HashMap<>(); // O(1):
    freq -> ordered set of nodes at that freq

    LFUCache(int capacity) { this.capacity = capacity; }

    V get(K key) {
        Node node = keyToNode.get(key);
        if (node == null) return null;
        touch(node); // access bumps frequency
    }
}
```

```

        return node.value;
    }

    void put(K key, V value) {
        if (capacity == 0) return;
        if (keyToNode.containsKey(key)) {
            Node node = keyToNode.get(key);
            node.value = value;
            touch(node);
            return;
        }
        if (keyToNode.size() == capacity) {
            Node evictee =
freqToNodes.get(minFreq).iterator().next(); // first node at the MINIMUM frequency
            freqToNodes.get(minFreq).remove(evictee);
            keyToNode.remove(evictee.key);
        }
        Node node = new Node();
        node.key = key; node.value = value; node.freq = 1;
        keyToNode.put(key, node);
        freqToNodes.computeIfAbsent(1, k -> new LinkedHashSet<>()).add(node);
        minFreq = 1; // a brand-new node always resets the minimum to 1
    }

    private void touch(Node node) {
        int oldFreq = node.freq;
        freqToNodes.get(oldFreq).remove(node); // leave the old frequency
        bucket
        if (freqToNodes.get(oldFreq).isEmpty() && minFreq == oldFreq) {
            minFreq++; // that bucket is now empty
            AND it was the minimum - bump it
        }
        node.freq++;
        freqToNodes.computeIfAbsent(node.freq, k -> new LinkedHashSet<>()).add(node); //
        join the new frequency bucket
    }
}

```

Walking through the code. `keyToNode` plays the same role as in the plain LRU cache — $O(1)$ lookup by key. The new structure is `freqToNodes`, a map from an integer frequency to a `LinkedHashSet<Node>` of every node currently at that exact access count (a `LinkedHashSet` rather than a plain `HashSet` so that, within one frequency bucket, insertion order is preserved — this is what breaks ties by recency, since `iterator().next()` on it returns the *oldest* node at that frequency). `touch()` is the core operation: it removes the node from its current frequency's set, and — the tricky part — checks `if (freqToNodes.get(oldFreq).isEmpty() && minFreq == oldFreq)`; only when the bucket the node just left is now empty *and* that bucket was tracking the minimum frequency does `minFreq` need to increment, because if other nodes remain at `oldFreq`, or if `oldFreq` wasn't the minimum to begin with, the true minimum hasn't changed. The node's `freq` is then incremented and it's inserted into the (possibly new) bucket for that frequency. Eviction in `put()` reads `freqToNodes.get(minFreq).iterator().next()` — the oldest node in the minimum-frequency bucket — combining both LFU's frequency requirement and the recency tiebreak in one lookup. Compare the bookkeeping load to `LRUCache.moveToFront()` in Q212: LRU updates one linked list's pointers per access; LFU here updates two hash map entries, conditionally adjusts a separate `minFreq`

counter, and moves the node between two different `LinkedHashSet` buckets — meaningfully more moving parts kept consistent on every single access, which is exactly the extra bookkeeping the answer is describing.

Case Study 7: Design an In-Memory Rate Limiter (as a standalone class, connecting back to Q84–Q86)

Q216. How would you implement a per-user token bucket rate limiter as a standalone, thread-safe class (not distributed — single process)?

```
class TokenBucketRateLimiter {
    private final long capacity;
    private final double refillRatePerMs;
    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>(); // thread-safe
    // map — different users never contend

    private static class Bucket {
        double tokens;
        long lastRefillTimestamp;
    }

    TokenBucketRateLimiter(long capacity, double refillRatePerSecond) {
        this.capacity = capacity;
        this.refillRatePerMs = refillRatePerSecond / 1000.0;
    }

    boolean allowRequest(String userId) {
        Bucket bucket = buckets.computeIfAbsent(userId, k -> {
            Bucket b = new Bucket();
            b.tokens = capacity;
            b.lastRefillTimestamp = System.currentTimeMillis();
            return b;
        });
        synchronized (bucket) { // lock per-BUCKET, not globally
            long now = System.currentTimeMillis();
            long elapsed = now - bucket.lastRefillTimestamp;
            bucket.tokens = Math.min(capacity, bucket.tokens + elapsed * refillRatePerMs); //
            // refill, capped at capacity
            bucket.lastRefillTimestamp = now;
            if (bucket.tokens >= 1) {
                bucket.tokens -= 1;
                return true;
            }
            return false;
        }
    }
}
```

Note the per-bucket (not global) synchronization — locking per-user-bucket rather than around the entire method means requests from different users never contend on the same lock, which matters a great deal under high concurrency across many distinct users.

Walking through the code. `buckets` is a `ConcurrentHashMap`, which already gives thread-safe reads/writes to the map structure itself (adding a new user's bucket,

looking up an existing one) without any additional locking. `computeIfAbsent()` atomically creates a fresh `Bucket` — initialized to full (`tokens = capacity`) — the first time a given `userId` is seen, and simply returns the existing one on every subsequent call. The actual rate-limiting logic happens inside `synchronized (bucket)` — critically, `synchronized` on the specific `Bucket` object for *this* user, not on `this` (the whole `TokenBucketRateLimiter`) or on some shared lock object. That means a request from user A and a request from user B can execute their token-bucket updates fully in parallel, on two different CPU cores, with zero contention — only two concurrent requests from the *same* user would ever actually block each other, which is correct, since it's exactly those requests that are racing to consume from the same finite token pool. Inside the lock: elapsed time since the last refill is computed, converted into new tokens via `refillRatePerMs` and added back in (capped at `capacity` so a long-idle bucket doesn't accumulate unbounded credit), and then the request is allowed only if at least one token remains, in which case that token is immediately deducted — the check and the deduction happen in the same `synchronized` block, so two threads can't both observe "1 token available" and both proceed, overshooting the limit.

Q217. How would you design this class so it can support multiple rate-limiting algorithms (token bucket, sliding window) interchangeably, tying back to the Strategy pattern?

```
interface RateLimitingAlgorithm {
    boolean allowRequest(String key);
}

class TokenBucketAlgorithm implements RateLimitingAlgorithm {
    private final long capacity;
    private final double refillRatePerMs;
    private final Map<String, TokenBucketRateLimiter.Bucket> buckets = new ConcurrentHashMap<>();

    // ... same bucket-refill logic as Q216, now behind the shared interface ...
    public boolean allowRequest(String key) { /* identical body to Q216's allowRequest */ return true; }

    TokenBucketAlgorithm(long capacity, double refillRatePerSecond) {
        this.capacity = capacity;
        this.refillRatePerMs = refillRatePerSecond / 1000.0;
    }
}

class SlidingWindowAlgorithm implements RateLimitingAlgorithm {
    private final int maxRequests;
    private final long windowMillis;
    private final Map<String, Deque<Long>> requestLog = new ConcurrentHashMap<>(); // per-key timestamp queue

    SlidingWindowAlgorithm(int maxRequests, long windowMillis) {
        this.maxRequests = maxRequests; this.windowMillis = windowMillis;
    }

    public boolean allowRequest(String key) {
        Deque<Long> log = requestLog.computeIfAbsent(key, k -> new ArrayDeque<>());
        synchronized (log) {
            long now = System.currentTimeMillis();
            while (!log.isEmpty() && log.peekFirst() < now - windowMillis) {

```

```

        log.pollFirst(); // drop timestamps that have slid
        outside the window
    }
    if (log.size() < maxRequests) {
        log.addLast(now);
        return true;
    }
    return false;
}
}

class RateLimiter { // the FACADE – configured with
    whichever algorithm
    private final RateLimitingAlgorithm algorithm;

    RateLimiter(RateLimitingAlgorithm algorithm) { this.algorithm = algorithm; }

    boolean allowRequest(String key) { return algorithm.allowRequest(key); } // never
    branches on which algorithm
}

RateLimiter limiter = new RateLimiter(new TokenBucketAlgorithm(10, 5)); // 10 tokens,
    refill 5/sec
// or, interchangeably:
RateLimiter slidingLimiter = new RateLimiter(new SlidingWindowAlgorithm(100, 60_000)); //
    100 req / 60s window

```

Walking through the code. `RateLimitingAlgorithm` is a one-method interface — `allowRequest(String key)` — and both `TokenBucketAlgorithm` and `SlidingWindowAlgorithm` implement it with entirely different internal data structures and logic.

`TokenBucketAlgorithm` wraps the exact same refill-based bucket logic from Q216 behind this interface. `SlidingWindowAlgorithm` instead keeps a `Deque<Long>` of raw request timestamps per key: on each call it first evicts everything from the front of the deque older than `windowMillis` (`log.peekFirst() < now - windowMillis`, i.e., timestamps that have “slid” outside the current window), then checks whether what remains is still under `maxRequests` — if so, it records this request’s timestamp and allows it; otherwise it rejects. `RateLimiter` itself is the facade the answer describes: its constructor takes a `RateLimitingAlgorithm`, and `allowRequest()` does nothing but delegate to whichever one it holds — there is no `if (algorithmType == “token_bucket”)` anywhere in `RateLimiter`. Swapping the whole rate-limiting strategy for a given limiter instance is a one-line change at construction time (`new RateLimiter(new SlidingWindowAlgorithm(...))` instead of `new RateLimiter(new TokenBucketAlgorithm(...))`), with zero changes to `RateLimiter` itself — precisely the same Strategy shape as `PricingStrategy` (Q193), `SplitStrategy` (Q209), and `SpotAssignmentStrategy` (Q192) elsewhere in this guide.

Case Study 8: Design a Vending Machine

Q218. What is the primary reason this is a classic LLD interview question, more than any real-world complexity? It’s a small, self-contained state machine with a genuinely rich set of states and transitions (idle, coin inserted, selection made, dispensing, out of stock, making change, returning coins) — making it an efficient way

for an interviewer to test whether you can model a state machine cleanly (via the State pattern, Q182) in a short amount of interview time, without needing a large, sprawling domain like the other case studies here.

Q219. Model the core states and describe why using the State pattern beats a single class with many boolean flags.

States: `IdleState`, `HasMoneyState`, `SelectionState`, `DispenseState`, `OutOfStockState`. Each state class implements the same interface (`insertCoin()`, `selectProduct()`, `dispense()`, `refund()`) but only some transitions are valid from each state — e.g., calling `dispense()` while `IdleState` should either be a no-op or an explicit error, not silently succeed. Modeling this with boolean flags (`hasMoney`, `hasSelection`, `isDispensing`) on a single class quickly produces combinatorially many flag combinations to validate in every method (and invalid combinations, like `hasMoney=false`, `isDispensing=true`, become representable even though they should never occur) — the State pattern makes invalid states/transitions structurally harder to represent at all, rather than needing to be defensively checked everywhere.

```
interface VendingMachineState {
    void insertCoin(VendingMachine machine, Coin coin);
    void selectProduct(VendingMachine machine, String code);
    void dispense(VendingMachine machine);
    void refund(VendingMachine machine);
}

class IdleState implements VendingMachineState {
    public void insertCoin(VendingMachine machine, Coin coin) {
        machine.addBalance(coin.value());
        machine.setState(new HasMoneyState());
    }
    public void selectProduct(VendingMachine machine, String code) {
        throw new IllegalStateException("Insert coin first");
    }
    public void dispense(VendingMachine machine) {
        throw new IllegalStateException("No selection made");
    }
    public void refund(VendingMachine machine) {
        throw new IllegalStateException("No money to refund");
    }
}

class HasMoneyState implements VendingMachineState {
    public void insertCoin(VendingMachine machine, Coin coin) {
        machine.addBalance(coin.value()); // stays in HasMoneyState – just adds more
        balance
    }
    public void selectProduct(VendingMachine machine, String code) {
        Product product = machine.getInventory().lookup(code);
        if (product == null || machine.getInventory().quantityOf(code) == 0) {
            throw new IllegalStateException("Out of stock");
        }
        if (machine.getBalance() < product.price) {
            throw new IllegalStateException("Insufficient balance");
        }
        machine.setSelectedProduct(product);
        machine.setState(new DispenseState());
    }
}
```

```

    public void dispense(VendingMachine machine) {
        throw new IllegalStateException("No selection made yet");
    }
    public void refund(VendingMachine machine) {
        machine.refundBalance();
        machine.setState(new IdleState());
    }
}

class DispenseState implements VendingMachineState {
    public void insertCoin(VendingMachine machine, Coin coin) {
        throw new IllegalStateException("Dispensing in progress");
    }
    public void selectProduct(VendingMachine machine, String code) {
        throw new IllegalStateException("Dispensing in progress");
    }
    public void dispense(VendingMachine machine) {
        Product product = machine.getSelectedProduct();
        machine.getInventory().decrement(product.code);
        double change = machine.getBalance() - product.price;
        machine.dispenseChange(change);
        machine.reset();
        machine.setState(new IdleState());
    }
    public void refund(VendingMachine machine) {
        throw new IllegalStateException("Dispensing already started – cannot cancel");
    }
}

```

Walking through the code. Every state class implements the same four-method `VendingMachineState` interface, but each one only makes real transitions valid for that state — `IdleState.selectProduct()` doesn't check a `hasMoney` flag and branch; it simply *throws*, because there is no code path in `IdleState` that leads anywhere else.

`IdleState.insertCoin()` is the only method across the whole `IdleState` class that actually does something productive: it adds to the balance and transitions to `HasMoneyState` via `machine.setState(...)`. `HasMoneyState.selectProduct()` does real validation (stock check, balance check) before transitioning to `DispenseState` — and note it's only reachable from `HasMoneyState`, never from `IdleState` directly, since `IdleState.selectProduct()` always throws first. `DispenseState.dispense()` is where the actual product leaves the machine: it decrements inventory, computes and dispenses change, resets the machine's transient selection state, and transitions back to `IdleState` — and note `DispenseState.refund()` explicitly throws rather than silently allowing a cancel, since once dispensing has begun, backing out mid-transaction isn't a valid operation. Compare this to a boolean-flag version: a single `VendingMachine` class checking `if (!hasMoney) throw ...; if (hasSelection) throw ...` inside every single method would need to enumerate and defend against every combination of flags at every call site — here, `IdleState` simply *has no method body* that knows how to dispense a product, because that logic was never written into this class at all, making the invalid transition unrepresentable rather than merely rejected at runtime.

Q220. How do you handle inventory (limited stock per product slot) and making change, and where do those responsibilities live? Inventory is a separate class (SRP again) mapping product codes to `(Product, quantity)`, queried by the

`SelectionState` when validating a selection (reject with an “out of stock” transition if quantity is zero) and decremented atomically on successful dispense. A

`ChangeDispenser` class encapsulates coin denominations and the greedy (or dynamic-programming, if denominations don’t guarantee greedy optimality) algorithm for making exact change — kept separate from the state machine itself, since “how to make change from available denominations” is a distinct algorithmic concern that a state class calls out to, not something that belongs baked into transition logic.

```
class Product {
    String code;
    String name;
    double price;
}

class Inventory {
    private final Map<String, Product> products = new HashMap<>();
    private final Map<String, Integer> quantities = new HashMap<>();

    Product lookup(String code) { return products.get(code); }
    int quantityOf(String code) { return quantities.getOrDefault(code, 0); }

    synchronized void decrement(String code) { // atomic – two dispenses
        can't both take the last unit
        int current = quantities.getOrDefault(code, 0);
        if (current == 0) throw new IllegalStateException("Cannot decrement – already out of stock");
        quantities.put(code, current - 1);
    }
}

class ChangeDispenser {
    private static final int[] DENOMINATIONS = {100, 25, 10, 5, 1}; // cents – sorted descending

    List<Integer> makeChange(int amountCents) {
        List<Integer> coinsUsed = new ArrayList<>();
        int remaining = amountCents;
        for (int denom : DENOMINATIONS) { // greedy – correct
            because these denominations are "canonical"
            while (remaining >= denom) {
                coinsUsed.add(denom);
                remaining -= denom;
            }
        }
        if (remaining != 0) {
            throw new IllegalStateException("Cannot make exact change for " + amountCents + " cents");
        }
        return coinsUsed;
    }
}
```

Walking through the code. `Inventory` is its own class, separate from `VendingMachine` and separate from any state class — `HasMoneyState.selectProduct()` in Q219 calls `machine.getInventory().lookup(code)` and `.quantityOf(code)`, treating inventory concerns as something it consults rather than something it implements. `decrement()` is `synchronized`, which matters for the same reason atomic spot assignment mattered in

the parking lot case study (Q194): if two dispense operations for the last remaining unit of a product happened concurrently without this, both could read `current == 1`, both compute `current - 1 == 0`, and both believe they successfully decremented — physically dispensing a product the machine no longer had.

`ChangeDispenser.makeChange()` implements the greedy algorithm the answer describes: iterate denominations from largest to smallest, taking as many of the current denomination as fit before moving to the next. The code comment flags the caveat explicitly — greedy is only *correct* (not just fast) because U.S.-style denominations (100, 25, 10, 5, 1) happen to be “canonical,” meaning greedy always finds the true minimum coin count; with an arbitrary denomination set (say, coins of 1, 3, and 4, needing to make 6), greedy could pick sub-optimally and a dynamic-programming approach would be needed instead — worth mentioning explicitly if an interviewer probes on this, since assuming greedy always works is a subtle but real correctness gap.

Case Study 9: Design a Chess Game

Q221. Why is a naive “hardcode movement rules with if/else per piece type inside the `Board` class” a poor design, and what’s the better structure? It

violates SRP and OCP simultaneously: `Board` would need to know the movement rules of every piece type, and adding a new piece variant (or a chess variant with different rules) would require modifying the already-complex `Board` class. Better: each piece type (`King`, `Queen`, `Rook`, `Bishop`, `Knight`, `Pawn`) is its own class implementing a common `Piece` interface with a `canMove(Board board, Position from, Position to)` method — movement legality logic lives with the piece it concerns, `Board` only needs to ask a piece “can you make this move given the current board state,” and adding a new piece type never touches `Board` at all.

```
enum Color { WHITE, BLACK }

class Position {
    int row, col;
    Position(int row, int col) { this.row = row; this.col = col; }
}

abstract class Piece {
    Color color;
    abstract boolean canMove(Board board, Position from, Position to);
}

class Bishop extends Piece {
    boolean canMove(Board board, Position from, Position to) {
        if (Math.abs(from.row - to.row) != Math.abs(from.col - to.col)) return false; //
        must be a pure diagonal
        return board.isPathClear(from, to) && !board.isSameColorAt(to, this.color);
    }
}

class Rook extends Piece {
    boolean canMove(Board board, Position from, Position to) {
        if (from.row != to.row && from.col != to.col) return false; // must be a straight
        line
        return board.isPathClear(from, to) && !board.isSameColorAt(to, this.color);
    }
}
```

```

    }
}

class Knight extends Piece {
    boolean canMove(Board board, Position from, Position to) {
        int dRow = Math.abs(from.row - to.row);
        int dCol = Math.abs(from.col - to.col);
        boolean isLShape = (dRow == 2 && dCol == 1) || (dRow == 1 && dCol ==
2); // knights jump – no path-clear check needed
        return isLShape && !board.isSameColorAt(to, this.color);
    }
}

class Board {
    Piece[][] squares = new Piece[8][8];

    boolean isPathClear(Position from, Position to) { /* walks squares between from and to,
checks for occupancy */ return true; }
    boolean isSameColorAt(Position pos, Color color) {
        Piece p = squares[pos.row][pos.col];
        return p != null && p.color == color;
    }
    // Board contains ZERO piece-specific movement logic – it only offers these two generic
helpers
}

```

Walking through the code. `Piece.canMove()` is abstract, so each concrete piece (Bishop, Rook, Knight shown here) provides its own geometry check with no involvement from `Board` beyond two small generic helper methods it exposes: `isPathClear()` (are the squares between `from` and `to` empty — relevant for sliding pieces like bishops and rooks) and `isSameColorAt()` (would this move capture your own piece — illegal in chess). `Bishop.canMove()` checks that the move is a perfect diagonal ($\text{Math.abs}(\text{from.row} - \text{to.row}) == \text{Math.abs}(\text{from.col} - \text{to.col})$) — the number of rows moved must equal the number of columns moved — then defers to `Board`'s generic path-clear and same-color checks. `Knight.canMove()` doesn't call `isPathClear()` at all, because knights jump over other pieces — that's a piece-specific fact correctly encoded only in `Knight`'s own class, not as a special case `Board` has to know about.

`Board.squares` is just an 8×8 grid of `Piece` references — `Board` itself contains no `if` (piece instanceof Bishop) ... else if (piece instanceof Rook) logic anywhere; it asks whichever piece occupies a square to judge its own move's legality. Adding a chess variant with a custom piece (say, an "archbishop" combining bishop and knight moves) means writing one new `Piece` subclass — `Board` never changes.

Q222. How do you detect check, checkmate, and stalemate, and why is this logic kept separate from each piece's own `canMove` method? "Check" detection asks: after a hypothetical move, is the current player's king under attack by any opposing piece (i.e., does any opposing piece's `canMove` return true targeting the king's square)? This must be checked at the `Game` / `Board` level — not inside any single piece's `canMove` — because it requires evaluating all opposing pieces against the resulting board state, which is a board-wide concern, not something any individual piece can determine about itself. "Checkmate" is: the player is in check, and no legal move exists (across all their pieces) that would result in a board state where they're no longer in

check. “Stalemate” is the same “no legal move exists” condition but while not currently in check — this shared “does any legal move exist” computation is a natural candidate for its own method, called with different pre-conditions to distinguish checkmate from stalemate.

```
class Game {
    Board board;
    Color currentTurn = Color.WHITE;

    boolean isInCheck(Color kingColor) {
        Position kingPos = board.findKing(kingColor);
        for (Position p : board.allOccupiedPositions()) {
            Piece piece = board.pieceAt(p);
            if (piece.color != kingColor && piece.canMove(board, p, kingPos)) {
                return true; // some opposing piece can legally reach
the king's square
            }
        }
        return false;
    }

    boolean hasAnyLegalMove(Color color) {
        for (Position from : board.allOccupiedPositions()) {
            Piece piece = board.pieceAt(from);
            if (piece.color != color) continue;
            for (Position to : board.allSquares()) {
                if (!piece.canMove(board, from, to)) continue;
                Board hypothetical = board.copy(); // try the move on a COPY,
never the real board
                hypothetical.applyMove(from, to);
                if (!wouldBeInCheck(hypothetical, color)) {
                    return
true; // found at least one move that escapes/avoids check
                }
            }
        }
        return false;
    }

    boolean isCheckmate(Color color) { return isInCheck(color) && !hasAnyLegalMove(color); }
    boolean isStalemate(Color color) { return !isInCheck(color) && !hasAnyLegalMove(color); }

    private boolean wouldBeInCheck(Board hypothetical, Color color) { /* same logic as
isInCheck, on hypothetical */ return false; }
}
```

Walking through the code. `isInCheck()` loops over `board.allOccupiedPositions()` — every square with a piece on it — and, for each opposing piece, asks `piece.canMove(board, p, kingPos)`: can this piece legally move to the exact square the king occupies? If any single opposing piece answers yes, the king is in check. This has to live in `Game`, not in any individual `Piece`, precisely because it requires iterating over every opposing piece — a `Bishop.canMove()` call only ever knows about itself and the two positions it’s asked about, with no visibility into what every other piece on the board could also do. `hasAnyLegalMove()` is the more expensive shared computation: for every piece the given `color` owns, for every square it could theoretically move to (`piece.canMove` passes), it clones the board (`board.copy()`), applies the hypothetical

move on that *copy* only, and checks whether the player would still be in check afterward — critically, this never mutates the real, current board, since testing a move must not have side effects on the actual game state. `isCheckmate()` and `isStalemate()` are then both thin wrappers around the exact same `hasAnyLegalMove()` call, differing only in whether `isInCheck()` is also required to be true — exactly the “same underlying computation, different pre-conditions” relationship the answer describes.

Q223. How would you implement move history and support for “undo” and eventually a chess clock/timed games? Move history as a list of `Move` objects (from-position, to-position, piece moved, captured piece if any, and special-move flags like castling/en passant) — this is essentially the Command pattern (Q181): each `Move` object encapsulates enough information to be reversed (`undo()` restores the captured piece and moves the piece back), rather than the `Board` needing separate, bespoke undo logic disconnected from how moves were originally represented. For a chess clock, a `Timer` per player, started/stopped on each turn transition, is a cleanly separable concern (Observer pattern — the `Game`’s turn-change events trigger the appropriate timer to start/stop) rather than logic embedded inside move validation.

```
class Move {
    Position from, to;
    Piece pieceMoved;
    Piece capturedPiece;           // null if this move didn't capture anything
    boolean wasCastling;
    boolean wasEnPassant;

    void execute(Board board) {
        capturedPiece = board.pieceAt(to);           // remember what was captured, BEFORE
        overwriting it
        board.squares[to.row][to.col] = pieceMoved;
        board.squares[from.row][from.col] = null;
    }

    void undo(Board board) {                         // exact mirror of execute() – Command
        pattern's undo()
        board.squares[from.row][from.col] = pieceMoved;
        board.squares[to.row][to.col] = capturedPiece; // restores whatever was there
        before – possibly null
    }
}

class Game {
    Board board;
    Deque<Move> moveHistory = new ArrayDeque<>();      // stack – most recent move on top

    void makeMove(Position from, Position to) {
        Piece piece = board.pieceAt(from);
        Move move = new Move();
        move.from = from; move.to = to; move.pieceMoved = piece;
        move.execute(board);
        moveHistory.push(move);
    }

    void undoLastMove() {
        if (moveHistory.isEmpty()) return;
        moveHistory.pop().undo(board);               // generic – Game doesn't know WHAT
        it's undoing
    }
}
```

```

    }
}

interface TurnObserver {
    void onTurnStarted(Color player);
    void onTurnEnded(Color player);
}

class ChessClock implements TurnObserver {
    Map<Color, Duration> remainingTime;
    public void onTurnStarted(Color player) { /* start counting down
remainingTime.get(player) */ }
    public void onTurnEnded(Color player) { /* pause that player's countdown */ }
}

```

Walking through the code. `Move.execute()` does two things worth noting: it captures `board.pieceAt(to)` into `capturedPiece` *before* overwriting that square — if it wrote the moving piece into `to` first, whatever was there would be lost and `undo()` would have no way to restore it. `undo()` is then a near-exact mirror: put `pieceMoved` back at `from`, and restore whatever `capturedPiece` was (which is simply `null` if nothing was captured, correctly leaving `to` empty again) at `to`. `Game.undoLastMove()` calls `moveHistory.pop().undo(board)` — it has no idea whether the move being undone was a simple pawn push or a capture; the `Move` object itself carries everything needed to reverse itself, exactly the Command pattern shape from Q181 where `CommandHistory.undoLast()` called `history.pop().undo()` generically. For the chess clock, `ChessClock` implements `TurnObserver` and reacts to `onTurnStarted`/`onTurnEnded` callbacks — `Game` would call these at the same point it currently flips `currentTurn`, without `Game`’s core move-validation logic needing to know timers exist at all; a game played without a clock simply has no `TurnObserver` subscribed, and every other part of `Game` behaves identically either way.

Q224. How would you design the system to support multiple game variants (Chess960/Fischer Random, King of the Hill) without duplicating the entire engine? Isolate variant-specific rules behind interfaces the core engine depends on rather than hardcodes: a `BoardSetupStrategy` (Chess960 randomizes back-rank piece placement at game start), and a `WinConditionStrategy` (standard checkmate vs. King of the Hill’s “reach the center squares” alternate win condition) — both Strategy-pattern injection points into an otherwise-shared `Game` engine, so a new variant is new strategy implementations plugged into the existing engine, not a forked/duplicated copy of the whole game logic.

```

interface BoardSetupStrategy {
    void setupInitialPosition(Board board);
}

class StandardChessSetup implements BoardSetupStrategy {
    public void setupInitialPosition(Board board) { /* standard back-rank order: R N B Q K B
N R */ }
}

class Chess960Setup implements BoardSetupStrategy {
    public void setupInitialPosition(Board board) {

```

```

        List<Character> backRank = randomizedBackRankRespectingConstraints(); // bishops on
        opposite colors, king between rooks
        /* place backRank pieces on row 0 and row 7 */
    }
    private List<Character> randomizedBackRankRespectingConstraints() { return List.of(); }
}

interface WinConditionStrategy {
    boolean hasWon(Board board, Color player);
}

class CheckmateWinCondition implements WinConditionStrategy {
    public boolean hasWon(Board board, Color player) { /* opponent is in checkmate – reuses
    Game.isCheckmate() logic */ return false; }
}

class KingOfTheHillWinCondition implements WinConditionStrategy {
    private static final Set<Position> CENTER_SQUARES = Set.of(
        new Position(3, 3), new Position(3, 4), new Position(4, 3), new Position(4, 4)
    );
    public boolean hasWon(Board board, Color player) {
        Position kingPos = board.findKing(player);
        return CENTER_SQUARES.contains(kingPos); // reaching the center wins,
        independent of check/checkmate at all
    }
}

class Game {
    Board board;
    WinConditionStrategy winCondition;
    BoardSetupStrategy setupStrategy;

    Game(BoardSetupStrategy setupStrategy, WinConditionStrategy winCondition) {
        this.setupStrategy = setupStrategy;
        this.winCondition = winCondition;

        setupStrategy.setupInitialPosition(board); // variant-specific, injected – Game doesn't
        know which one
    }

    void afterEachMove(Color mover) {
        if (winCondition.hasWon(board, mover)) { /* declare mover the winner */ }
    }
}

new Game(new StandardChessSetup(), new CheckmateWinCondition()); // regular chess
new Game(new Chess960Setup(), new KingOfTheHillWinCondition()); // a hybrid variant –
same Game class

```

Walking through the code. `Game`’s constructor takes both a `BoardSetupStrategy` and a `WinConditionStrategy` as parameters and calls

`setupStrategy.setupInitialPosition(board)` once, at construction — it never contains the standard “R N B Q K B N R” back-rank layout itself, nor does it know how Chess960’s randomization works; that logic lives entirely inside `Chess960Setup`.

`afterEachMove()` similarly just calls `winCondition.hasWon(board, mover)` after every move — for `CheckmateWinCondition` this delegates to the same checkmate-detection logic from Q222, while `KingOfTheHillWinCondition` implements a completely unrelated check (has this player’s king physically reached one of the four center squares), with neither

implementation needing to know the other exists. The two `new Game(...)` calls at the bottom show the payoff directly: standard chess and a King-of-the-Hill-on-a-Chess960-board hybrid are both just different strategy combinations passed into the *same* `Game` class — there is no `StandardChessGame` and `Chess960Game` as separate, partially-duplicated classes, which is exactly what “without duplicating the entire engine” requires.

Case Study 10: Design the Core Order-Assignment Flow for a Food Delivery App (LLD layer, complementing the HLD ride-sharing/matching ideas)

Q225. What are the core entities specific to the LLD of order-to-delivery-partner assignment, distinct from the restaurant/menu/cart part of the system?

`Order` (has status: placed/accepted/preparing/ready/picked-up/delivered), `DeliveryPartner` (has current location, availability status, active order capacity), `AssignmentService` (the component actually responsible for matching a ready order to an available partner), and — importantly — an `OrderStateMachine` since an order’s status transitions have real business rules attached (an order can’t go from “placed” directly to “delivered” without passing through the intermediate states, and certain transitions trigger side effects like notifying the customer).

Q226. How would you design the `AssignmentService` so the actual matching algorithm (nearest partner, batching multiple orders to one partner, etc.) can evolve independently of the surrounding order-management code? Same recurring shape as this entire section: `AssignmentService` depends on an injected `PartnerMatchingStrategy` interface, with concrete implementations like `NearestAvailablePartnerStrategy` or a more advanced `BatchedDeliveryStrategy` (assigning multiple nearby orders to one partner’s route for efficiency) — the order-management code that calls “find and assign a partner for this ready order” never needs to change as the matching algorithm itself is iterated on, tuned, or entirely replaced by a more sophisticated approach (e.g., swapping in an ML-based ETA/matching model later).

```
class DeliveryPartner {
    String partnerId;
    Location currentLocation;
    boolean available;
    int activeOrderCount;
    int maxConcurrentOrders = 1;
}

class Location {
    double lat, lng;
    double distanceTo(Location other) { /* haversine or simple euclidean approximation */ return 0.0; }
}

interface PartnerMatchingStrategy {
    Optional<DeliveryPartner> findPartner(Order order, List<DeliveryPartner> candidates);
}
```

```

}

class NearestAvailablePartnerStrategy implements PartnerMatchingStrategy {
    public Optional<DeliveryPartner> findPartner(Order order, List<DeliveryPartner>
candidates) {
        return candidates.stream()
            .filter(p -> p.available && p.activeOrderCount < p.maxConcurrentOrders)
            .min(Comparator.comparingDouble(p -> p.currentLocation.distanceTo(order.restaurantLocation)));
    }
}

class AssignmentService {
    private final PartnerMatchingStrategy strategy;           // injected – the ONLY point
matching logic enters

    AssignmentService(PartnerMatchingStrategy strategy) { this.strategy = strategy; }

    boolean assignOrder(Order order, List<DeliveryPartner> candidates) {
        Optional<DeliveryPartner> match = strategy.findPartner(order,
candidates); // never inlines matching logic itself
        if (match.isEmpty()) return false;
        DeliveryPartner partner = match.get();
        partner.activeOrderCount++;
        order.assignedPartner = partner;
        return true;
    }
}

AssignmentService service = new AssignmentService(new NearestAvailablePartnerStrategy());
// later, swapping in a smarter strategy requires touching NOTHING but this one construction
line:
// AssignmentService smarter = new AssignmentService(new BatchedDeliveryStrategy());

```

Walking through the code. `NearestAvailablePartnerStrategy.findPartner()` first filters `candidates` down to partners who are both `available` and under their `maxConcurrentOrders` capacity, then picks the minimum by distance to the restaurant using `Comparator.comparingDouble` — this is the entire “nearest available partner” algorithm, contained in one small class. `AssignmentService.assignOrder()` calls `strategy.findPartner(order, candidates)` and does nothing else algorithm-related — its own responsibility is limited to acting on whatever match comes back (incrementing the partner’s load, recording the assignment on the order). A more sophisticated `BatchedDeliveryStrategy`, which might look at several `ready` orders at once and decide to route two nearby orders to the same partner’s trip for efficiency, would implement the exact same `findPartner()` signature and could be substituted in with the one-line change shown in the comment at the bottom — `AssignmentService`’s own code, and anything that calls `assignmentService.assignOrder(...)`, is completely unaffected by that substitution.

Q227. How do you model the order state machine so that invalid transitions (like re-marking an already-delivered order as “preparing”) are impossible to represent, not just defensively checked? Define an explicit, finite `OrderStatus` enum and an allowed-transitions map (`Map<OrderStatus, Set<OrderStatus>>`) that every status-changing method consults before applying a transition, throwing a clear domain

exception on an invalid one — or, more robustly, apply the State pattern (Q182, Q199, Q219 — same idea, fourth appearance in this guide) so each status is its own class and simply doesn't expose a method for a transition that shouldn't be reachable from it at all, rather than exposing every transition method universally and relying on a runtime check to reject the illegal ones.

```
enum OrderStatus { PLACED, ACCEPTED, PREPARING, READY, PICKED_UP, DELIVERED, CANCELLED }

class InvalidTransitionException extends RuntimeException {
    InvalidTransitionException(String message) { super(message); }
}

class OrderStateMachine {
    private static final Map<OrderStatus, Set<OrderStatus>> ALLOWED_TRANSITIONS = Map.of(
        OrderStatus.PLACED,    Set.of(OrderStatus.ACCEPTED, OrderStatus.CANCELLED),
        OrderStatus.ACCEPTED, Set.of(OrderStatus.PREPARING, OrderStatus.CANCELLED),
        OrderStatus.PREPARING,  Set.of(OrderStatus.READY, OrderStatus.CANCELLED),
        OrderStatus.READY,      Set.of(OrderStatus.PICKED_UP),
        OrderStatus.PICKED_UP,  Set.of(OrderStatus.DELIVERED),
        OrderStatus.DELIVERED, Set.of(), // terminal - no
        outgoing transitions at all
        OrderStatus.CANCELLED, Set.of() // terminal
    );

    void transition(Order order, OrderStatus newStatus) {
        Set<OrderStatus> allowed = ALLOWED_TRANSITIONS.get(order.status);
        if (!allowed.contains(newStatus)) {
            throw new InvalidTransitionException(
                "Cannot go from " + order.status + " to " + newStatus);
        }
        order.status = newStatus;
        onTransitioned(order, newStatus); // side effects, e.g.
        notify customer
    }

    private void onTransitioned(Order order, OrderStatus newStatus) {
        if (newStatus == OrderStatus.PICKED_UP) { /* notify customer: "your order is on the
way" */ }
        if (newStatus == OrderStatus.DELIVERED) { /* notify customer: "delivered", close out
the order */ }
    }
}
```

Walking through the code. `ALLOWED_TRANSITIONS` is a single, explicit map from each status to the exact set of statuses reachable from it — `DELIVERED` and `CANCELLED` both map to `Set.of()`, an empty set, meaning no transition is ever considered valid from either of those states, which is exactly what “terminal” means here. `transition()` looks up `order.status`’s allowed set and checks `allowed.contains(newStatus)` *before* mutating anything — attempting to move a `DELIVERED` order to `PREPARING` looks up `ALLOWED_TRANSITIONS.get(DELIVERED)`, gets back the empty set, finds `PREPARING` isn’t in it, and throws *before* `order.status` is ever touched. This is the “defensively checked” version the answer describes as the simpler (though not the most robust) option — the map-and-check approach centralizes the rule in one place rather than scattering `if` checks, but it’s still a runtime check that *could* theoretically be bypassed by code that sets `order.status` directly instead of going through `transition()`. The fully robust

version (mentioned in the answer, following the exact same shape as `ElevatorContext / ElevatorStateHandler` in Q199) would make each status its own class implementing only the methods that represent its actually-valid outgoing transitions — a `DeliveredOrderState` class simply wouldn't have a `markAsPreparing()` method to call at all, making the illegal transition not just rejected but structurally absent from the API surface.

Q228. What's the closing point worth making to an interviewer about why the same handful of patterns (Strategy, State, SRP, OCP, DIP) recur across every one of these LLD case studies? This isn't a coincidence or a limitation of the example problems chosen — it reflects a real and important truth: the vast majority of good object-oriented design in practice comes down to a genuinely small set of recurring ideas (isolate what varies behind an interface, keep each class's reason to change singular, depend on abstractions not concretions) applied consistently, rather than needing dozens of exotic patterns. An interviewer is far more impressed by a candidate who fluently and correctly applies this small toolkit to a novel problem they've never seen before than by one who has memorized bespoke solutions to twenty specific named problems but struggles when handed a twenty-first — which is exactly why this guide has deliberately shown the same underlying moves surfacing again and again, rather than presenting each case study as an unrelated, disconnected puzzle.

A5. More LLD Coding-Round Patterns

These are additional standalone machine-coding problems that show up frequently in product-based-company LLD rounds, beyond the case studies in A4. Each follows the same format: entities, key design decisions, code, and a line-by-line walkthrough.

Pattern 1: Design a Thread-Safe Bounded Blocking Queue

Q229. What is a bounded blocking queue, and why can't you just use a plain `ArrayList` with synchronized methods to build one? A bounded blocking queue has a fixed maximum capacity; a producer calling `put()` when the queue is full *blocks* (waits) until space frees up, and a consumer calling `take()` when the queue is empty blocks until an item arrives — rather than either operation throwing an exception or busy-spinning. A plain `synchronized ArrayList` can be made thread-safe for individual add/remove calls, but it has no built-in mechanism for a thread to efficiently *wait* for a condition (space available, item available) — you'd either need to busy-loop (wasting CPU, checking repeatedly) or hand-roll the wait/notify coordination, which `synchronized` alone doesn't give you; that coordination is exactly what `wait()` / `notify()` (or higher-level `Condition` objects) exist for.

```
class BoundedBlockingQueue<T> {  
    private final Queue<T> queue = new LinkedList<>();  
    private final int capacity;  
    private final Object lock = new Object();  
}
```

```

BoundedBlockingQueue(int capacity) { this.capacity = capacity; }

void put(T item) throws InterruptedException {
    synchronized (lock) {
        while (queue.size() == capacity)
            // WHILE, not IF – see the walkthrough on spurious wakeups
            lock.wait(); // releases the lock and sleeps
        until notified
    }
    queue.add(item);
    lock.notifyAll(); // wake up any thread waiting in
    take()
}

T take() throws InterruptedException {
    synchronized (lock) {
        while (queue.isEmpty()) {
            lock.wait();
        }
        T item = queue.poll();
        lock.notifyAll(); // wake up any thread waiting in
    }
    put()
    return item;
}
}

```

Walking through the code. `put()` and `take()` both synchronize on the same `lock` object, so only one of them can be actively executing at a time — but the interesting part is `lock.wait()`. When a producer calls `put()` on a full queue, `queue.size() == capacity` is true, so it calls `lock.wait()`, which does two things atomically: it releases the lock (so other threads, including consumers calling `take()`, can proceed) and puts the calling thread to sleep until some other thread calls `notify()` / `notifyAll()` on the same lock. When a consumer later calls `take()` and successfully removes an item, `notifyAll()` wakes up *every* thread currently waiting on `lock` — including any producers blocked in `put()`. The `while` loop (rather than a single `if`) around each `wait()` call is deliberate and important: when a waiting thread wakes up, it re-acquires the lock and re-checks the condition from scratch, because between being notified and actually resuming execution, another thread could have already grabbed the freed-up space (or, in rarer cases, the JVM can wake a thread up “spuriously,” with no `notify()` call at all) — the `while` loop protects against both, re-blocking immediately if the condition it was waiting for is no longer actually true.

Pattern 2: Design Snake and Ladder (Game State + Randomness)

Q230. What are the core entities, and how do you cleanly represent the fact that landing on certain cells triggers a jump? `Board` (a linear sequence of cells, 1 to N), `Player` (current position), `Dice` (an interface, so a real random die and a fixed/mock die for testing are interchangeable), and critically — snakes and ladders are both

just a `Map<Integer, Integer>` from a “trigger” cell to a “destination” cell; a snake’s trigger cell maps to a *smaller* number, a ladder’s trigger cell maps to a *larger* number, but the `Board`’s movement logic doesn’t need to know or care which kind it is — it just looks up whether the cell a player landed on has an entry in that map, and if so, moves them again.

```
interface Dice {
    int roll();
}

class RandomDice implements Dice {
    private final Random random = new Random();
    public int roll() { return random.nextInt(6) + 1; }
}

class FixedSequenceDice implements Dice { // for deterministic tests –
    same interface as the real die
    private final Queue<Integer> sequence;
    FixedSequenceDice(List<Integer> rolls) { this.sequence = new LinkedList<>(rolls); }
    public int roll() { return sequence.poll(); }
}

class Board {
    private final int size;
    private final Map<Integer, Integer> jumps = new
    HashMap<>(); // one map for BOTH snakes and ladders

    Board(int size) { this.size = size; }

    void addSnake(int head, int tail) {
        if (tail >= head) throw new IllegalArgumentException("Snake must move you DOWN the
board");
        jumps.put(head, tail);
    }

    void addLadder(int bottom, int top) {
        if (top <= bottom) throw new IllegalArgumentException("Ladder must move you UP the
board");
        jumps.put(bottom, top);
    }

    int resolveLanding(int position) {
        return jumps.getOrDefault(position,
position); // no snake/ladder here → stay where you landed
    }

    int getSize() { return size; }
}

class Player {
    String name;
    int position = 0;
}

class Game {
    Board board;
    List<Player> players;
    Dice dice;
    int currentPlayerIndex = 0;
}
```

```

Player playTurn() {
    Player current = players.get(currentPlayerIndex);
    int roll = dice.roll();
    int newPosition = current.position + roll;
    if (newPosition > board.getSize()) {
        // overshoot – stay put; some rule variants require an EXACT roll to finish
    } else {
        current.position =
board.resolveLanding(newPosition); // may trigger a snake or ladder jump
        if (current.position == board.getSize()) return current; // winner
    }
    currentPlayerIndex = (currentPlayerIndex + 1) % players.size();
    return null;
}
}

```

Walking through the code. `Board.jumps` is the entire mechanism for both snakes and ladders — `addSnake(head, tail)` and `addLadder(bottom, top)` both simply insert an entry into the *same* map, with each method's only job being to validate the direction makes sense for what it claims to be (a snake's `tail` must be a smaller number than its `head`, since a snake moves you backward; a ladder's `top` must be larger than its `bottom`). `resolveLanding()` — called after every dice roll — does one `getOrDefault()` lookup: if the landed-on cell happens to be a key in `jumps` (whether it was registered as a snake's head or a ladder's bottom makes no difference to this method), the player's position becomes the mapped value instead; otherwise they simply stay where they landed. This is why `Game.playTurn()` never needs an `if (isSnakeCell(newPosition))` branch — `board.resolveLanding(newPosition)` already returns the correct final position whether a jump happened or not. `Dice` being an interface, with both `RandomDice` and `FixedSequenceDice` implementing it, is what makes this whole game deterministically testable: a unit test can construct a `Game` with a `FixedSequenceDice` primed with a known roll sequence and assert on the exact resulting positions, without any of `Game`'s or `Board`'s code needing to know it isn't talking to a real die.

Pattern 3: Design an In-Memory File System

Q231. How do you model files and directories so that operations like computing total size, listing contents, and searching by name work uniformly across both? This is a direct application of the Composite pattern (Q178) — a `FileSystemEntry` interface (or abstract class) implemented by both `File` (a leaf, holding actual content/size) and `Directory` (a composite, holding a collection of child `FileSystemEntry`s, which may themselves be files or further directories) — any operation defined on the interface (`getSize()`, `find(name)`) is implemented recursively on `Directory` by delegating to its children, so calling it on the root works uniformly no matter how deep the tree is.

```

interface FileSystemEntry {
    String getName();
    long getSize();
    List<FileSystemEntry> find(String name); // returns all matches at or below this entry
}

```

```

class File implements FileSystemEntry {
    private final String name;
    private String content = "";

    File(String name) { this.name = name; }

    public String getName() { return name; }
    public long getSize() { return content.length(); }
    public List<FileSystemEntry> find(String name) {
        return this.name.equals(name) ? List.of(this) :
List.of(); // a leaf either matches itself, or matches nothing
    }
    void write(String content) { this.content = content; }
}

class Directory implements FileSystemEntry {
    private final String name;
    private final Map<String, FileSystemEntry> children = new
LinkedHashMap<>(); // ordered – stable listing order

    Directory(String name) { this.name = name; }

    public String getName() { return name; }

    public long getSize() {
        long total = 0;
        for (FileSystemEntry child : children.values()) {
            total +=
child.getSize(); // recurses – works whether child is a File or another Directory
        }
        return total;
    }

    public List<FileSystemEntry> find(String name) {
        List<FileSystemEntry> results = new ArrayList<>();
        if (this.name.equals(name)) results.add(this);
        for (FileSystemEntry child : children.values()) {
            results.addAll(child.find(name)); // recurses into every child, gathering
matches from the whole subtree
        }
        return results;
    }

    void add(FileSystemEntry entry) { children.put(entry.getName(), entry); }
    FileSystemEntry get(String name) { return children.get(name); }
    Collection<FileSystemEntry> list() { return children.values(); }
}

```

Walking through the code. `File.getSize()` is the base case of the recursion — it just returns `content.length()`, with no children to consider. `Directory.getSize()` is the recursive case: it loops over `children.values()` (typed as `FileSystemEntry`, never specifically `File` or `Directory`) and sums each child's `getSize()` — because a child that happens to be a `Directory` has its own `getSize()` implementation that does the exact same thing one level down, calling `root.getSize()` correctly totals an arbitrarily deep tree without the caller or any single method needing to know how deep it goes. `find()` follows the identical shape: `File.find()` is a one-line base case (does my own name match?), while `Directory.find()` checks itself *and* recursively collects matches

from every child, flattening the results of arbitrarily many nested subdirectories into one list via `results.addAll(child.find(name))`. `children` is a `LinkedHashMap` rather than a plain `HashMap` specifically so that `list()` returns entries in a stable, insertion-consistent order — relevant if a caller displays directory contents and expects `ls`-like consistent ordering across calls, rather than the unspecified iteration order a plain `HashMap` would give.

Pattern 4: Design a Pub-Sub / Event Bus (Observer, generalized)

Q232. How does a general-purpose, multi-topic event bus differ from the single-subject `Observer` example in Q180, and what concurrency issue does a naive implementation introduce? Q180's `StockTicker` had exactly one kind of event and one list of observers. A real event bus needs to support many independent topics, each with its own independent subscriber list, and — critically — needs to handle a subscriber unsubscribing *while an event is being published* (e.g., a subscriber's own callback triggers an unsubscribe) without throwing a `ConcurrentModificationException` from mutating a list while it's being iterated.

```
interface EventListener<T> {
    void onEvent(T event);
}

class EventBus {
    private final Map<String, List<EventListener<?>>> topicListeners = new ConcurrentHashMap<>();

    <T> void subscribe(String topic, EventListener<T> listener) {
        topicListeners.computeIfAbsent(topic, k -> new
CopyOnWriteArrayList<>()).add(listener);
    }

    <T> void unsubscribe(String topic, EventListener<T> listener) {
        List<EventListener<?>> listeners = topicListeners.get(topic);
        if (listeners != null) listeners.remove(listener);
    }

    @SuppressWarnings("unchecked")
    <T> void publish(String topic, T event) {
        List<EventListener<?>> listeners = topicListeners.get(topic);
        if (listeners == null) return;
        for (EventListener<?> listener : listeners) {           // safe even if a listener
unsubscribes mid-iteration
            ((EventListener<T>) listener).onEvent(event);
        }
    }
}

EventBus bus = new EventBus();
EventListener<String> logger = event -> System.out.println("Log: " + event);
bus.subscribe("order.created", logger);
bus.publish("order.created", "Order #123 placed");
bus.unsubscribe("order.created", logger);
```

Walking through the code. `topicListeners` maps each topic string to its own independent list of listeners — `subscribe()` uses `computeIfAbsent` to lazily create that list the first time a topic is subscribed to, and every topic's listener list is fully independent of every other topic's, which is the multi-topic generalization the answer describes over Q180's single-subject design. The concurrency detail is in the choice of `CopyOnWriteArrayList` for each topic's listener list: unlike a plain `ArrayList`, iterating over a `CopyOnWriteArrayList` with a `for` loop (as `publish()` does) is guaranteed never to throw a `ConcurrentModificationException` even if another thread — or the very listener callback currently executing inside the loop — calls `unsubscribe()` on that same list mid-iteration. That safety comes from `CopyOnWriteArrayList`'s internal behavior: any mutation (`add` / `remove`) creates an entirely new backing array rather than modifying the one an in-progress iterator is reading from, so the iteration sees a stable snapshot of the list as it existed when the loop started, regardless of concurrent changes. This trades away write efficiency (every mutation copies the whole array) for read/iteration safety — the right trade-off here, since publishing (iterating) happens far more often than subscribing/unsubscribing in a typical event bus.

Pattern 5: Design a Movie Ticket Booking Seat-Lock System

Q233. How do you prevent two users from simultaneously booking the same seat, while also handling users who select seats but abandon checkout without completing payment? This connects directly back to the HLD ticket-booking concurrency answer (Q142) and the parking-spot atomic-assignment answer (Q194) — the same underlying problem recurs at LLD scale: seat selection must be an atomic “lock this seat only if it's currently unlocked” operation, and critically, a lock must have an expiry (a temporary hold, not a permanent claim) so a user who closes their browser mid-checkout doesn't permanently block that seat for everyone else.

```
enum SeatStatus { AVAILABLE, LOCKED, BOOKED }

class Seat {
    String seatId;
    SeatStatus status = SeatStatus.AVAILABLE;
    String lockedByUserId;
    Instant lockExpiresAt;
}

class SeatLockService {
    private final Map<String, Seat> seats;           // seatId -> Seat
    private final Duration lockDuration = Duration.ofMinutes(5);

    SeatLockService(Map<String, Seat> seats) { this.seats = seats; }

    synchronized boolean tryLock(String seatId, String userId)
    {
        // synchronized - atomic check-then-set
        Seat seat = seats.get(seatId);
        releaseIfExpired(seat);                       // lazily reclaim
        expired locks first
        if (seat.status != SeatStatus.AVAILABLE) return false;
        seat.status = SeatStatus.LOCKED;
        seat.lockedByUserId = userId;
    }
}
```

```

        seat.lockExpiresAt = Instant.now().plus(lockDuration);
        return true;
    }

    synchronized boolean confirmBooking(String seatId, String userId) {
        Seat seat = seats.get(seatId);
        releaseIfExpired(seat);
        if (seat.status != SeatStatus.LOCKED || !seat.lockedByUserId.equals(userId)) {
            return false; // lock expired, or
// belongs to someone else now
        }
        seat.status = SeatStatus.BOOKED; // permanent – no
// expiry on a real booking
        return true;
    }

    private void releaseIfExpired(Seat seat) {
        if (seat.status == SeatStatus.LOCKED && Instant.now().isAfter(seat.lockExpiresAt)) {
            seat.status = SeatStatus.AVAILABLE; // abandoned
            seat.lockedByUserId = null;
        }
    }
}

```

Walking through the code. `tryLock()` and `confirmBooking()` are both `synchronized`, making each one an atomic check-then-set operation on the whole `SeatLockService` — necessary because `tryLock` needs to read `seat.status` and then write to it without any other thread's `tryLock` or `confirmBooking` call interleaving in between (the same compare-and-swap concern as Q194's `SafeParkingSpot.tryAssign()`).

`releaseIfExpired()` is called as the first step inside *both* locked methods, and it's what turns a `LOCKED` seat back into `AVAILABLE` once `Instant.now()` has passed `seat.lockExpiresAt` — this is deliberately lazy (checked on the next access to that seat, not via a background timer sweeping the whole seat map), which is a common simplification: no separate cleanup thread is needed, since any subsequent `tryLock` or `confirmBooking` call on an expired seat will notice and reclaim it itself.

`confirmBooking()` double-checks `seat.lockedByUserId.equals(userId)` — not just that the seat is `LOCKED`, but that it's locked by *this specific user* — which handles the case where User A's lock expired and User B has since locked the same seat before User A's delayed payment confirmation arrives; without this check, User A's stale confirmation could incorrectly book a seat User B now holds.

Pattern 6: Design Consistent Hashing (as a standalone data structure)

Q234. How would you implement the consistent hashing ring itself as a class — the data structure referenced conceptually in the HLD sharding discussion — including virtual nodes? A `TreeMap<Long, String>` (sorted map from hash value to node/server name) represents the ring — inserting a node means inserting it (and several virtual-node aliases of it, for load balancing — see Q141) at multiple hash positions around the ring, and looking up which node owns a given key means hashing

the key and finding the next node clockwise, which `TreeMap.ceilingKey()` gives directly in $O(\log n)$.

```
class ConsistentHashRing {
    private final TreeMap<Long, String> ring = new TreeMap<>(); // hash position ->
    physical node name
    private final int virtualNodesPerServer;

    ConsistentHashRing(int virtualNodesPerServer) { this.virtualNodesPerServer =
    virtualNodesPerServer; }

    void addServer(String serverName) {
        for (int i = 0; i < virtualNodesPerServer; i++) {
            long hash = hash(serverName + "#" + i); // each virtual node gets its
            OWN ring position
            ring.put(hash, serverName);
        }
    }

    void removeServer(String serverName) {
        for (int i = 0; i < virtualNodesPerServer; i++) {
            ring.remove(hash(serverName + "#" + i));
        }
    }

    String getServerForKey(String key) {
        if (ring.isEmpty()) throw new IllegalStateException("No servers in the ring");
        long hash = hash(key);
        Map.Entry<Long, String> entry = ring.ceilingEntry(hash); // next node CLOCKWISE
        from this key's position
        if (entry == null) entry = ring.firstEntry(); // wrapped past the
        highest hash - go to the start
        return entry.getValue();
    }

    private long hash(String input) {
        return input.hashCode() & 0xFFFFFFFFL; // simplified - a real implementation uses
        MD5/MurmurHash for better distribution
    }
}
```

Walking through the code. `ring` is a `TreeMap<Long, String>`, and a `TreeMap` keeps its keys sorted, which is what makes `ceilingEntry(hash)` — “the smallest key greater than or equal to this one” — an $O(\log n)$ binary-search operation rather than a linear scan; that single method call is the entire “find the next node clockwise on the ring” operation. `addServer()` doesn’t insert `serverName` at a single ring position — it loops `virtualNodesPerServer` times, hashing a *different* string each time (`serverName + "#" + i`) so the same physical server occupies many scattered positions around the ring; this is what prevents one server from ending up responsible for a disproportionately large, contiguous arc of the hash space purely due to where its single hash happened to land, spreading its share of keys more evenly instead. `getServerForKey()` hashes the incoming key and calls `ceilingEntry()` — the crucial edge case is the `if (entry == null)` branch: if the key’s hash is greater than every node’s position currently in the ring, there’s no “next key clockwise” in the normal sense, so the ring wraps around and the correct owner is whichever node sits at the very *lowest* hash position

(`ring.firstEntry()`), which is what makes this genuinely a ring rather than a simple sorted list with a hard end.

A6. Microservices Deep Dive

A1 covered the vocabulary (Q62–Q68). This section goes deeper on the patterns that actually show up as follow-up/whiteboard-plus-code questions once an interviewer confirms you know the definitions — API composition, Sagas with real orchestration code, circuit breakers, the outbox pattern, service mesh internals, and contract testing.

API Composition & Aggregation

Q235. When a client screen needs data that spans multiple services (e.g., an order detail page needing order, customer, and product data from three different services), what are the two main strategies, and when is each appropriate? **API composition** — a composer (often the API Gateway, or a dedicated Backend-for-Frontend) calls each service directly and joins the results in application code at request time. Simple to build and always returns fresh data, but adds latency (calls can be parallelized, but you're still bound by the slowest one) and couples the composer to every downstream service's availability. **CQRS-style materialized view** — a separate read-optimized store is kept continuously in sync (via events) with a pre-joined view of the data, so a read is a single fast query against one store. Composition is appropriate for low-traffic, non-latency-critical, ad-hoc queries; a materialized view is worth the added complexity once the same composed query is hit at high volume or has strict latency requirements.

```
class OrderDetailComposer {
    private final OrderServiceClient orderClient;
    private final CustomerServiceClient customerClient;
    private final ProductServiceClient productClient;

    OrderDetailComposer(OrderServiceClient o, CustomerServiceClient c, ProductServiceClient
p) {
        this.orderClient = o; this.customerClient = c; this.productClient = p;
    }

    OrderDetailView getOrderDetail(String orderId) {
        CompletableFuture<Order> orderFuture = orderClient.getOrderAsync(orderId);

        // customer and product lookups depend on the order's contents, so they start once
        // orderFuture resolves –
        // but the two of THEM run in parallel with each other via thenCompose + allOf
        CompletableFuture<OrderDetailView> result = orderFuture.thenCompose(order -> {
            CompletableFuture<Customer> customerFuture = customerClient.getCustomerAsync(orde
r.customerId);
            CompletableFuture<List<Product>> productsFuture =
productClient.getProductsAsync(order.productId);

            return CompletableFuture.allOf(customerFuture, productsFuture)
                .thenApply(v -> new OrderDetailView(order, customerFuture.join(),
```

```

productsFuture.join());
    });

    try {
        return result.get(2, TimeUnit.SECONDS); // bounded wait – never hang the
request indefinitely
    } catch (TimeoutException e) {
        throw new CompositionTimeoutException("Order detail composition exceeded 2s
budget");
    } catch (Exception e) {
        throw new CompositionFailedException("Failed to compose order detail", e);
    }
}
}
}

```

Walking through the code. `orderClient.getOrderAsync(orderId)` kicks off the first call and returns a `CompletableFuture<Order>` immediately rather than blocking the calling thread. `.thenCompose(order -> ...)` runs once that future resolves, and *inside* that callback, `customerClient.getCustomerAsync(...)` and `productClient.getProductsAsync(...)` are both started — since neither depends on the other’s result, they run concurrently rather than sequentially, which is the latency optimization the answer alludes to (you’re bound by the slower of the two, not their sum). `CompletableFuture.allOf(customerFuture, productsFuture)` produces a future that completes only once *both* inputs have completed, and `.thenApply(...)` then combines their results (via `.join()`, safe to call here since `allOf` already guarantees both are done) into the final `OrderDetailView`. The `result.get(2, TimeUnit.SECONDS)` at the call site is deliberately bounded — a composition that calls three downstream services must decide up front how long it’s willing to wait total, since without a bound, one slow or hung downstream service would otherwise stall this request indefinitely; the `TimeoutException` and generic failure paths are handled as distinct, explicit outcomes rather than left to propagate an unclear exception to the caller.

Q236. What is the Backend-for-Frontend (BFF) pattern, and how does it differ from a single shared API Gateway serving every client type? A BFF is a dedicated composition/aggregation layer built specifically for one client type (a mobile app, a web app, a partner API) rather than one generic gateway trying to serve all of them identically. The problem it solves: a mobile client typically wants a small, denormalized payload (battery/bandwidth constraints, simpler client-side logic) while a web dashboard for the same domain might want a richer, more granular response — a single shared gateway either has to expose a lowest-common-denominator API that satisfies neither well, or accumulate client-specific query parameters and conditional logic that make it increasingly hard to maintain. Each BFF is owned by (or at least tightly coupled to the needs of) the team building that specific client, and can evolve its response shape independently of every other client’s BFF, at the cost of some duplicated composition logic across BFFs that would otherwise live in one place.

Distributed Transactions — Sagas in Code

Q237. Implement the orchestration-style Saga for a simplified order-placement flow (reserve inventory → charge payment → confirm order), including the compensating actions if a later step fails.

```
interface SagaStep {
    void execute(SagaContext context) throws SagaStepException;
    void compensate(SagaContext context);    // undoes execute() – only called if a LATER
step fails
}

class SagaContext {
    String orderId;
    String reservationId;
    String paymentId;
    Map<String, Object> data = new HashMap<>();
}

class ReserveInventoryStep implements SagaStep {
    private final InventoryServiceClient inventoryClient;
    ReserveInventoryStep(InventoryServiceClient client) { this.inventoryClient = client; }

    public void execute(SagaContext context) throws SagaStepException {
        try {
            context.reservationId = inventoryClient.reserve(context.orderId);
        } catch (Exception e) {
            throw new SagaStepException("Inventory reservation failed", e);
        }
    }

    public void compensate(SagaContext context) {
        if (context.reservationId != null) {
            inventoryClient.releaseReservation(context.reservationId);    // undo: give the
stock back
        }
    }
}

class ChargePaymentStep implements SagaStep {
    private final PaymentServiceClient paymentClient;
    ChargePaymentStep(PaymentServiceClient client) { this.paymentClient = client; }

    public void execute(SagaContext context) throws SagaStepException {
        try {
            context.paymentId = paymentClient.charge(context.orderId);
        } catch (Exception e) {
            throw new SagaStepException("Payment charge failed", e);
        }
    }

    public void compensate(SagaContext context) {
        if (context.paymentId != null) {
            paymentClient.refund(context.paymentId);    // undo: refund the charge
        }
    }
}

class ConfirmOrderStep implements SagaStep {
    private final OrderServiceClient orderClient;
```

```

    ConfirmOrderStep(OrderServiceClient client) { this.orderClient = client; }

    public void execute(SagaContext context) throws SagaStepException {
        orderClient.markConfirmed(context.orderId);
    }

    public void compensate(SagaContext context) {
        orderClient.markFailed(context.orderId);           // undo: mark the order as
        // failed, not silently stuck
    }
}

class SagaOrchestrator {
    private final List<SagaStep> steps;

    SagaOrchestrator(List<SagaStep> steps) { this.steps = steps; }

    void run(SagaContext context) {
        List<SagaStep> completedSteps = new ArrayList<>(); // tracks what actually needs
        // undoing
        try {
            for (SagaStep step : steps) {
                step.execute(context);
            }
            completedSteps.add(step); // only added AFTER successful execute()
        } catch (SagaStepException e) {
            // roll back everything that succeeded, in REVERSE order
            for (int i = completedSteps.size() - 1; i >= 0; i--) {
                completedSteps.get(i).compensate(context);
            }
            throw new SagaFailedException("Saga failed and was compensated", e);
        }
    }
}

SagaOrchestrator orderSaga = new SagaOrchestrator(List.of(
    new ReserveInventoryStep(inventoryClient),
    new ChargePaymentStep(paymentClient),
    new ConfirmOrderStep(orderClient)
));
orderSaga.run(new SagaContext());

```

Walking through the code. Every `SagaStep` implements both `execute()` and its own `compensate()` — the class that knows how to reserve inventory is the same class that knows how to release that reservation, keeping the “do” and “undo” logic for each step co-located rather than scattered. `SagaOrchestrator.run()` walks `steps` in order, and — critically — only appends a step to `completedSteps` *after* its `execute()` call returns successfully; if `ChargePaymentStep.execute()` throws (say, the card is declined), `completedSteps` at that point contains only `ReserveInventoryStep`, not `ChargePaymentStep` itself, since that step never successfully completed. The `catch` block then iterates `completedSteps` **in reverse** (`i = completedSteps.size() - 1` down to `0`) and calls `compensate()` on each — reverse order matters because compensations should unwind in the opposite sequence they were applied, mirroring how you’d back out of a partially-completed physical process. `ReserveInventoryStep.compensate()` and `ChargePaymentStep.compensate()` both guard with `if (context.reservationId != null) /`

`if (context.paymentId != null)` — defensive checks in case `compensate()` is ever called on a step whose `execute()` didn't fully populate that field, though the orchestrator's own bookkeeping (only adding to `completedSteps` after success) should already prevent that from happening under normal operation. Note `ConfirmOrderStep.compensate()` doesn't literally reverse "confirm" back to some prior state — it explicitly marks the order `failed`, which is the correct compensating action for a step whose real-world effect (informing the customer) can't be silently undone.

Q238. What's the key difference between the orchestration Saga shown above and a choreography-based Saga, and what does the orchestrator become a risk of if overused? Orchestration (shown above) has one central coordinator (`SagaOrchestrator`) that explicitly calls each service and decides what happens next — it's easy to see the whole flow in one place, and equally easy to test in isolation, but that central coordinator now knows about (and directly depends on) every service involved, and it's a single component every new saga variant has to be built through. Choreography instead has each service publish an event when it completes its part (`InventoryReserved`, `PaymentCharged`), and each subsequent service *subscribes* to the event that should trigger its own action — with compensation handled the same way (a `PaymentFailed` event triggers the inventory service to subscribe and release its own reservation), so there's no central coordinator at all. The risk with orchestration, if it accumulates logic for many different multi-step business processes over time, is becoming a "god service" that every team's feature has to touch, re-centralizing the very coupling microservices were meant to avoid; choreography avoids that single-service coupling but makes the overall flow much harder to observe and reason about, since the business process is now implicitly encoded across the independent event-subscription logic of several separate services rather than visible in one orchestrator's code.

Circuit Breaker Pattern (Code-Level)

Q239. Implement a circuit breaker as a standalone class with the three states (closed, open, half-open), and explain the transition logic. A circuit breaker wraps calls to a potentially-failing downstream dependency: in the `CLOSED` state, calls pass through normally while failures are tracked; once failures cross a threshold, the breaker trips to `OPEN`, where calls fail immediately without even attempting the downstream call (protecting a struggling dependency from further load, and failing fast instead of making every caller wait out a timeout); after a cooldown period, the breaker moves to `HALF_OPEN`, allowing a small number of trial calls through to test whether the dependency has recovered — success moves back to `CLOSED`, failure moves back to `OPEN`.

```
enum CircuitState { CLOSED, OPEN, HALF_OPEN }

class CircuitBreaker {
    private volatile CircuitState state = CircuitState.CLOSED;
    private final int failureThreshold;
    private final Duration openDuration;
```

```

private final AtomicInteger consecutiveFailures = new AtomicInteger(0);
private volatile Instant openedAt;

CircuitBreaker(int failureThreshold, Duration openDuration) {
    this.failureThreshold = failureThreshold;
    this.openDuration = openDuration;
}

<T> T call(Supplier<T> downstreamCall, Supplier<T> fallback) {
    if (state == CircuitState.OPEN) {
        if (Instant.now().isAfter(openedAt.plus(openDuration))) {
            state = CircuitState.HALF_OPEN;           // cooldown elapsed – allow one
trial call through
        } else {
            return fallback.get();                     // still cooling down – fail
fast, don't even try
        }
    }

    try {
        T result = downstreamCall.get();
        onSuccess();
        return result;
    } catch (Exception e) {
        onFailure();
        return fallback.get();
    }
}

private void onSuccess() {
    consecutiveFailures.set(0);
    state =
CircuitState.CLOSED;                               // a successful HALF_OPEN trial call fully
close the breaker
}

private void onFailure() {
    int failures = consecutiveFailures.incrementAndGet();
    if (state == CircuitState.HALF_OPEN || failures >= failureThreshold) {
        state =
CircuitState.OPEN;                                // trip open – either a fresh threshold breach,
or a failed trial
        openedAt = Instant.now();
    }
}
}

CircuitBreaker breaker = new CircuitBreaker(5, Duration.ofSeconds(30));
String result = breaker.call(
    () -> recommendationServiceClient.getRecommendations(userId), // the real, possibly-
failing call
    () -> Collections.emptyList().toString()                       // the fallback –
degrade gracefully (Q72)
);

```

Walking through the code. `call()` first checks `state == CircuitState.OPEN` — if so, and the `openDuration` cooldown hasn't elapsed yet, it returns `fallback.get()` immediately, without ever invoking `downstreamCall`, which is the “fail fast” behavior: a struggling downstream service isn't sent any more traffic while the breaker is open, and the caller doesn't have to wait out a slow timeout to find that out. Once the

cooldown has elapsed, the state is flipped to `HALF_OPEN` and execution falls through to actually attempt the call — this is the one trial call the answer describes. `onSuccess()` unconditionally sets `state = CircuitState.CLOSED` and resets the failure counter — notably, this means a successful call while `HALF_OPEN` fully closes the breaker in one shot (no requirement for multiple consecutive successes in this simplified version, though production implementations often do require a few). `onFailure()` increments `consecutiveFailures` and trips to `OPEN` under either of two conditions: `state == CircuitState.HALF_OPEN` (the trial call itself failed, meaning the dependency clearly hasn't recovered — trip back open immediately, don't wait for the failure count to climb again) or `failures >= failureThreshold` (the normal closed-state path — enough consecutive failures in a row to conclude the dependency is actually down, not just having one transient blip). The `fallback` supplier passed into `call()` is what implements graceful degradation (Q72) at the call site — the caller decides what “acceptable degraded behavior” looks like for this specific call, whether that's an empty list, a cached stale value, or a default.

Outbox Pattern (Reliable Event Publishing)

Q240. When a service needs to both update its own database *and* publish an event about that change (e.g., “order created” — update the orders table, and publish an `OrderCreated` event for other services to consume), what's the correctness problem with just doing both operations back to back, and how does the outbox pattern solve it? Writing to the database and publishing to a message broker are two separate systems with no shared transaction — if the DB write succeeds but the process crashes (or the broker call fails) before publishing, the event is lost forever even though the order was actually created, and any downstream service relying on that event (inventory, notifications) never finds out. The outbox pattern solves this by writing the event into an `outbox` table in the *same local database transaction* as the actual business data change — since it's the same database and the same transaction, both either commit together or neither does, which is a guarantee the database's own transactional atomicity already gives you for free — and then a separate, independent process (a poller, or a change-data-capture stream reading the database's write-ahead log) reads unpublished rows from the outbox table and actually publishes them to the message broker, retrying until it succeeds.

```
class OrderService {
    private final DataSource dataSource;

    void createOrder(Order order) throws SQLException {
        try (Connection conn = dataSource.getConnection()) {
            conn.setAutoCommit(false);
            try {
                insertOrder(conn, order); // business data
                insertOutboxEvent(conn, "OrderCreated", order.toJson()); // event write -
            } catch (SQLException e) {
                conn.rollback(); // both succeed
            }
        }
    }
}
```

```

        conn.rollback();
        throw e;
    }
}

private void insertOrder(Connection conn, Order order) throws SQLException {
    PreparedStatement stmt = conn.prepareStatement(
        "INSERT INTO orders (id, customer_id, status) VALUES (?, ?, ?)");
    stmt.setString(1, order.id); stmt.setString(2, order.customerId); stmt.setString(3, "
    CREATED");
    stmt.executeUpdate();
}

private void insertOutboxEvent(Connection conn, String eventType, String payload) throws
SQLException {
    PreparedStatement stmt = conn.prepareStatement(
        "INSERT INTO outbox (id, event_type, payload, published) VALUES (?, ?, ?,
    false)");
    stmt.setString(1, UUID.randomUUID().toString());
    stmt.setString(2, eventType);
    stmt.setString(3, payload);
    stmt.executeUpdate();
}

class OutboxPoller implements Runnable {                                // runs as an independent,
    separate process/thread
    private final DataSource dataSource;
    private final MessageBroker broker;

    public void run() {
        while (true) {
            List<OutboxRow> pending = fetchUnpublishedRows();    // WHERE published = false,
            e.g. LIMIT 100
            for (OutboxRow row : pending) {
                try {
                    broker.publish(row.eventType, row.payload);
                    markPublished(row.id);                        // only mark done AFTER
                    the broker confirms receipt
                } catch (Exception e) {
                    // leave it unpublished – the next poll iteration retries it
                    automatically
                }
            }
            sleep(Duration.ofSeconds(1));
        }
    }

    private List<OutboxRow> fetchUnpublishedRows() { return List.of(); }
    private void markPublished(String id) { /* UPDATE outbox SET published = true WHERE id
    = ? */ }
    private void sleep(Duration d) { /* Thread.sleep */ }
}

```

Walking through the code. `createOrder()` explicitly sets `conn.setAutoCommit(false)` and wraps both `insertOrder()` and `insertOutboxEvent()` inside the same `try / conn.commit() / catch / conn.rollback()` block using the *same* `Connection` object — this is what guarantees they're part of one atomic database transaction. If `insertOrder()` succeeds but `insertOutboxEvent()` somehow fails, the `catch` block rolls back the *entire*

transaction, undoing the order insert too — there is no possible outcome where the order exists in the database but no corresponding outbox row does, or vice versa. Notice what `createOrder()` does *not* do: it never calls the message broker directly. `OutboxPoller` is a completely separate, independent loop (its own thread or process) that repeatedly queries for `published = false` rows, attempts to `broker.publish(...)` each one, and only calls `markPublished(row.id)` — flipping that row’s flag — *after* the broker confirms the message was accepted. If the broker call throws, the `catch` block in the poller’s loop deliberately does nothing except let that row remain `published = false`; the next iteration of the `while (true)` loop will simply pick it up again and retry, with no special retry-tracking logic needed beyond “the flag is still false.” This decouples “did the business data change happen” (guaranteed atomic via the local DB transaction) from “did the event definitely reach the broker” (guaranteed eventually, via indefinite retry from the poller) — two different reliability guarantees that would otherwise require a much more complex distributed transaction (like two-phase commit) to achieve together in one step.

Service Mesh Internals

Q241. Concretely, what does a sidecar proxy in a service mesh (like Istio’s Envoy) actually intercept and do, that the application code itself doesn’t have to? Every outbound and inbound network call to/from the service is transparently routed through the sidecar proxy running alongside it (typically via iptables rules redirecting traffic at the pod/container network level, so application code doesn’t need to be aware this is happening or change how it makes calls). The sidecar then handles, entirely outside application code: mutual TLS encryption/decryption between services (the app makes a plain HTTP call; the sidecar upgrades it to mTLS on the wire and the receiving sidecar decrypts it before handing plain HTTP to the receiving app); retries with backoff on transient failures; circuit breaking (the mesh-level equivalent of Q239, configured declaratively rather than coded into every service); fine-grained traffic splitting for canary deployments (route 5% of traffic to a new service version based purely on proxy configuration, no application-level feature-flagging code needed); and automatic collection of request-level metrics/tracing spans for every single service-to-service call, since every call already passes through this uniform interception point. The trade-off: added latency per hop (every call now goes through two proxies, sender’s and receiver’s), and genuinely significant operational complexity in running and correctly configuring the mesh control plane itself.

Contract Testing

Q242. What problem does consumer-driven contract testing (e.g., Pact) solve that end-to-end integration tests don’t, in a microservices architecture with many independently-deployed services? End-to-end tests that spin up multiple real services together are slow, flaky (any one service being briefly unhealthy fails the whole test run), and — most importantly for this specific problem — they only catch a breaking API change *after* it’s already been made, typically in a shared staging

environment where the blame and the fix are both hard to localize. Consumer-driven contract testing inverts this: each consumer of a service (e.g., the mobile app team consuming the Order Service’s API) writes a “contract” describing exactly what fields/shape it expects from specific API calls, from its own test suite, with no real Order Service running at all — those contracts are published to a shared broker. The **provider** (Order Service) then, as part of *its own* CI pipeline, runs those same published contracts against its actual code — verifying it still satisfies every consumer’s expectations — which means a breaking change is caught in the provider’s own build, before it’s ever deployed, and the failure message points directly at exactly which consumer’s exact expectation broke, rather than a vague end-to-end test failure days into a shared staging environment’s life.

```
// Consumer side (e.g., the mobile app's test suite) – defines the CONTRACT, no real Order
Service involved
@Pact(consumer = "MobileApp", provider = "OrderService")
RequestResponsePact getOrderContract(PactDslWithProvider builder) {
    return builder
        .given("order 123 exists") // the STATE the
        provider must set up to satisfy this
        .uponReceiving("a request for order 123")
        .path("/orders/123")
        .method("GET")
        .willRespondWith()
        .status(200)
        .body(new PactDslJsonBody()
            .stringType("id", "123")
            .stringType("status") // consumer only asserts the SHAPE it actually
            uses – not every field
            .numberType("totalAmount"))
        .toPact();
}

// Provider side (Order Service's OWN CI pipeline) – verifies its real code against every
published consumer contract
@Provider("OrderService")
@PactBroker(url = "https://pact-broker.internal")
class OrderServiceContractTest {
    @State("order 123 exists")
    void setupOrder123() {
        orderRepository.save(new Order("123", "CREATED", 49.99)); // provider sets up the
        state the contract expects
    }

    @TestTemplate
    @ExtendWith(PactVerificationInvocationContextProvider.class)
    void verifyContract(PactVerificationContext context) {
        context.verifyInteraction(); // actually calls the real OrderService code and
        checks the response matches
    }
}
```

Walking through the code. The consumer-side `getOrderContract()` method doesn’t call any real Order Service — it declaratively describes an expected interaction: given a precondition (`"order 123 exists"`), a request (`GET /orders/123`), Pact will mock out a fake Order Service that returns exactly the response shape defined in `.body(...)` , and

the mobile app's own test suite runs against that mock. Critically, `.body(...)` uses `stringType(...)` / `numberType(...)` matchers rather than exact hardcoded values — the consumer is asserting “this field must exist and must be a string,” not “this field must equal 123 forever,” which is what lets the provider's actual data change without breaking the contract, so long as the shape the consumer depends on is preserved. This generated contract file is what gets published to the shared Pact Broker. On the provider side, `OrderServiceContractTest` runs as part of Order Service's own build — `@State("order 123 exists")` maps directly to the same string the consumer's contract declared as its precondition, and `setupOrder123()` is the provider-side code that actually creates that state in a real (test) database. `context.verifyInteraction()` then makes the actual HTTP call against the real running Order Service code and checks the real response against what the consumer's contract expects. If a developer later removes the `totalAmount` field from the real API response, this test fails immediately in Order Service's own CI run — the failure message identifies exactly which consumer (`MobileApp`) and which interaction broke, well before that change could ever reach a shared staging environment where a generic end-to-end test would eventually (and far more vaguely) also have caught it.

Idempotency in a Microservices Context

Q243. Given that network calls between services can fail in ways where the caller doesn't know if the request actually succeeded on the other end (timeout with no response received), how do you make a service's write endpoints safe to blindly retry? The caller, on a timeout, cannot distinguish “the request never reached the server” from “the request succeeded but the response was lost” — so a naive retry risks double-processing (charging a customer twice, creating two orders for one checkout click). The fix is requiring the caller to generate and attach a unique idempotency key to the request (typically a UUID generated once per logical operation, reused across all retries of that same operation), and having the receiving service check that key against a store of already-processed keys *before* doing any real work — if the key has been seen before, it returns the *original* stored response immediately without re-executing the operation at all.

```
class IdempotencyStore {
    private final Map<String, StoredResponse> processedKeys = new ConcurrentHashMap<>();

    static class StoredResponse {
        int statusCode;
        String body;
    }

    Optional<StoredResponse> get(String idempotencyKey) {
        return Optional.ofNullable(processedKeys.get(idempotencyKey));
    }

    void store(String idempotencyKey, StoredResponse response) {
        processedKeys.put(idempotencyKey, response);
    }
}
```

```

class PaymentController {
    private final IdempotencyStore idempotencyStore;
    private final PaymentProcessor paymentProcessor;

    Response chargeCard(String idempotencyKey, ChargeRequest request) {
        Optional<IdempotencyStore.StoredResponse> existing = idempotencyStore.get(idempotency
Key);
        if (existing.isPresent()) {
            return Response.of(existing.get().statusCode, existing.get().body); // already
handled – return the SAME result, don't re-charge
        }

        // NOTE: in production this lookup-then-store sequence itself needs to be atomic
(e.g. a DB unique
// constraint on idempotencyKey, or a distributed lock) to correctly handle two
retries arriving
// near-simultaneously – omitted here for clarity, but this is the same class of race
as Q194/Q216.
        ChargeResult result = paymentProcessor.charge(request);
        IdempotencyStore.StoredResponse toStore = new IdempotencyStore.StoredResponse();
        toStore.statusCode = 200;
        toStore.body = result.toJson();
        idempotencyStore.store(idempotencyKey, toStore);
        return Response.of(200, toStore.body);
    }
}

```

Walking through the code. The client is responsible for generating `idempotencyKey` exactly once per logical charge attempt (typically a UUID created client-side before the first attempt) and sending that *same* key on every retry of that specific charge — a genuinely new charge (a different purchase) must use a new key. `chargeCard()` checks `idempotencyStore.get(idempotencyKey)` as its very first action: if a response is already stored under this key, it's returned immediately (`Response.of(existing.get().statusCode, existing.get().body)`) and `paymentProcessor.charge(request)` — the actual money-moving call — is never invoked a second time. Only when the key is genuinely new does the real charge happen, followed by storing the response under that key before returning it, so that a subsequent retry with the identical key will find it on its own lookup. The inline comment flags an important omission: as written, there's a race window between the `get()` check and the later `store()` call — if two retries of the same request arrive close enough together, both could pass the initial `get()` check before either has stored a result, and both could end up calling `paymentProcessor.charge()`. A correct production implementation closes this the same way Q194 and Q216 closed their own concurrent-access gaps: a single atomic check-and-reserve operation (a database unique constraint on `idempotencyKey` that the second concurrent insert would violate, or a lock held across the whole check-then-charge-then-store sequence) rather than the two separate, non-atomic steps shown here for clarity.